Experiment 1: Data Preprocessing and Cleaning — Explanation of Each Step

Name: S. Pargavi

Register Number: 241801196

Subject: AD23532 – Principles of Data Science

Step 1: Import the required libraries

Code:

Python

Run

```
None
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

Explanation:

- `pandas (pd)`: Used to load, organize, and manipulate the dataset in table format.
- `numpy (np)`: Used for numerical operations and handling missing numerical values.
- `matplotlib.pyplot (plt)`: Used to create graphs and plots.

- `seaborn (sns)`: Used to create statistical visualizations such as histograms, count plots, box plots, and heatmaps.
- `LabelEncoder`: Converts categorical values into numerical labels.
- `StandardScaler`: Standardizes numerical features to have a mean close to 0 and a standard deviation close to 1.

Output:

```
None
Step 1: Libraries imported successfully
```

Step 2: Load and display the dataset

Code:

Python

Run

```
None
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

print(df.head(10))
print(df.tail(10))
print(df.shape)
print(df.columns.tolist())
```

Explanation:

- `pd.read_csv()`: Loads the CSV file into a Pandas DataFrame named `df`.
- `df.head(10)`: Displays the first 10 records.
- `df.tail(10)`: Displays the last 10 records.
- `df.shape`: Returns the number of rows and columns.
- `df.columns.tolist()`: Displays all column names in a list.

Expected output example:

```
None
Shape of dataset: (7043, 21)
```

The commonly distributed Telco Customer Churn dataset contains 7,043 customer records and 21 columns. Your result may differ if your dataset is different.

Step 3: Explore the dataset

Code:

Python

Run

```
None
df.info()
print(df.describe(include="all").T)
print(df.isnull().sum())
print(df.duplicated().sum())
```

Explanation:

- `df.info()`: Displays column names, data types, and non-null counts.
- `df.describe(include="all").T`: Displays summary statistics for numerical and categorical columns. `.T` transposes the table for easier reading.
- `df.isnull().sum()`: Counts missing values in each column.
- `df.duplicated().sum()`: Counts duplicate rows in the dataset.

Expected output example:

```
None
Missing values
customerID       0
gender           0
SeniorCitizen    0
...
```

The actual missing-value and duplicate counts depend on the CSV file.

Step 4: Clean the dataset

Code:

Python

Run

```
None
df = df.drop_duplicates().copy()

df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"], errors="coerce"
)

numeric_cols = df.select_dtypes(include=np.number).columns

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

categorical_cols = df.select_dtypes(
    include=["object", "category", "bool"]
).columns

for col in categorical_cols:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].mode()[0])

print(df.isnull().sum())
print(df.isnull().sum().sum())
```

Explanation:

1. `drop_duplicates()`: Removes duplicate records.
2. `pd.to_numeric()`: Converts `TotalCharges` into numeric data. Invalid or blank entries become missing values (`NaN`).
3. `select_dtypes(include=np.number)`: Selects numerical columns.
4. `median()`: Calculates the middle value of a numerical column.
5. `fillna()`: Replaces missing numerical values with their column median.
6. `mode()[0]`: Finds the most frequent value in a categorical column.
7. The final statements verify whether missing values remain.

Expected output:

```
None
Total missing values remaining: 0
```

This is expected if all missing values can be filled successfully.

Step 5: Encode categorical columns

Code:

Python

Run

```
None
columns_to_encode = [
    "gender", "Partner", "Dependents", "Contract",
    "PaymentMethod", "InternetService", "Churn"
]

encoders = {}

for col in columns_to_encode:
    if col in df.columns:
        encoder = LabelEncoder()
        df[col] = encoder.fit_transform(df[col].astype(str))
        encoders[col] = encoder
        print(f"{col} encoded successfully")
```

Explanation:

Categorical columns contain text values. Machine-learning algorithms often require numerical input, so `LabelEncoder` assigns an integer to each distinct category.

For example, a binary column might be encoded as:

Original value	Encoded value
No	0

Yes 1

The exact mapping depends on the categories and the encoder. The code stores each fitted encoder in `encoders` so its mapping can be reused.

Expected output:

```
None
gender encoded successfully
Partner encoded successfully
Dependents encoded successfully
Contract encoded successfully
PaymentMethod encoded successfully
InternetService encoded successfully
Churn encoded successfully
```

Step 6: Standardize numerical features

Code:

Python

Run

```
None
scale_cols = [
    "tenure", "MonthlyCharges", "TotalCharges"
]

scaler = StandardScaler()
df[scale_cols] = scaler.fit_transform(df[scale_cols])

print(df[scale_cols].head())
print(df[scale_cols].mean().round(3))
print(df[scale_cols].std(ddof=0).round(3))
```

Explanation:

- `scale_cols`: Specifies the three numerical features to standardize.

- `StandardScaler()`: Creates the standardization object.
- `fit_transform()`: Calculates each feature's mean and standard deviation and transforms its values.
- `mean()`: Checks whether the standardized means are approximately 0.
- `std(ddof=0)`: Checks whether the population standard deviations are approximately 1.

Standardization uses the formula:

z=x−μσz=\frac{x-\mu}{\sigma}z=σx−μ

Where:

- xxx = original value
- μ\muμ = feature mean
- σ\sigmaσ = feature standard deviation
- zzz = standardized value

Expected output example:

```
None
Means after standardization:
tenure            0.000
MonthlyCharges    0.000
TotalCharges      0.000
```

The values are approximately zero for the means and one for the population standard deviations, allowing for rounding.

Step 7: Plot the distributions of MonthlyCharges and Tenure

Code:

Python

Run

```
None
plt.figure(figsize=(12, 4))
```

```
plt.subplot(1, 2, 1)
sns.histplot(df["MonthlyCharges"], kde=True)
plt.title("Distribution of MonthlyCharges")

plt.subplot(1, 2, 2)
sns.histplot(df["tenure"], kde=True)
plt.title("Distribution of Tenure")

plt.tight_layout()
plt.show()
```

Explanation:

- `plt.figure(figsize=(12, 4))`: Sets the overall figure size.
- `plt.subplot(1, 2, 1)`: Creates the first plot in a layout of one row and two columns.
- `sns.histplot()`: Displays the distribution of values as bars.
- `kde=True`: Adds a smooth density curve.
- `plt.title()`: Adds a title to each graph.
- `plt.tight_layout()`: Adjusts spacing between the plots.
- `plt.show()`: Displays the graphs.

Output: Two histograms appear: one for MonthlyCharges and one for Tenure.

Step 8: Create a Contract vs Churn count plot

Code:

Python

Run

```
None
plt.figure(figsize=(8, 5))

sns.countplot(data=df, x="Contract", hue="Churn")

plt.title("Contract vs Churn")
```

```
plt.xlabel("Contract Type")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
```

Explanation:

- `countplot()`: Counts the number of records in each contract category.
- `x="Contract"`: Places contract types on the horizontal axis.
- `hue="Churn"`: Separates the bars by churn category.
- `xlabel()` and `ylabel()`: Label the axes.
- `show()`: Displays the chart.

Output: A bar chart comparing the number of customers who churned and did not churn across contract types. Because Churn was encoded in Step 5, its legend uses numerical labels.

Step 9: Draw a box plot of MonthlyCharges grouped by Churn

Code:

Python

Run

```
None
plt.figure(figsize=(8, 5))

sns.boxplot(data=df, x="Churn", y="MonthlyCharges")

plt.title("MonthlyCharges Grouped by Churn")
plt.xlabel("Churn (0 = No, 1 = Yes)")
plt.ylabel("Standardized MonthlyCharges")
plt.tight_layout()
plt.show()
```

Explanation:

- `sns.boxplot()`: Creates a box plot to compare distributions.
- `x="Churn"`: Separates customers by churn category.
- `y="MonthlyCharges"`: Displays standardized monthly charges.
- The line inside each box represents the median.
- The box represents the interquartile range.
- The whiskers and individual points help show spread and potential outliers.

Output: A box plot showing the distribution of MonthlyCharges for each churn category.

Step 10: Display the correlation heatmap

Code:

Python

Run

```
None
plt.figure(figsize=(14, 10))

numeric_df = df.select_dtypes(include=np.number)

sns.heatmap(
    numeric_df.corr(),
    cmap="coolwarm",
    center=0,
    annot=False
)

plt.title("Correlation Heatmap of Numerical Features")
plt.tight_layout()
plt.show()
```

Explanation:

- `select_dtypes(include=np.number)`: Selects numerical columns.
- `corr()`: Calculates pairwise correlations between numerical features.
- `sns.heatmap()`: Displays the correlation matrix as a color-coded grid.
- `cmap="coolwarm"`: Uses contrasting colors to represent negative and positive correlations.

- `center=0`: Centers the color scale at zero.
- `annot=False`: Does not print the numerical correlation value inside every cell.

Interpretation:

- Correlation near `+1`: Strong positive linear relationship.
- Correlation near `-1`: Strong negative linear relationship.
- Correlation near `0`: Weak linear relationship.

Encoded categorical columns are included among the numeric columns, so their correlations must be interpreted carefully.

Step 11: Save the preprocessed dataset

Code:

Python

Run

```
None
df.to_csv("telco_churn_preprocessed.csv", index=False)

print("Preprocessed dataset saved successfully")
print("Final dataset shape:", df.shape)
```

Explanation:

- `to_csv()`: Saves the cleaned DataFrame as a CSV file.
- `index=False`: Prevents Pandas from adding a separate index column.
- `df.shape`: Displays the final number of rows and columns.

Output:

```
None
Preprocessed dataset saved successfully
Final dataset shape: (7043, 21)
```

Full code:

```python
#S.pargavi#241801196

# STEP 1: Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler

print("Step 1: Libraries imported successfully")


# STEP 2: Load the dataset
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

print("\nStep 2: First 10 records")
print(df.head(10))

print("\nLast 10 records")
print(df.tail(10))

print("\nShape of dataset:", df.shape)

print("\nColumn names:")
print(df.columns.tolist())


# STEP 3: Explore the dataset
print("\nStep 3: Dataset information")
df.info()

print("\nSummary statistics")
print(df.describe(include="all").T)

print("\nMissing values")
print(df.isnull().sum())

print("\nDuplicate records:", df.duplicated().sum())
```

```python
# STEP 4: Clean the dataset
df = df.drop_duplicates().copy()

# Convert TotalCharges into numeric values
df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"], errors="coerce"
)

# Fill missing numerical values with median
numeric_cols = df.select_dtypes(include=np.number).columns

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

# Fill missing categorical values with mode
categorical_cols = df.select_dtypes(
    include=["object", "category", "bool"]
).columns

for col in categorical_cols:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].mode()[0])

print("\nStep 4: Missing values after cleaning")
print(df.isnull().sum())

print("Total missing values remaining:",
      df.isnull().sum().sum())


# STEP 5: Encode categorical columns
columns_to_encode = [
    "gender",
    "Partner",
    "Dependents",
    "Contract",
    "PaymentMethod",
    "InternetService",
    "Churn"
]
```

```
encoders = {}

for col in columns_to_encode:
    if col in df.columns:
        encoder = LabelEncoder()
        df[col] = encoder.fit_transform(df[col].astype(str))
        encoders[col] = encoder
        print(f"{col} encoded successfully")

print("\nEncoded dataset preview")
print(df.head())


# STEP 6: Standardize numerical features
scale_cols = [
    "tenure",
    "MonthlyCharges",
    "TotalCharges"
]

scaler = StandardScaler()
df[scale_cols] = scaler.fit_transform(df[scale_cols])

print("\nStep 6: Standardized numerical features")
print(df[scale_cols].head())

print("\nFeature means (approximately 0)")
print(df[scale_cols].mean().round(3))

print("\nFeature standard deviations (approximately 1)")
print(df[scale_cols].std(ddof=0).round(3))


# STEP 7: Distribution plots
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
sns.histplot(df["MonthlyCharges"], kde=True)
plt.title("Distribution of MonthlyCharges")
```

```
plt.xlabel("Standardized MonthlyCharges")

plt.subplot(1, 2, 2)
sns.histplot(df["tenure"], kde=True)
plt.title("Distribution of Tenure")
plt.xlabel("Standardized Tenure")

plt.tight_layout()
plt.show()

print("Step 7: Distribution plots displayed")


# STEP 8: Contract vs Churn count plot
plt.figure(figsize=(8, 5))

sns.countplot(data=df, x="Contract", hue="Churn")

plt.title("Contract vs Churn")
plt.xlabel("Contract Type")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()

print("Step 8: Contract vs Churn plot displayed")


# STEP 9: Box plot of MonthlyCharges grouped by Churn
plt.figure(figsize=(8, 5))

sns.boxplot(data=df, x="Churn", y="MonthlyCharges")

plt.title("MonthlyCharges Grouped by Churn")
plt.xlabel("Churn (0 = No, 1 = Yes)")
plt.ylabel("Standardized MonthlyCharges")
plt.tight_layout()
plt.show()

print("Step 9: Box plot displayed")
```

```
# STEP 10: Correlation heatmap
plt.figure(figsize=(14, 10))

numeric_df = df.select_dtypes(include=np.number)

sns.heatmap(
    numeric_df.corr(),
    cmap="coolwarm",
    center=0,
    annot=False
)

plt.title("Correlation Heatmap of Numerical Features")
plt.tight_layout()
plt.show()

print("Step 10: Correlation heatmap displayed")


# STEP 11: Save the preprocessed dataset
df.to_csv("telco_churn_preprocessed.csv", index=False)

print("\nPreprocessed dataset saved successfully")
print("Final dataset shape:", df.shape)
```

Output:

Step 1: Libraries imported successfully

Step 2: First 10 records

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	\
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	
4	9237-HQITU	Female	0	No	No	2	Yes	
5	9305-CDSKC	Female	0	No	No	8	Yes	
6	1452-KIOVK	Male	0	No	Yes	22	Yes	
7	6713-OKOMC	Female	0	No	No	10	No	

```
8  7892-POOKP  Female              0     Yes         No      28          Yes
9  6388-TABGU    Male              0      No        Yes      62          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service           DSL            No  ...               No
1                No           DSL           Yes  ...              Yes
2                No           DSL           Yes  ...               No
3  No phone service           DSL           Yes  ...              Yes
4                No     Fiber optic           No  ...               No
5               Yes     Fiber optic           No  ...              Yes
6               Yes     Fiber optic           No  ...               No
7  No phone service           DSL           Yes  ...              No
8               Yes     Fiber optic           No  ...              Yes
9                No           DSL           Yes  ...               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year             No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year             No
4          No          No              No  Month-to-month              Yes
5          No         Yes             Yes  Month-to-month              Yes
6          No         Yes              No  Month-to-month              Yes
7          No          No              No  Month-to-month               No
8         Yes         Yes             Yes  Month-to-month              Yes
9          No          No              No        One year             No

              PaymentMethod MonthlyCharges  TotalCharges Churn
0          Electronic check          29.85         29.85    No
1              Mailed check          56.95        1889.5    No
2              Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4          Electronic check          70.70        151.65   Yes
5          Electronic check          99.65         820.5   Yes
6    Credit card (automatic)          89.10        1949.4    No
7              Mailed check          29.75         301.9    No
8          Electronic check         104.80       3046.05   Yes
9  Bank transfer (automatic)          56.15       3487.95    No

[10 rows x 21 columns]

Last 10 records
      customerID  gender  SeniorCitizen Partner Dependents  tenure  \
7033  9767-FFLEM    Male              0      No         No      38
7034  0639-TSIQW  Female              0      No         No      67
7035  8456-QDAVC    Male              0      No         No      19
```

```
7036  7750-EYXWZ  Female              0      No         No      12
7037  2569-WGERO  Female              0      No         No      72
7038  6840-RESVB    Male              0     Yes        Yes      24
7039  2234-XADUH  Female              0     Yes        Yes      72
7040  4801-JZAZL  Female              0     Yes        Yes      11
7041  8361-LTMKD    Male              1     Yes         No       4
7042  3186-AJIEK    Male              0      No         No      66

     PhoneService     MultipleLines InternetService       OnlineSecurity  ... \
7033          Yes                No     Fiber optic                   No  ...
7034          Yes               Yes     Fiber optic                  Yes  ...
7035          Yes                No     Fiber optic                   No  ...
7036           No  No phone service             DSL                   No  ...
7037          Yes                No              No  No internet service  ...
7038          Yes               Yes             DSL                  Yes  ...
7039          Yes               Yes     Fiber optic                   No  ...
7040           No  No phone service             DSL                  Yes  ...
7041          Yes               Yes     Fiber optic                   No  ...
7042          Yes                No     Fiber optic                  Yes  ...

       DeviceProtection          TechSupport          StreamingTV  \
7033                 No                   No                   No
7034                Yes                   No                  Yes
7035                 No                   No                  Yes
7036                Yes                  Yes                  Yes
7037  No internet service  No internet service  No internet service
7038                Yes                  Yes                  Yes
7039                Yes                   No                  Yes
7040                 No                   No                   No
7041                 No                   No                   No
7042                Yes                  Yes                  Yes

        StreamingMovies        Contract PaperlessBilling  \
7033                 No  Month-to-month              Yes
7034                 No  Month-to-month              Yes
7035                 No  Month-to-month              Yes
7036                Yes        One year               No
7037  No internet service        Two year              Yes
7038                Yes        One year              Yes
7039                Yes        One year              Yes
7040                 No  Month-to-month              Yes
7041                 No  Month-to-month              Yes
7042                Yes        Two year              Yes

              PaymentMethod MonthlyCharges  TotalCharges Churn
7033  Credit card (automatic)         69.50       2625.25    No
```

```
7034    Credit card (automatic)         102.95       6886.25   Yes
7035  Bank transfer (automatic)          78.70        1495.1    No
7036           Electronic check          60.65         743.3    No
7037  Bank transfer (automatic)          21.15        1419.4    No
7038               Mailed check          84.80        1990.5    No
7039    Credit card (automatic)         103.20        7362.9    No
7040           Electronic check          29.60        346.45    No
7041               Mailed check          74.40         306.6   Yes
7042  Bank transfer (automatic)         105.65        6844.5    No

[10 rows x 21 columns]

Shape of dataset: (7043, 21)

Column names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService',
'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges',
'TotalCharges', 'Churn']

Step 3: Dataset information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
```

```
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

Summary statistics

	count	unique	top	freq	mean	std
customerID	7043	7043	3186-AJIEK	1	NaN	NaN
gender	7043	2	Male	3555	NaN	NaN
SeniorCitizen	7043.0	NaN	NaN	NaN	0.162147	0.368612
Partner	7043	2	No	3641	NaN	NaN
Dependents	7043	2	No	4933	NaN	NaN
tenure	7043.0	NaN	NaN	NaN	32.371149	24.559481
PhoneService	7043	2	Yes	6361	NaN	NaN
MultipleLines	7043	3	No	3390	NaN	NaN
InternetService	7043	3	Fiber optic	3096	NaN	NaN
OnlineSecurity	7043	3	No	3498	NaN	NaN
OnlineBackup	7043	3	No	3088	NaN	NaN
DeviceProtection	7043	3	No	3095	NaN	NaN
TechSupport	7043	3	No	3473	NaN	NaN
StreamingTV	7043	3	No	2810	NaN	NaN
StreamingMovies	7043	3	No	2785	NaN	NaN
Contract	7043	3	Month-to-month	3875	NaN	NaN
PaperlessBilling	7043	2	Yes	4171	NaN	NaN
PaymentMethod	7043	4	Electronic check	2365	NaN	NaN
MonthlyCharges	7043.0	NaN	NaN	NaN	64.761692	30.090047
TotalCharges	7043	6531		11	NaN	NaN
Churn	7043	2	No	5174	NaN	NaN

	min	25%	50%	75%	max
customerID	NaN	NaN	NaN	NaN	NaN
gender	NaN	NaN	NaN	NaN	NaN
SeniorCitizen	0.0	0.0	0.0	0.0	1.0
Partner	NaN	NaN	NaN	NaN	NaN
Dependents	NaN	NaN	NaN	NaN	NaN
tenure	0.0	9.0	29.0	55.0	72.0
PhoneService	NaN	NaN	NaN	NaN	NaN
MultipleLines	NaN	NaN	NaN	NaN	NaN
InternetService	NaN	NaN	NaN	NaN	NaN
OnlineSecurity	NaN	NaN	NaN	NaN	NaN
OnlineBackup	NaN	NaN	NaN	NaN	NaN
DeviceProtection	NaN	NaN	NaN	NaN	NaN
TechSupport	NaN	NaN	NaN	NaN	NaN
StreamingTV	NaN	NaN	NaN	NaN	NaN
StreamingMovies	NaN	NaN	NaN	NaN	NaN
Contract	NaN	NaN	NaN	NaN	NaN
PaperlessBilling	NaN	NaN	NaN	NaN	NaN

```
PaymentMethod       NaN   NaN    NaN    NaN     NaN
MonthlyCharges    18.25  35.5  70.35  89.85  118.75
TotalCharges        NaN   NaN    NaN    NaN     NaN
Churn               NaN   NaN    NaN    NaN     NaN

Missing values
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Duplicate records: 0

Step 4: Missing values after cleaning
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
```

```
StreamingMovies      0
Contract             0
PaperlessBilling     0
PaymentMethod        0
MonthlyCharges       0
TotalCharges         0
Churn                0
dtype: int64
Total missing values remaining: 0
gender encoded successfully
Partner encoded successfully
Dependents encoded successfully
Contract encoded successfully
PaymentMethod encoded successfully
InternetService encoded successfully
Churn encoded successfully

Encoded dataset preview
   customerID  gender  SeniorCitizen  Partner  Dependents  tenure  \
0  7590-VHVEG       0              0        1           0       1
1  5575-GNVDE       1              0        0           0      34
2  3668-QPYBK       1              0        0           0       2
3  7795-CFOCW       1              0        0           0      45
4  9237-HQITU       0              0        0           0       2

  PhoneService     MultipleLines  InternetService OnlineSecurity  ...  \
0           No  No phone service                0             No  ...
1          Yes                No                0            Yes  ...
2          Yes                No                0            Yes  ...
3           No  No phone service                0            Yes  ...
4          Yes                No                1             No  ...

  DeviceProtection TechSupport StreamingTV StreamingMovies Contract  \
0               No          No          No              No        0
1              Yes          No          No              No        1
2               No          No          No              No        0
3              Yes         Yes          No              No        1
4               No          No          No              No        0

   PaperlessBilling PaymentMethod  MonthlyCharges  TotalCharges  Churn
0               Yes             2           29.85         29.85      0
1                No             3           56.95       1889.50      0
2               Yes             3           53.85        108.15      1
3                No             0           42.30       1840.75      0
4               Yes             2           70.70        151.65      1
```

[5 rows x 21 columns]

Step 6: Standardized numerical features

	tenure	MonthlyCharges	TotalCharges
0	-1.277445	-1.160323	-0.994242
1	0.066327	-0.259629	-0.173244
2	-1.236724	-0.362660	-0.959674
3	0.514251	-0.746535	-0.194766
4	-1.236724	0.197365	-0.940470

Feature means (approximately 0)

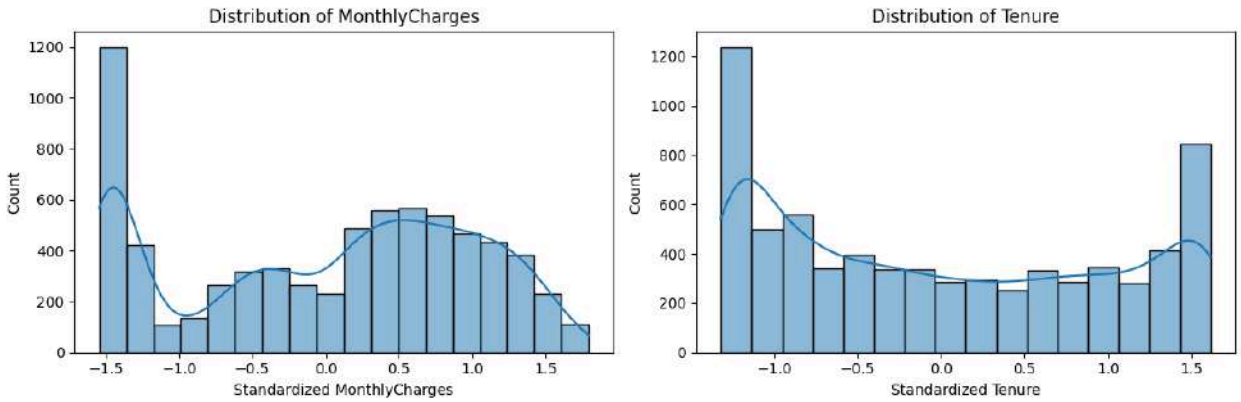
tenure	-0.0
MonthlyCharges	-0.0
TotalCharges	-0.0

dtype: float64

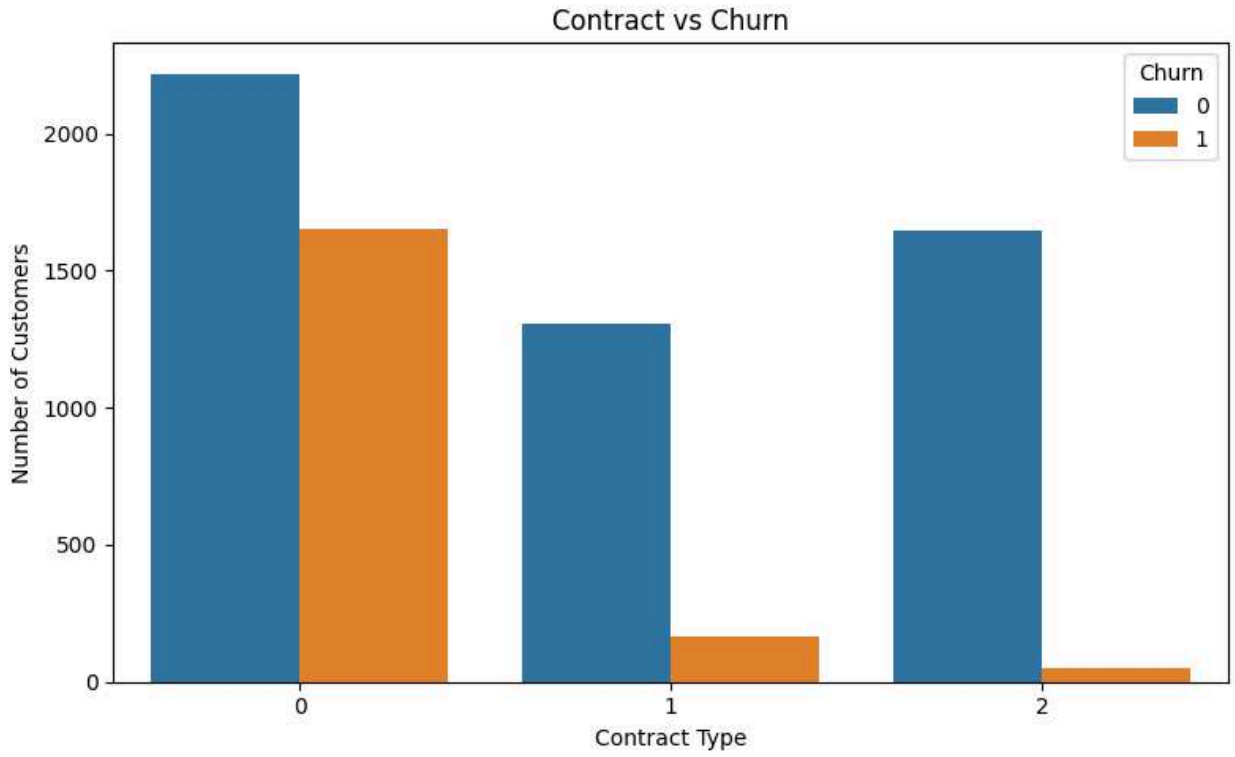
Feature standard deviations (approximately 1)

tenure	1.0
MonthlyCharges	1.0
TotalCharges	1.0

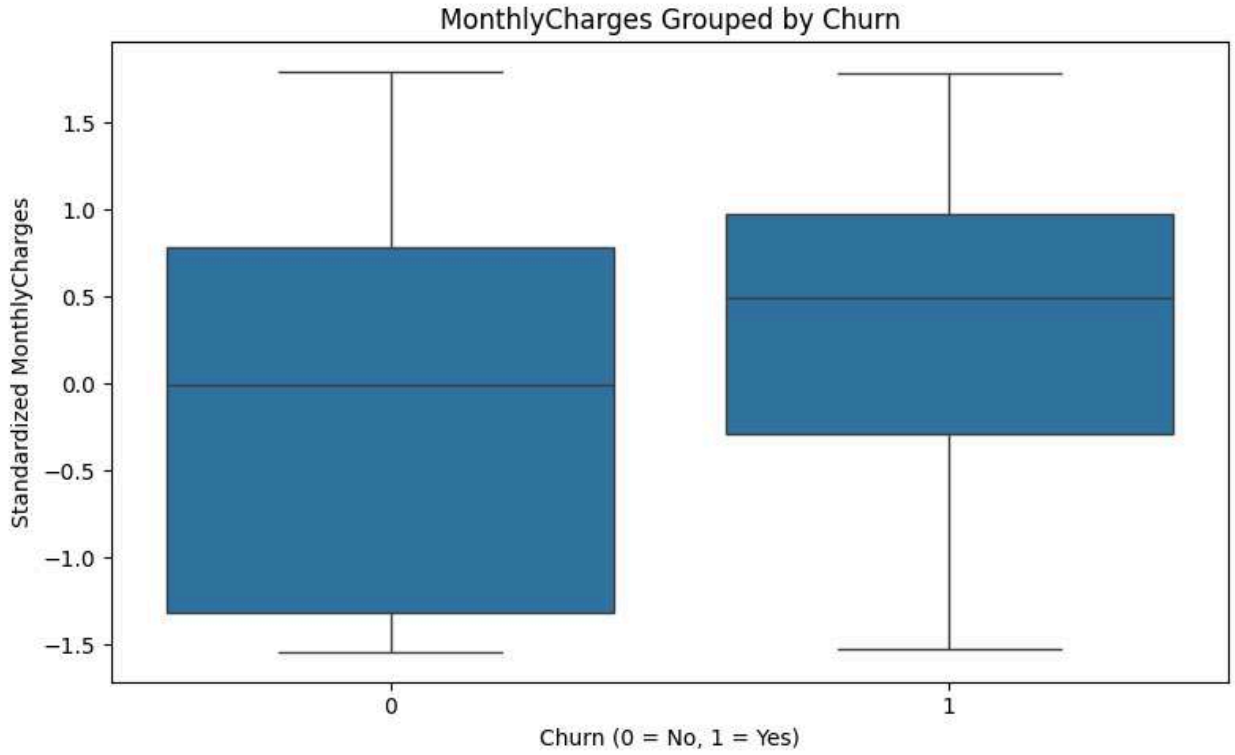
dtype: float64



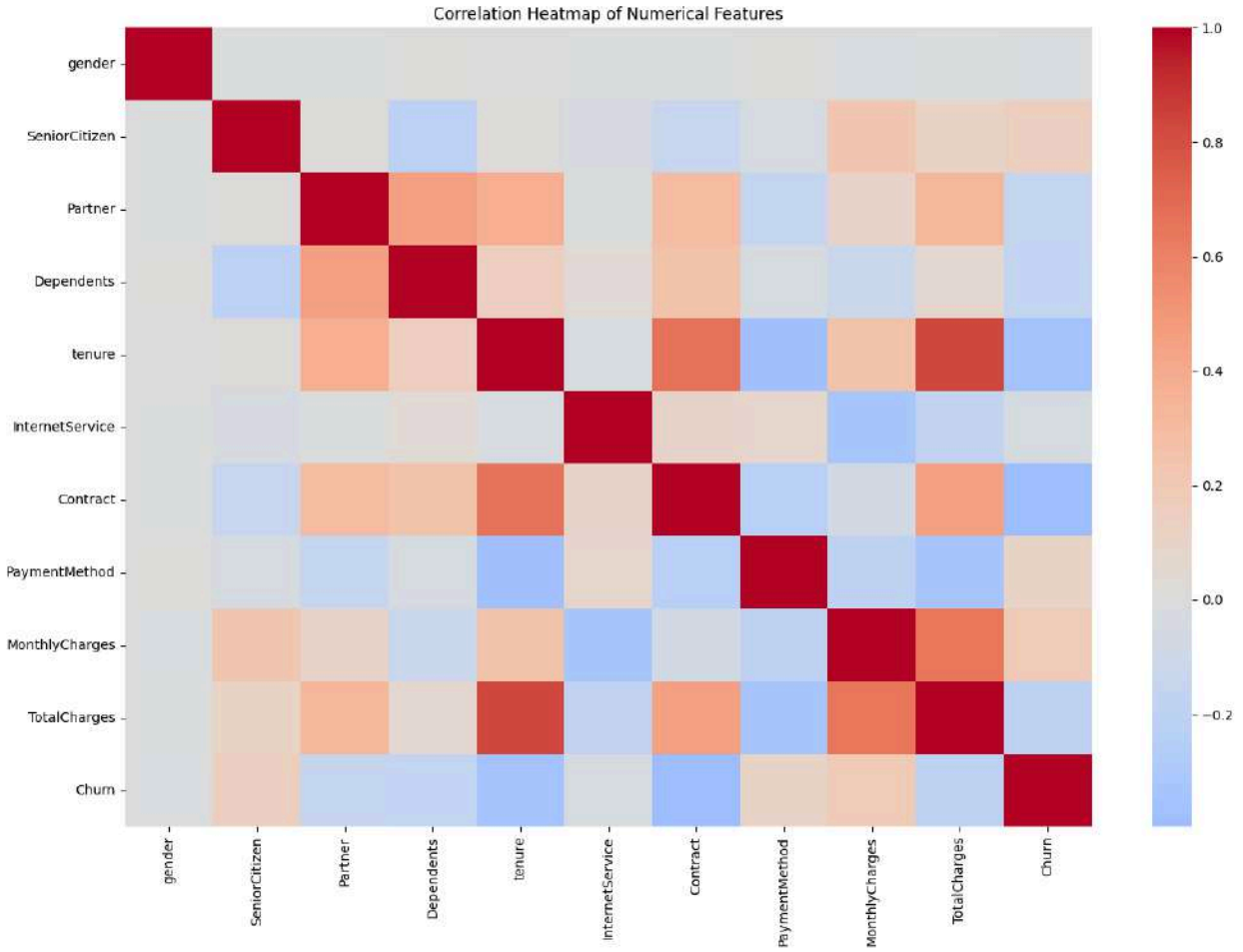
Step 7: Distribution plots displayed



Step 8: Contract vs Churn plot displayed



Step 9: Box plot displayed



Step 10: Correlation heatmap displayed

Preprocessed dataset saved successfully
Final dataset shape: (7043, 21)

This shape is an example for the standard dataset; it can change if duplicate records were removed or a different file was used.

Experiment 2: Data Discovery and Preparation — Explanation of Each Step

Name: S. Pargavi

Register Number: 241801196

Subject: AD23532 – Principles of Data Science

This explanation follows the tasks given in your Experiment 2 PDF and the Python program provided above.

Step 1: Import the required libraries

Python

Run

```
None
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
```

Explanation:

- pandas: Loads and manipulates customer data.
- numpy: Performs numerical operations and handles missing values.

- `matplotlib.pyplot`: Creates graphs and charts.
- `seaborn`: Creates statistical visualizations.
- `LabelEncoder`: Converts categorical values into numerical labels.
- `StandardScaler`: Standardizes numerical features.
- `train_test_split`: Splits the dataset into training and testing sets.

Output:

```
None
Step 1: Libraries imported successfully
```

Step 2: Load and display the dataset

Python

Run

```
None
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

print(df.head(10))
print(df.tail(10))
print("Dataset shape:", df.shape)
print(df.columns.tolist())
```

Explanation:

- `pd.read_csv()`: Loads the Telco Customer Churn CSV file into a DataFrame.
- `head(10)`: Displays the first 10 customer records.
- `tail(10)`: Displays the last 10 customer records.
- `shape`: Returns the number of rows and columns.
- `columns.tolist()`: Displays the names of all columns.

Expected output example:

```
None
Dataset shape: (7043, 21)
```

The commonly distributed dataset contains 7,043 records and 21 columns. Your output may differ if your CSV is different.

Step 3: Explore the dataset

Python

Run

```
None
df.info()
print(df.describe(include="all").T)
print(df.isnull().sum())
print("Duplicate records:", df.duplicated().sum())
```

Explanation:

1. `df.info()` displays column names, data types, and non-missing record counts.
2. `describe(include="all")` summarizes numerical and categorical columns.
3. `isnull().sum()` counts missing values in each column.
4. `duplicated().sum()` counts duplicate records.

Output: Dataset information, summary statistics, missing-value counts, and the number of duplicate records are displayed.

Step 4: Clean the dataset

Python

Run

None

```
df = df.drop_duplicates().copy()

df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"], errors="coerce"
)

numeric_cols = df.select_dtypes(include=np.number).columns

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

categorical_cols = df.select_dtypes(
    include=["object", "category", "bool"]
).columns

for col in categorical_cols:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].mode()[0])

print(df.isnull().sum())
```

Explanation:

- `drop_duplicates()` removes duplicate rows.
- `pd.to_numeric()` converts `TotalCharges` into numerical data.
- `errors="coerce"` converts invalid entries into missing values.
- `select_dtypes()` identifies numerical and categorical columns.
- `median()` finds the middle value of a numerical column.
- `mode()[0]` finds the most frequent value in a categorical column.
- `fillna()` replaces missing values with the selected median or mode.

Expected result: Missing-value counts should be zero after cleaning if all missing values are successfully filled.

Step 5: Encode categorical columns

Python

Run

```
None
columns_to_encode = [
    "gender", "Partner", "Dependents",
    "Contract", "PaymentMethod",
    "InternetService", "Churn"
]

encoders = {}

for col in columns_to_encode:
    if col in df.columns:
        encoder = LabelEncoder()
        df[col] = encoder.fit_transform(df[col].astype(str))
        encoders[col] = encoder
        print(f"{col} encoded successfully")
```

Explanation:

Categorical features contain text values, which are converted into numerical labels using `LabelEncoder`.

For example:

Original value	Encoded value
No	0
Yes	1

The program encodes Gender, Partner, Dependents, Contract, Payment Method, Internet Service, and Churn.

Output example:

```
None
gender encoded successfully
Partner encoded successfully
Contract encoded successfully
Churn encoded successfully
```

The actual program also prints the remaining requested columns.

Step 6: Standardize numerical features

Python

Run

```
None
scale_cols = ["tenure", "MonthlyCharges", "TotalCharges"]

scaler = StandardScaler()
df[scale_cols] = scaler.fit_transform(df[scale_cols])

print(df[scale_cols].head())
```

Explanation:

Standardization transforms the three numerical features to a common scale.

The formula is:

z=x−μσz=\frac{x-\mu}{\sigma}z=σx−μ

Where:

- xxx is the original value.
- μ\muμ is the feature mean.
- σ\sigmaσ is the feature standard deviation.
- zzz is the standardized value.

This helps prevent differences in numerical scales from dominating certain machine-learning algorithms.

Output: The first five standardized values for Tenure, MonthlyCharges, and TotalCharges are displayed.

Step 7: Display the correlation heatmap

Python

Run

None

```
numeric_df = df.select_dtypes(include=np.number)

sns.heatmap(
    numeric_df.corr(),
    cmap="coolwarm",
    center=0,
    annot=False
)

plt.title("Correlation Heatmap")
plt.show()
```

Explanation:

- `select_dtypes()` selects numerical columns.
- `corr()` calculates the correlation between pairs of features.
- `sns.heatmap()` displays the correlation matrix using colors.
- `center=0` centers the color scale at zero.
- `plt.show()` displays the heatmap.

Interpretation:

- Correlation near `+1`: Strong positive linear relationship.
- Correlation near `-1`: Strong negative linear relationship.

- Correlation near 0: Weak linear relationship.

Encoded categorical variables are also included, so their correlations should be interpreted carefully.

Step 8: Analyze categorical distributions

Python

Run

None

```
categorical_features = [
    ("Churn", "Customer Churn Distribution"),
    ("gender", "Gender Distribution"),
    ("Contract", "Contract Type Distribution"),
    ("PaymentMethod", "Payment Method Distribution")
]

for col, title in categorical_features:
    plt.figure(figsize=(7, 4))
    sns.countplot(data=df, x=col)
    plt.title(title)
    plt.xlabel(col)
    plt.ylabel("Number of Customers")
    plt.xticks(rotation=25)
    plt.tight_layout()
    plt.show()

    print(df[col].value_counts())
```

Explanation:

This step examines four categorical features:

- Churn: Number of customers who left or stayed.
- Gender: Distribution of customer genders.

- Contract: Distribution of contract categories.
- PaymentMethod: Distribution of payment methods.

`countplot()` displays the number of records in each category, while `value_counts()` prints the corresponding counts.

Output: Four count plots and their numerical frequency tables.

Step 9: Plot histograms and box plots

Python

Run

None

```
numerical_features = [
    "MonthlyCharges", "TotalCharges", "tenure"
]

for col in numerical_features:
    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    sns.histplot(df[col], kde=True)
    plt.title(f"Histogram of {col}")

    plt.subplot(1, 2, 2)
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}")

    plt.tight_layout()
    plt.show()
```

Explanation:

This step visualizes MonthlyCharges, TotalCharges, and Tenure.

- Histogram: Shows the distribution and frequency of values.
- Box plot: Shows the median, quartiles, spread, and potential outliers.
- `subplot(1, 2, 1)` creates the histogram panel.
- `subplot(1, 2, 2)` creates the box plot panel.
- `kde=True` adds a smooth density curve to each histogram.

Output: Three figures, each containing one histogram and one box plot.

Step 10: Detect outliers using the IQR method

Python

Run

None

```
for col in numerical_features:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    outliers = df[
        (df[col] < lower_limit) |
        (df[col] > upper_limit)
    ]

    print(f"\nFeature: {col}")
    print("Lower limit:", lower_limit)
    print("Upper limit:", upper_limit)
    print("Number of outliers:", len(outliers))
```

Explanation:

The Interquartile Range (IQR) method identifies values that lie unusually far from the middle 50% of the data.

IQR=Q3−Q1IQR=Q3-Q1IQR=Q3−Q1

Lower limit=Q1−1.5(IQR)\text{Lower limit}=Q1-1.5(IQR)Lower limit=Q1−1.5(IQR)

Upper limit=Q3+1.5(IQR)\text{Upper limit}=Q3+1.5(IQR)Upper limit=Q3+1.5(IQR)

- Q1: 25th percentile.
- Q3: 75th percentile.
- IQR: Difference between Q3 and Q1.
- outliers: Records below the lower limit or above the upper limit.

Output: Lower and upper limits and the number of potential outliers for each numerical feature.

Note: This step identifies outliers but does not remove them.

Step 11: Study Contract Type and Churn

Python

Run

```
None
sns.countplot(data=df, x="Contract", hue="Churn")
plt.title("Contract Type vs Customer Churn")
plt.xlabel("Contract Type (encoded)")
plt.ylabel("Number of Customers")
plt.show()
```

Explanation:

The count plot compares churned and retained customers across different contract types.

- `x="Contract"` represents contract categories.
- `hue="Churn"` separates the bars by churn status.

Output: A bar chart showing the relationship between contract type and churn.

Step 12: Study Internet Service and Churn

Python

Run

```
None
sns.countplot(data=df, x="InternetService", hue="Churn")
plt.title("Internet Service vs Customer Churn")
plt.xlabel("Internet Service (encoded)")
plt.ylabel("Number of Customers")
plt.show()
```

Explanation:

This chart compares customer churn across Internet Service categories.

It helps identify which Internet Service category has the highest churn proportion. The encoded category numbers should be interpreted using the original label mapping.

Output: A count plot showing customer counts by Internet Service and churn status.

Step 13: Study MonthlyCharges and Churn

Python

Run

```
None
sns.boxplot(data=df, x="Churn", y="MonthlyCharges")
```

```
plt.title("Monthly Charges vs Customer Churn")
plt.xlabel("Churn (0 = No, 1 = Yes)")
plt.ylabel("Standardized Monthly Charges")
plt.show()
```

Explanation:

The box plot compares the distribution of standardized MonthlyCharges for customers who churned and those who stayed.

Compare the medians, spread, and outliers to determine whether monthly charges differ between the two groups.

Output: A box plot of MonthlyCharges grouped by churn status.

Step 14: Study Tenure and Churn

Python

Run

None
```
sns.boxplot(data=df, x="Churn", y="tenure")
plt.title("Tenure vs Customer Churn")
plt.xlabel("Churn (0 = No, 1 = Yes)")
plt.ylabel("Standardized Tenure")
plt.show()
```

Explanation:

Tenure indicates how long a customer has used the service.

The box plot compares tenure between churned and retained customers. A lower median tenure among churned customers would suggest that customers with shorter service periods tend to leave more often.

Output: A box plot comparing tenure by churn status.

Step 15: Answer the analytical questions

The program uses the original category names for this analysis, rather than the encoded values.

15.1 Which contract type has the highest customer churn?

Python

Run

```
None
contract_churn = pd.crosstab(
    analysis_df["Contract"],
    analysis_df["Churn"],
    normalize="index"
) * 100

print(contract_churn.round(2))
print("Highest churn contract:",
      contract_churn["Yes"].idxmax())
```

Explanation: `crosstab()` calculates the percentage of churned and retained customers within each contract type. `idxmax()` identifies the contract with the highest churn rate.

15.2 Which payment method is most frequently used by customers who churn?

Python

Run

```
None
churned = analysis_df[
    analysis_df["Churn"].astype(str).str.lower() == "yes"
```

```
]

print(churned["PaymentMethod"].value_counts())
print("Most frequent payment method:",
      churned["PaymentMethod"].mode()[0])
```

Explanation: The code filters the dataset to churned customers, counts their payment methods, and identifies the most frequent one.

15.3 Do customers with shorter tenure tend to churn more often?

Python

Run

None

```
print(analysis_df.groupby("Churn")["tenure"].mean())

churned_tenure = analysis_df.loc[
    analysis_df["Churn"] == "Yes", "tenure"
].mean()

retained_tenure = analysis_df.loc[
    analysis_df["Churn"] == "No", "tenure"
].mean()

print("Churned average tenure:", churned_tenure)
print("Retained average tenure:", retained_tenure)
```

Explanation: The program calculates average tenure separately for churned and retained customers. If the churned group has a lower average tenure, it supports the observation that shorter-tenure customers tend to churn more often.

15.4 Which customer group has the highest monthly charges?

Python

Run

```
None
print(
    analysis_df.groupby("Contract")["MonthlyCharges"].mean()
)

print(
    analysis_df.groupby("Contract")["MonthlyCharges"].mean().idxmax()
)
```

Explanation: This code compares average monthly charges by contract type and identifies the contract group with the highest average.

15.5 Which Internet Service category experiences the highest churn?

Python

Run

```
None
internet_churn = pd.crosstab(
    analysis_df["InternetService"],
    analysis_df["Churn"],
    normalize="index"
) * 100

print(internet_churn.round(2))
print("Highest churn Internet Service:",
      internet_churn["Yes"].idxmax())
```

Explanation: The program calculates churn percentages for each Internet Service category and identifies the category with the highest churn rate.

15.6 Is there a relationship between monthly charges and churn?

Python

Run

```
None
print(
    analysis_df.groupby("Churn")["MonthlyCharges"].mean()
)
```

Explanation: The average monthly charges for churned and retained customers are compared. A difference indicates an association, but it does not prove that monthly charges cause churn.

15.7 Which features appear to be the strongest indicators of churn?

Python

Run

```
None
correlation_df = analysis_df.copy()

for col in correlation_df.select_dtypes(include="object").columns:
    correlation_df[col] = LabelEncoder().fit_transform(
        correlation_df[col].astype(str)
    )

correlation_df["TotalCharges"] = pd.to_numeric(
    correlation_df["TotalCharges"], errors="coerce"
)
```

```
correlation_df = correlation_df.fillna(
    correlation_df.median(numeric_only=True)
)

churn_correlations = (
    correlation_df.select_dtypes(include=np.number)
    .corr()["Churn"]
    .drop("Churn")
    .sort_values(key=abs, ascending=False)
)

print(churn_correlations.head(10).round(3))
```

Explanation:

1. A copy of the original dataset is created.
2. Categorical columns are converted into numerical labels.
3. Missing numerical values are filled.
4. Correlations between each numerical feature and Churn are calculated.
5. Features are sorted by the absolute magnitude of their correlations.

Output: The ten features with the largest absolute correlations with encoded Churn.

Important: This is a preliminary analysis, not a definitive feature-importance ranking. Label encoding can affect correlations, and predictive importance should be confirmed with an appropriately evaluated machine-learning model.

Step 16: Split the dataset into training and testing sets

Python

Run

```
None
model_df = df.drop(columns=["customerID"], errors="ignore")

X = model_df.drop(columns=["Churn"])
y = model_df["Churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Total samples:", len(model_df))
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Explanation:

- `customerID` is removed because it is an identifier rather than a meaningful predictive feature.
- `X` contains the input features.
- `y` contains the target variable, Churn.
- `test_size=0.20` allocates 20% of the data to testing.
- `random_state=42` makes the split reproducible.
- `stratify=y` helps maintain the churn-class proportions in both sets.

For 7,043 records, the expected split is approximately:

Dataset	Samples	Percentage
Training	5,634	80%

Testing	1,409	20%
Total	7,043	100%

The actual counts depend on the number of rows remaining after cleaning.

Step 17: Save the final processed dataset

Python

Run

```
None
df.to_csv("CustomerChurn_Preprocessed.csv", index=False)

print("Final processed dataset saved successfully")
print("Final dataset shape:", df.shape)
```

Explanation:

- `to_csv()` saves the processed dataset as a new CSV file.
- `index=False` prevents the DataFrame index from being saved as an extra column.
- `df.shape` displays the final dimensions.

Output:

```
None
Final processed dataset saved successfully
Final dataset shape: (7043, 21)
```

Full code:

```
#S.pargavi#241801196

# ============================================================
# EXPERIMENT 2: DATA DISCOVERY AND PREPARATION
# ============================================================

# STEP 1: Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

print("Step 1: Libraries imported successfully")


# STEP 2: Load the dataset
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

print("\nFirst 10 records:")
print(df.head(10))

print("\nLast 10 records:")
print(df.tail(10))

print("\nDataset shape:", df.shape)

print("\nColumn names:")
print(df.columns.tolist())


# STEP 3: Explore the dataset
print("\nDataset information:")
df.info()

print("\nSummary statistics:")
```

```
print(df.describe(include="all").T)

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate records:", df.duplicated().sum())


# STEP 4: Clean the dataset
df = df.drop_duplicates().copy()

# Convert TotalCharges to numeric
df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"], errors="coerce"
)

# Fill numerical missing values with median
numeric_cols = df.select_dtypes(include=np.number).columns

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

# Fill categorical missing values with mode
categorical_cols = df.select_dtypes(
    include=["object", "category", "bool"]
).columns

for col in categorical_cols:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].mode()[0])

print("\nMissing values after cleaning:")
print(df.isnull().sum())

print("Total missing values:", df.isnull().sum().sum())


# STEP 5: Encode categorical columns
columns_to_encode = [
    "gender",
```

```
    "Partner",
    "Dependents",
    "Contract",
    "PaymentMethod",
    "InternetService",
    "Churn"
]

encoders = {}

for col in columns_to_encode:
    if col in df.columns:
        encoder = LabelEncoder()
        df[col] = encoder.fit_transform(df[col].astype(str))
        encoders[col] = encoder
        print(f"{col} encoded successfully")


# STEP 6: Standardize numerical features
scale_cols = [
    "tenure",
    "MonthlyCharges",
    "TotalCharges"
]

scaler = StandardScaler()
df[scale_cols] = scaler.fit_transform(df[scale_cols])

print("\nStandardized numerical features:")
print(df[scale_cols].head())


# STEP 7: Correlation heatmap (reused from Experiment 1)
plt.figure(figsize=(14, 10))

numeric_df = df.select_dtypes(include=np.number)

sns.heatmap(
    numeric_df.corr(),
    cmap="coolwarm",
```

```
    center=0,
    annot=False
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()


# STEP 8: Analyze categorical distributions
categorical_features = [
    ("Churn", "Customer Churn Distribution"),
    ("gender", "Gender Distribution"),
    ("Contract", "Contract Type Distribution"),
    ("PaymentMethod", "Payment Method Distribution")
]

for col, title in categorical_features:
    plt.figure(figsize=(7, 4))
    sns.countplot(data=df, x=col)
    plt.title(title)
    plt.xlabel(col)
    plt.ylabel("Number of Customers")
    plt.xticks(rotation=25)
    plt.tight_layout()
    plt.show()

    print(f"\nDistribution of {col}:")
    print(df[col].value_counts())


# STEP 9: Histograms and boxplots of numerical features
numerical_features = [
    "MonthlyCharges",
    "TotalCharges",
    "tenure"
]

for col in numerical_features:
    plt.figure(figsize=(12, 4))
```

```
    plt.subplot(1, 2, 1)
    sns.histplot(df[col], kde=True)
    plt.title(f"Histogram of {col}")

    plt.subplot(1, 2, 2)
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}")

    plt.tight_layout()
    plt.show()


# STEP 10: Detect outliers using the IQR method
print("\nOUTLIER DETECTION USING IQR")

for col in numerical_features:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    outliers = df[
        (df[col] < lower_limit) |
        (df[col] > upper_limit)
    ]

    print(f"\nFeature: {col}")
    print("Lower limit:", round(lower_limit, 3))
    print("Upper limit:", round(upper_limit, 3))
    print("Number of outliers:", len(outliers))


# STEP 11: Relationship between Contract Type and Churn
plt.figure(figsize=(8, 5))

sns.countplot(data=df, x="Contract", hue="Churn")
```

```
plt.title("Contract Type vs Customer Churn")
plt.xlabel("Contract Type (encoded)")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()


# STEP 12: Relationship between Internet Service and Churn
# InternetService was encoded in Step 5
plt.figure(figsize=(8, 5))

sns.countplot(data=df, x="InternetService", hue="Churn")

plt.title("Internet Service vs Customer Churn")
plt.xlabel("Internet Service (encoded)")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()


# STEP 13: Monthly Charges and Churn
plt.figure(figsize=(8, 5))

sns.boxplot(data=df, x="Churn", y="MonthlyCharges")

plt.title("Monthly Charges vs Customer Churn")
plt.xlabel("Churn (0 = No, 1 = Yes)")
plt.ylabel("Standardized Monthly Charges")
plt.tight_layout()
plt.show()


# STEP 14: Tenure and Churn
plt.figure(figsize=(8, 5))

sns.boxplot(data=df, x="Churn", y="tenure")

plt.title("Tenure vs Customer Churn")
plt.xlabel("Churn (0 = No, 1 = Yes)")
```

```
plt.ylabel("Standardized Tenure")
plt.tight_layout()
plt.show()


# STEP 15: Analytical questions
print("\n========== ANALYTICAL RESULTS ==========")

# Recover original category names for analysis
analysis_df = pd.read_csv(
    "WA_Fn-UseC_-Telco-Customer-Churn.csv"
)

analysis_df["TotalCharges"] = pd.to_numeric(
    analysis_df["TotalCharges"], errors="coerce"
)

analysis_df = analysis_df.drop_duplicates().copy()

analysis_df["TotalCharges"] = analysis_df["TotalCharges"].fillna(
    analysis_df["TotalCharges"].median()
)

analysis_df["MonthlyCharges"] = analysis_df["MonthlyCharges"].fillna(
    analysis_df["MonthlyCharges"].median()
)

analysis_df["tenure"] = analysis_df["tenure"].fillna(
    analysis_df["tenure"].median()
)

# Question 1: Which contract type has the highest churn rate?
contract_churn = pd.crosstab(
    analysis_df["Contract"],
    analysis_df["Churn"],
    normalize="index"
) * 100

print("\n1. Churn rate by contract type (%):")
print(contract_churn.round(2))
```

```
if "Yes" in contract_churn.columns:
    print("Highest churn contract:",
          contract_churn["Yes"].idxmax())


# Question 2: Most frequently used payment method among churned customers
churned = analysis_df[
    analysis_df["Churn"].astype(str).str.lower() == "yes"
]

print("\n2. Payment methods among churned customers:")
print(churned["PaymentMethod"].value_counts())

if not churned.empty:
    print("Most frequent payment method:",
          churned["PaymentMethod"].mode()[0])


# Question 3: Do customers with shorter tenure churn more often?
print("\n3. Average tenure by churn status:")
print(analysis_df.groupby("Churn")["tenure"].mean().round(2))

if "Yes" in analysis_df["Churn"].values:
    churned_tenure = analysis_df.loc[
        analysis_df["Churn"] == "Yes", "tenure"
    ].mean()

    retained_tenure = analysis_df.loc[
        analysis_df["Churn"] == "No", "tenure"
    ].mean()

    print("Average tenure of churned customers:", round(churned_tenure,
2))
    print("Average tenure of retained customers:", round(retained_tenure,
2))


# Question 4: Which customer group has the highest monthly charges?
print("\n4. Average monthly charges by contract:")
```

```
print(analysis_df.groupby("Contract")["MonthlyCharges"].mean().round(2))

print("Contract with highest average monthly charges:",
      analysis_df.groupby("Contract")["MonthlyCharges"].mean().idxmax())


# Question 5: Which Internet Service category has the highest churn?
internet_churn = pd.crosstab(
    analysis_df["InternetService"],
    analysis_df["Churn"],
    normalize="index"
) * 100

print("\n5. Churn rate by Internet Service (%):")
print(internet_churn.round(2))

if "Yes" in internet_churn.columns:
    print("Internet Service with highest churn:",
          internet_churn["Yes"].idxmax())


# Question 6: Relationship between monthly charges and churn
print("\n6. Average monthly charges by churn status:")
print(analysis_df.groupby("Churn")["MonthlyCharges"].mean().round(2))


# Question 7: Strongest indicators of churn (correlation)
print("\n7. Numerical feature correlations with encoded Churn:")

correlation_df = analysis_df.copy()

for col in correlation_df.select_dtypes(include="object").columns:
    correlation_df[col] = LabelEncoder().fit_transform(
        correlation_df[col].astype(str)
    )

correlation_df["TotalCharges"] = pd.to_numeric(
    correlation_df["TotalCharges"], errors="coerce"
)
```

```
correlation_df = correlation_df.fillna(
    correlation_df.median(numeric_only=True)
)

churn_correlations = (
    correlation_df.select_dtypes(include=np.number)
    .corr()["Churn"]
    .drop("Churn")
    .sort_values(key=abs, ascending=False)
)

print(churn_correlations.head(10).round(3))


# STEP 16: Split data into training (80%) and testing (20%)
# Drop customerID because it is an identifier, not a predictive feature
model_df = df.drop(columns=["customerID"], errors="ignore")

X = model_df.drop(columns=["Churn"])
y = model_df["Churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\n========== TRAIN-TEST SPLIT ==========")
print("Total samples:", len(model_df))
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Training percentage:", round(len(X_train) / len(model_df) * 100,
2), "%")
print("Testing percentage:", round(len(X_test) / len(model_df) * 100, 2),
"%")


# STEP 17: Save the final processed dataset
```

```
df.to_csv("CustomerChurn_Preprocessed.csv", index=False)

print("\nFinal processed dataset saved as CustomerChurn_Preprocessed.csv")
print("Final dataset shape:", df.shape)
```

Output:

Step 1: Libraries imported successfully

First 10 records:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	\
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	
4	9237-HQITU	Female	0	No	No	2	Yes	
5	9305-CDSKC	Female	0	No	No	8	Yes	
6	1452-KIOVK	Male	0	No	Yes	22	Yes	
7	6713-OKOMC	Female	0	No	No	10	No	
8	7892-POOKP	Female	0	Yes	No	28	Yes	
9	6388-TABGU	Male	0	No	Yes	62	Yes	

	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	\
0	No phone service	DSL	No	...	No	
1	No	DSL	Yes	...	Yes	
2	No	DSL	Yes	...	No	
3	No phone service	DSL	Yes	...	Yes	
4	No	Fiber optic	No	...	No	
5	Yes	Fiber optic	No	...	Yes	
6	Yes	Fiber optic	No	...	No	
7	No phone service	DSL	Yes	...	No	
8	Yes	Fiber optic	No	...	Yes	
9	No	DSL	Yes	...	No	

	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	\
0	No	No	No	Month-to-month	Yes	
1	No	No	No	One year	No	
2	No	No	No	Month-to-month	Yes	
3	Yes	No	No	One year	No	
4	No	No	No	Month-to-month	Yes	
5	No	Yes	Yes	Month-to-month	Yes	
6	No	Yes	No	Month-to-month	Yes	
7	No	No	No	Month-to-month	No	

```
8         Yes        Yes           Yes  Month-to-month            Yes
9          No         No            No        One year             No

              PaymentMethod MonthlyCharges  TotalCharges Churn
0          Electronic check          29.85         29.85    No
1              Mailed check          56.95        1889.5    No
2              Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4          Electronic check          70.70        151.65   Yes
5          Electronic check          99.65         820.5   Yes
6    Credit card (automatic)          89.10        1949.4    No
7              Mailed check          29.75         301.9    No
8          Electronic check         104.80       3046.05   Yes
9  Bank transfer (automatic)          56.15       3487.95    No

[10 rows x 21 columns]

Last 10 records:
      customerID  gender  SeniorCitizen Partner Dependents  tenure  \
7033  9767-FFLEM    Male              0      No         No      38
7034  0639-TSIQW  Female              0      No         No      67
7035  8456-QDAVC    Male              0      No         No      19
7036  7750-EYXWZ  Female              0      No         No      12
7037  2569-WGERO  Female              0      No         No      72
7038  6840-RESVB    Male              0     Yes        Yes      24
7039  2234-XADUH  Female              0     Yes        Yes      72
7040  4801-JZAZL  Female              0     Yes        Yes      11
7041  8361-LTMKD    Male              1     Yes         No       4
7042  3186-AJIEK    Male              0      No         No      66

     PhoneService     MultipleLines InternetService       OnlineSecurity  ...  \
7033          Yes                No     Fiber optic                   No  ...
7034          Yes               Yes     Fiber optic                  Yes  ...
7035          Yes                No     Fiber optic                   No  ...
7036           No  No phone service             DSL                   No  ...
7037          Yes                No              No  No internet service  ...
7038          Yes               Yes             DSL                  Yes  ...
7039          Yes               Yes     Fiber optic                   No  ...
7040           No  No phone service             DSL                  Yes  ...
7041          Yes               Yes     Fiber optic                   No  ...
7042          Yes                No     Fiber optic                  Yes  ...

        DeviceProtection         TechSupport         StreamingTV  \
7033                  No                  No                  No
7034                 Yes                  No                 Yes
7035                  No                  No                 Yes
```

```
7036              Yes              Yes              Yes
7037  No internet service  No internet service  No internet service
7038              Yes              Yes              Yes
7039              Yes               No              Yes
7040               No               No               No
7041               No               No               No
7042              Yes              Yes              Yes

      StreamingMovies        Contract PaperlessBilling  \
7033              No  Month-to-month              Yes
7034              No  Month-to-month              Yes
7035              No  Month-to-month              Yes
7036             Yes        One year               No
7037  No internet service        Two year              Yes
7038             Yes        One year              Yes
7039             Yes        One year              Yes
7040              No  Month-to-month              Yes
7041              No  Month-to-month              Yes
7042             Yes        Two year              Yes

            PaymentMethod MonthlyCharges  TotalCharges Churn
7033   Credit card (automatic)          69.50       2625.25    No
7034   Credit card (automatic)         102.95       6886.25   Yes
7035  Bank transfer (automatic)          78.70        1495.1    No
7036          Electronic check          60.65         743.3    No
7037  Bank transfer (automatic)          21.15        1419.4    No
7038              Mailed check          84.80        1990.5    No
7039   Credit card (automatic)         103.20        7362.9    No
7040          Electronic check          29.60        346.45    No
7041              Mailed check          74.40         306.6   Yes
7042  Bank transfer (automatic)         105.65        6844.5    No

[10 rows x 21 columns]

Dataset shape: (7043, 21)

Column names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService',
'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges',
'TotalCharges', 'Churn']

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
```

```
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

Summary statistics:
                count unique                top  freq       mean        std  \
customerID       7043   7043         3186-AJIEK     1        NaN        NaN
gender           7043      2               Male  3555        NaN        NaN
SeniorCitizen  7043.0    NaN                NaN   NaN   0.162147   0.368612
Partner          7043      2                No  3641        NaN        NaN
Dependents       7043      2                No  4933        NaN        NaN
tenure         7043.0    NaN                NaN   NaN  32.371149  24.559481
PhoneService     7043      2               Yes  6361        NaN        NaN
MultipleLines    7043      3                No  3390        NaN        NaN
InternetService  7043      3       Fiber optic  3096        NaN        NaN
OnlineSecurity   7043      3                No  3498        NaN        NaN
OnlineBackup     7043      3                No  3088        NaN        NaN
DeviceProtection 7043      3                No  3095        NaN        NaN
TechSupport      7043      3                No  3473        NaN        NaN
StreamingTV      7043      3                No  2810        NaN        NaN
StreamingMovies  7043      3                No  2785        NaN        NaN
Contract         7043      3    Month-to-month  3875        NaN        NaN
PaperlessBilling 7043      2               Yes  4171        NaN        NaN
PaymentMethod    7043      4  Electronic check  2365        NaN        NaN
```

MonthlyCharges	7043.0	NaN	NaN	NaN	64.761692	30.090047
TotalCharges	7043	6531	11	NaN	NaN	
Churn	7043	2	No	5174	NaN	NaN

	min	25%	50%	75%	max
customerID	NaN	NaN	NaN	NaN	NaN
gender	NaN	NaN	NaN	NaN	NaN
SeniorCitizen	0.0	0.0	0.0	0.0	1.0
Partner	NaN	NaN	NaN	NaN	NaN
Dependents	NaN	NaN	NaN	NaN	NaN
tenure	0.0	9.0	29.0	55.0	72.0
PhoneService	NaN	NaN	NaN	NaN	NaN
MultipleLines	NaN	NaN	NaN	NaN	NaN
InternetService	NaN	NaN	NaN	NaN	NaN
OnlineSecurity	NaN	NaN	NaN	NaN	NaN
OnlineBackup	NaN	NaN	NaN	NaN	NaN
DeviceProtection	NaN	NaN	NaN	NaN	NaN
TechSupport	NaN	NaN	NaN	NaN	NaN
StreamingTV	NaN	NaN	NaN	NaN	NaN
StreamingMovies	NaN	NaN	NaN	NaN	NaN
Contract	NaN	NaN	NaN	NaN	NaN
PaperlessBilling	NaN	NaN	NaN	NaN	NaN
PaymentMethod	NaN	NaN	NaN	NaN	NaN
MonthlyCharges	18.25	35.5	70.35	89.85	118.75
TotalCharges	NaN	NaN	NaN	NaN	NaN
Churn	NaN	NaN	NaN	NaN	NaN

Missing values:

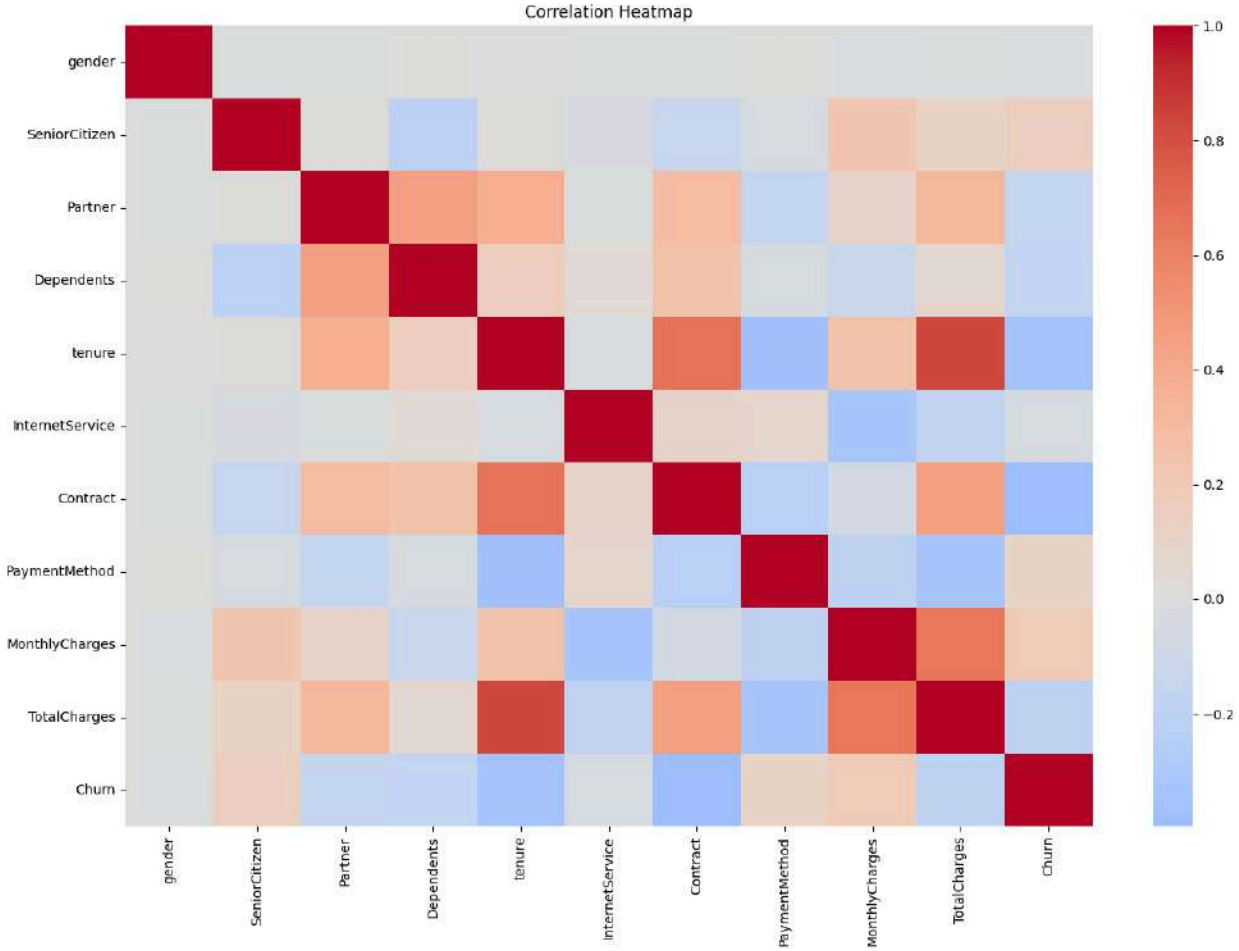
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0

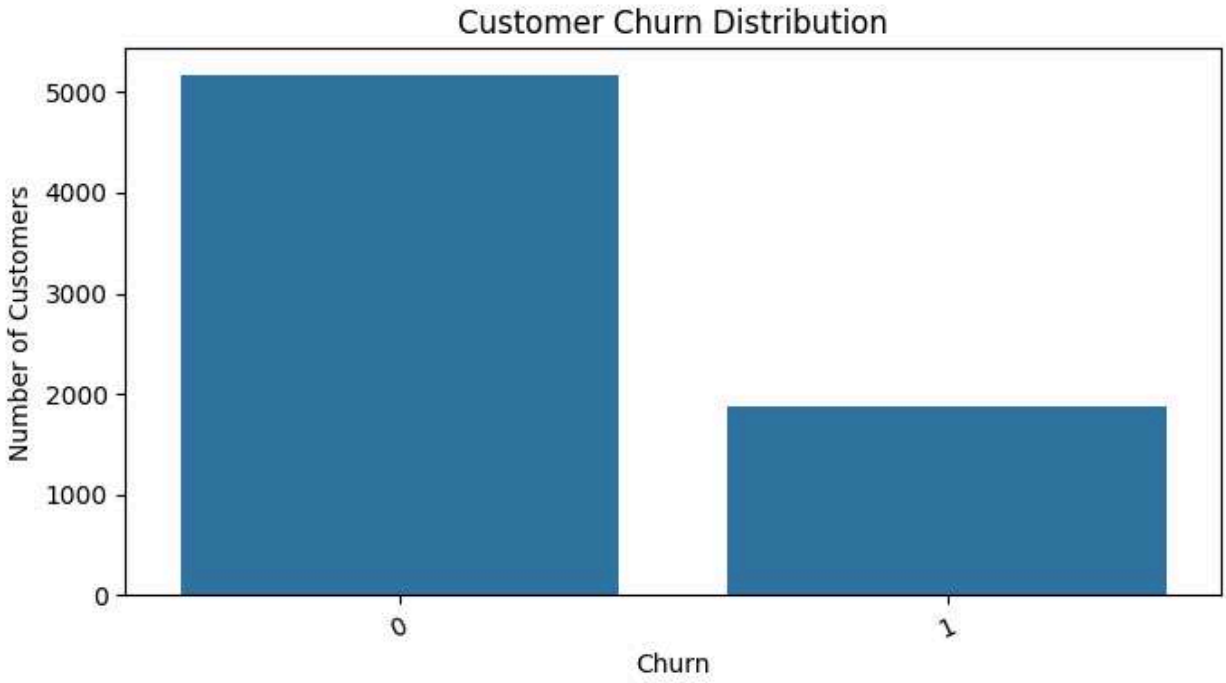
```
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Duplicate records: 0

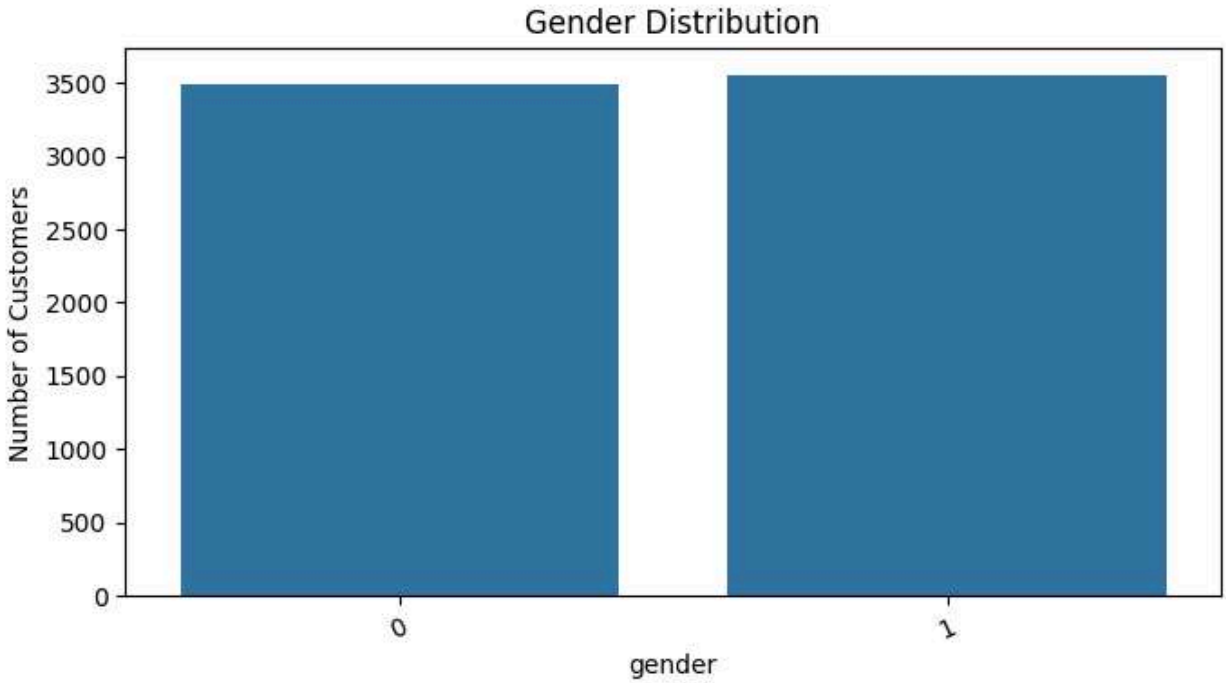
Missing values after cleaning:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
Total missing values: 0
gender encoded successfully
Partner encoded successfully
Dependents encoded successfully
Contract encoded successfully
PaymentMethod encoded successfully
InternetService encoded successfully
Churn encoded successfully

Standardized numerical features:
     tenure  MonthlyCharges  TotalCharges
0 -1.277445       -1.160323     -0.994242
1  0.066327       -0.259629     -0.173244
2 -1.236724       -0.362660     -0.959674
3  0.514251       -0.746535     -0.194766
4 -1.236724        0.197365     -0.940470
```


Correlation Heatmap
gender
SeniorCitizen
Partner
Dependents
tenure
InternetService
Contract
PaymentMethod
MonthlyCharges
TotalCharges
Churn
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

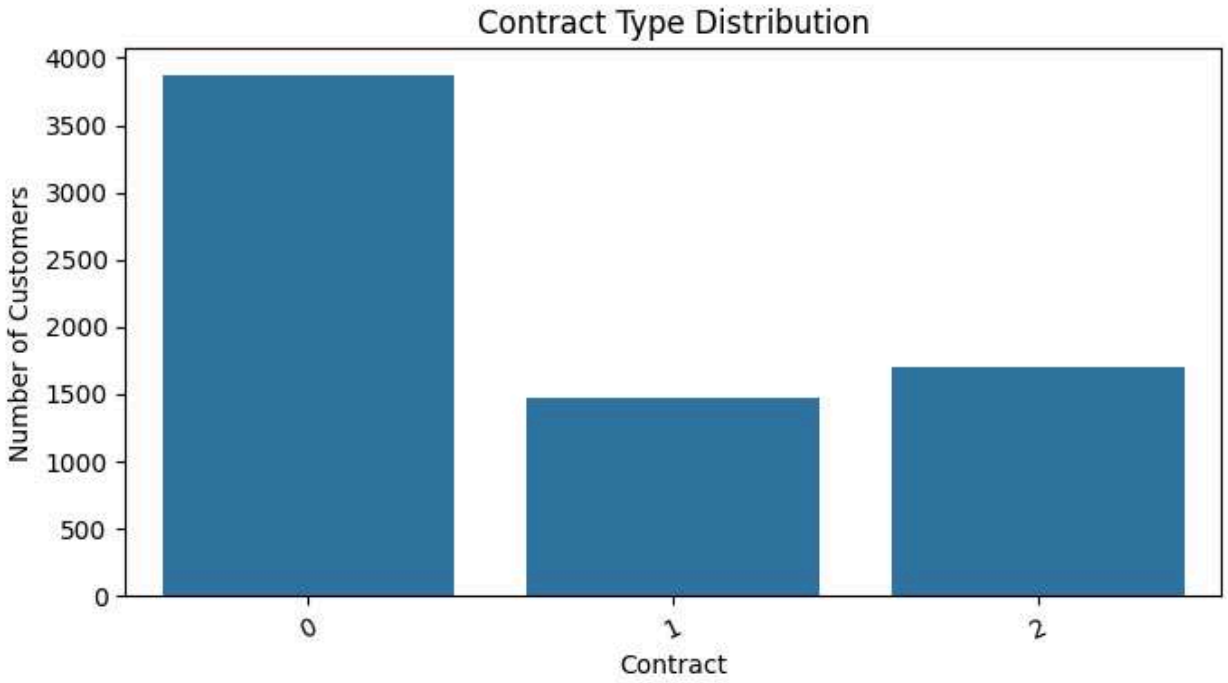


Distribution of Churn:
Churn
0 5174
1 1869
Name: count, dtype: int64

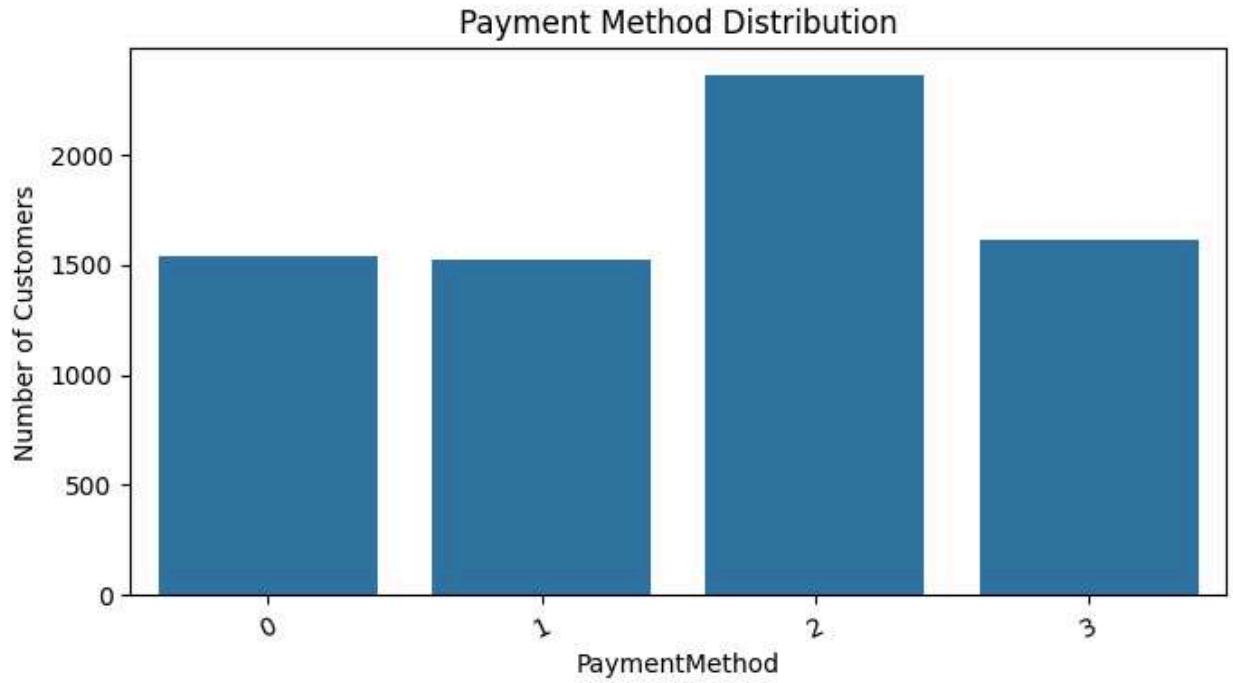


Distribution of gender:

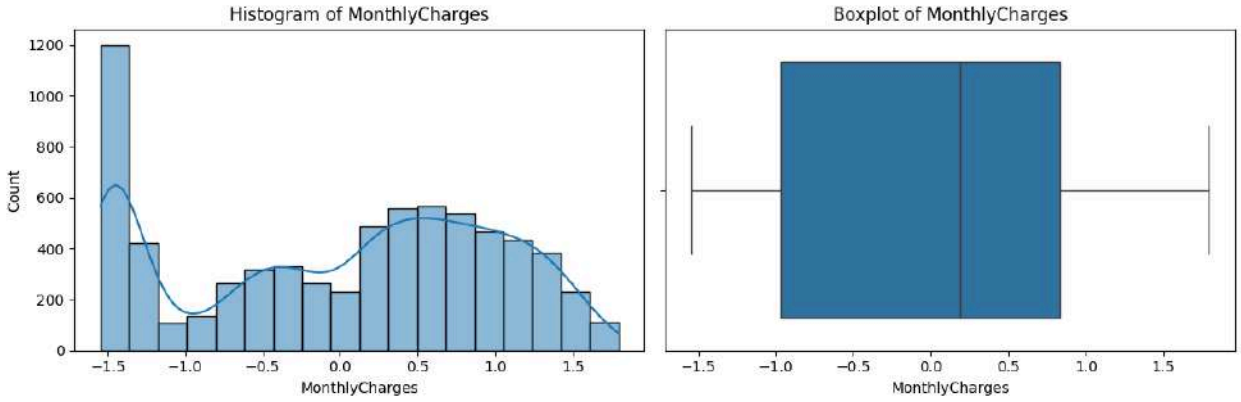
gender
1 3555
0 3488
Name: count, dtype: int64

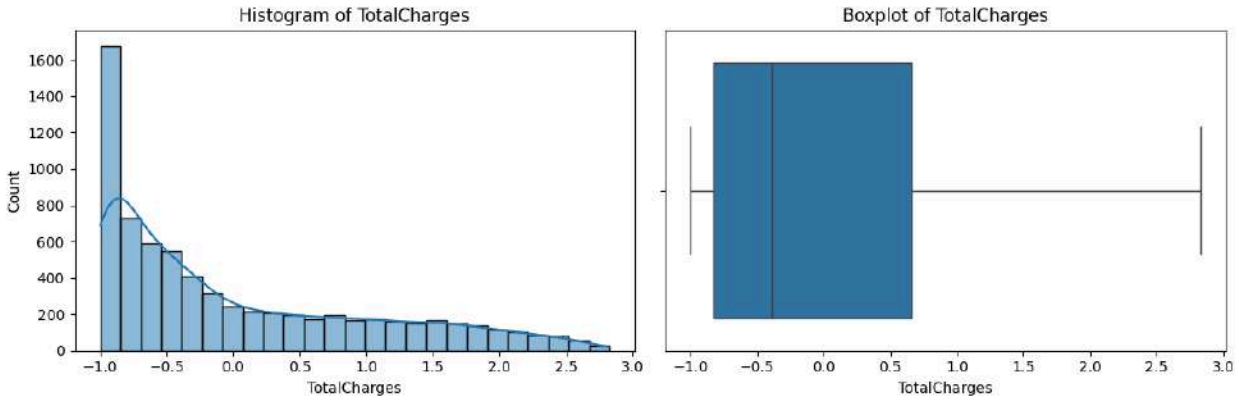


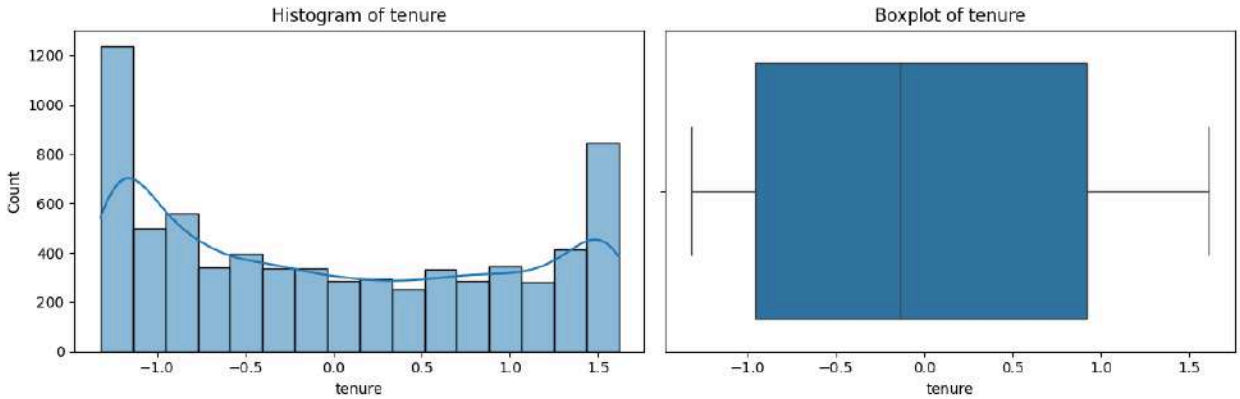
Distribution of Contract:
Contract
0 3875
2 1695
1 1473
Name: count, dtype: int64



Distribution of PaymentMethod:
PaymentMethod
2 2365
3 1612
0 1544
1 1522
Name: count, dtype: int64





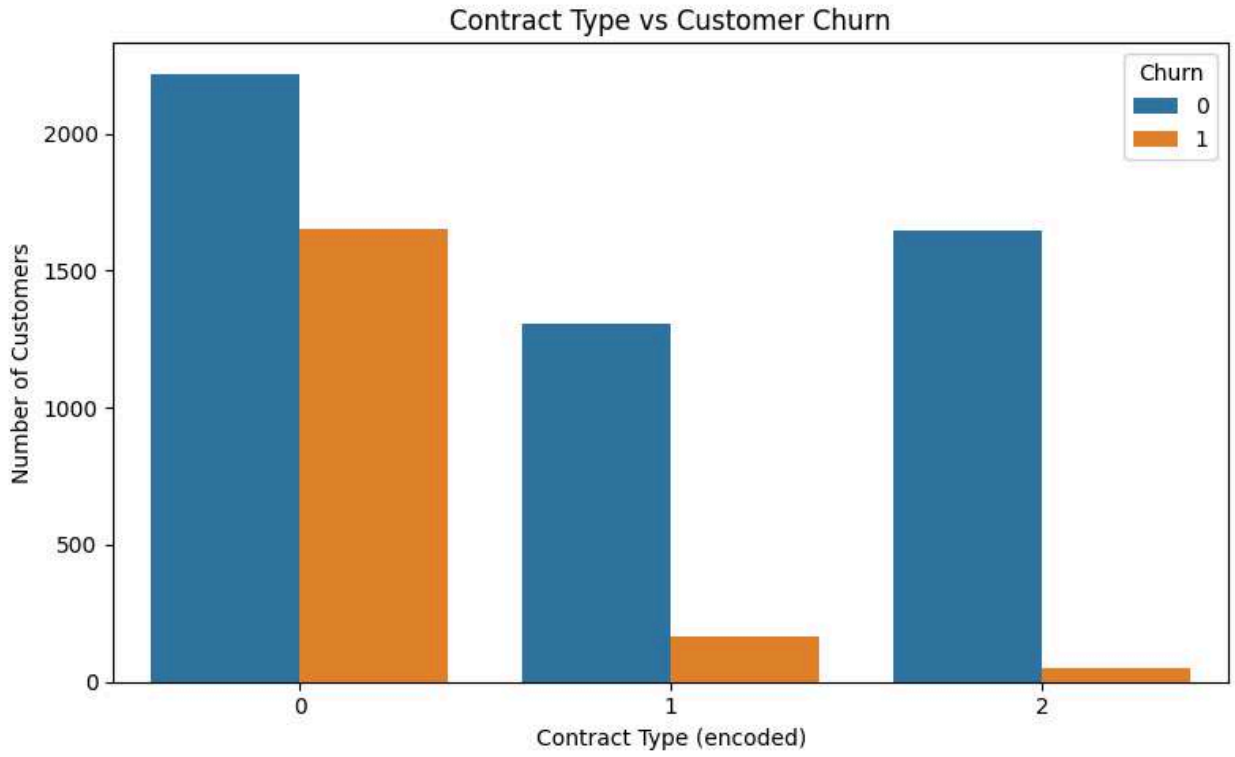


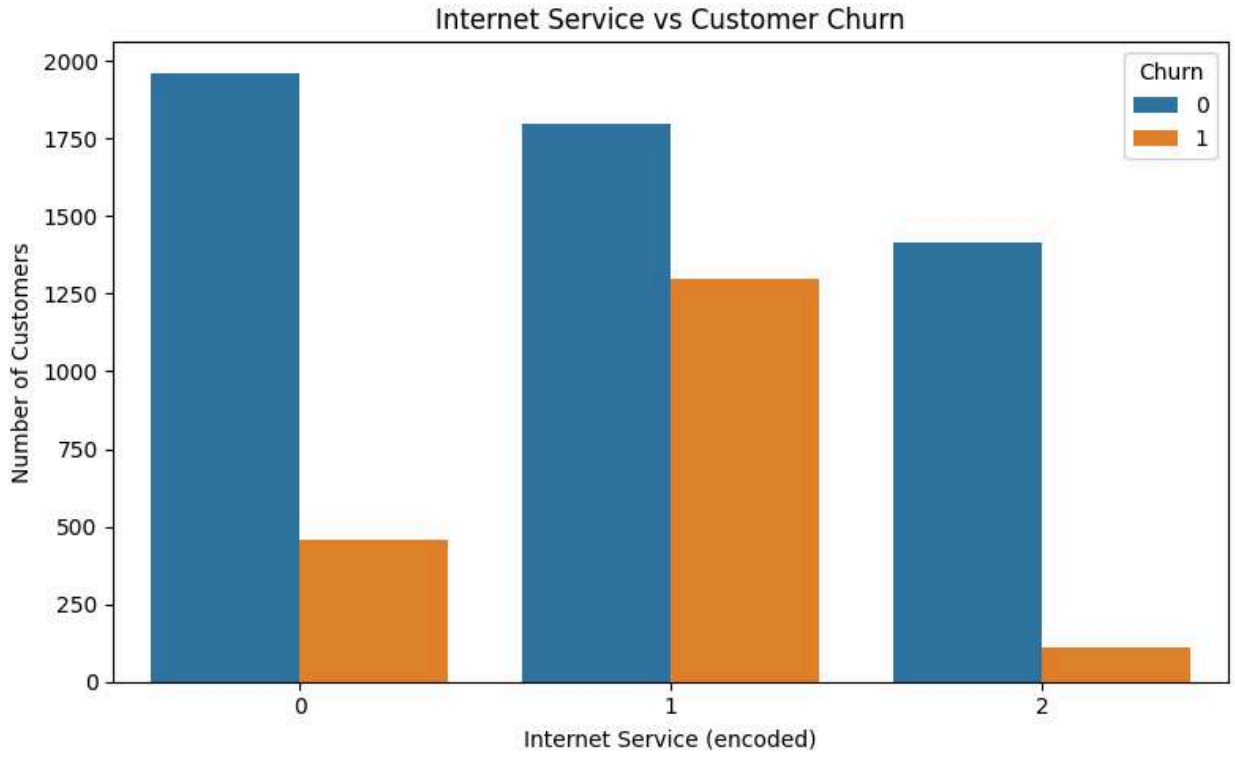
OUTLIER DETECTION USING IQR

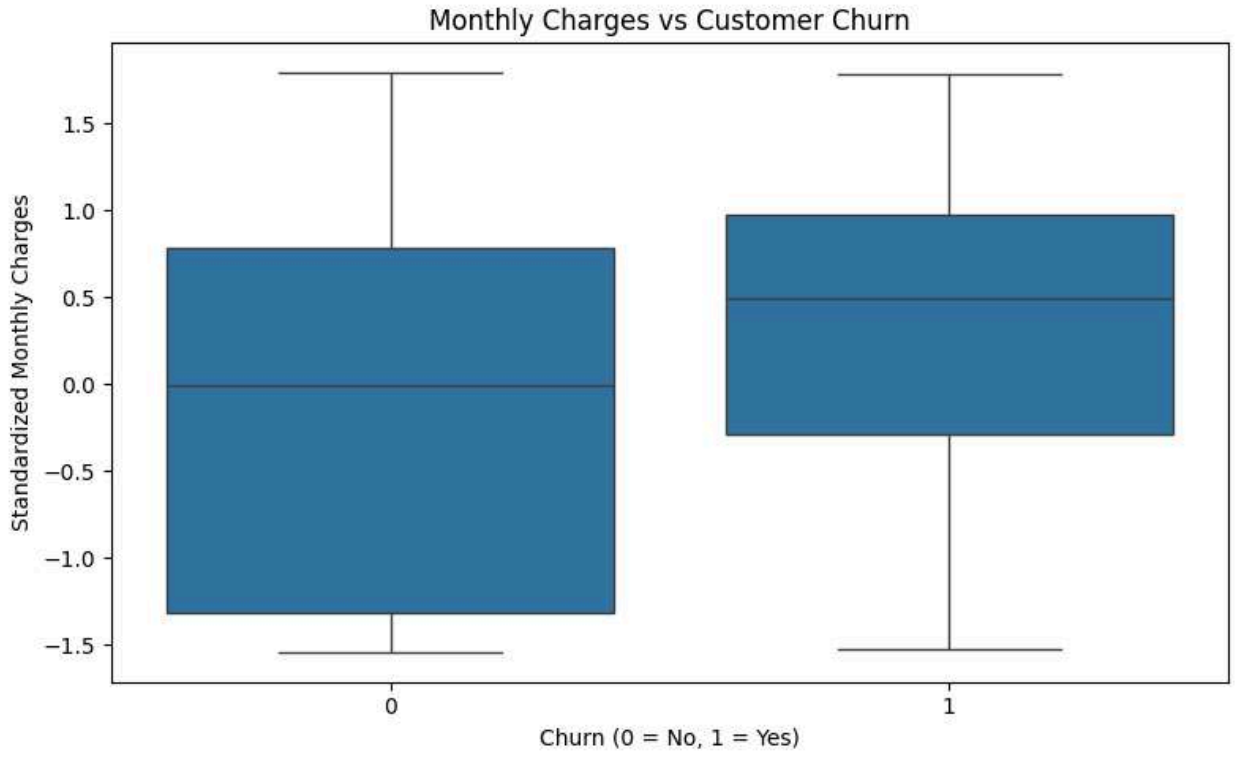
Feature: MonthlyCharges
Lower limit: -3.682
Upper limit: 3.543
Number of outliers: 0

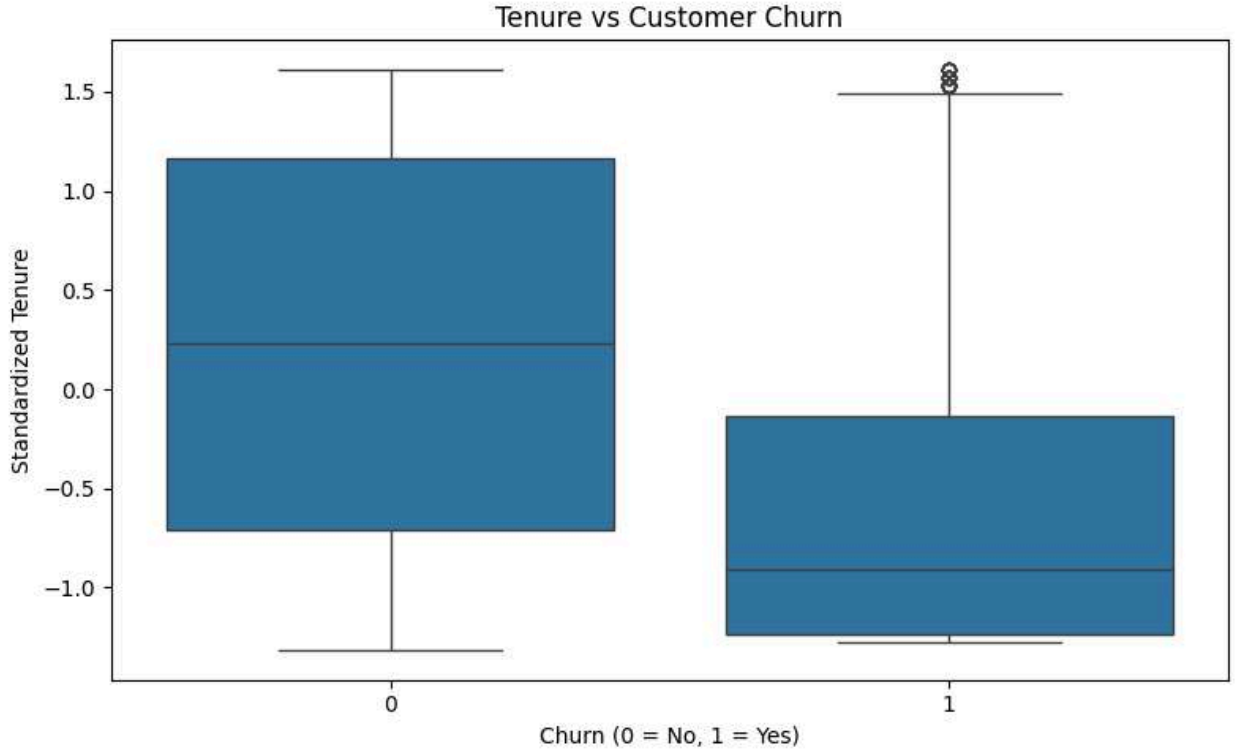
Feature: TotalCharges
Lower limit: -3.071
Upper limit: 2.905
Number of outliers: 0

Feature: tenure
Lower limit: -3.761
Upper limit: 3.731
Number of outliers: 0


Contract Type vs Customer Churn
Churn
0
1
Number of Customers
2000
1500
1000
500
0
0
1
2
Contract Type (encoded)


Internet Service vs Customer Churn
Churn
0
1
Number of Customers
2000
1750
1500
1250
1000
750
500
250
0
0
1
2
Internet Service (encoded)





========== ANALYTICAL RESULTS ==========

1. Churn rate by contract type (%):
Churn No Yes

Contract
Month-to-month 57.29 42.71
One year 88.73 11.27
Two year 97.17 2.83
Highest churn contract: Month-to-month

2. Payment methods among churned customers:
PaymentMethod
Electronic check 1071
Mailed check 308
Bank transfer (automatic) 258
Credit card (automatic) 232
Name: count, dtype: int64
Most frequent payment method: Electronic check

3. Average tenure by churn status:
Churn
No 37.57
Yes 17.98
Name: tenure, dtype: float64
Average tenure of churned customers: 17.98
Average tenure of retained customers: 37.57

4. Average monthly charges by contract:
Contract
Month-to-month 66.40
One year 65.05
Two year 60.77
Name: MonthlyCharges, dtype: float64
Contract with highest average monthly charges: Month-to-month

5. Churn rate by Internet Service (%):
Churn No Yes
InternetService
DSL 81.04 18.96
Fiber optic 58.11 41.89
No 92.60 7.40
Internet Service with highest churn: Fiber optic

6. Average monthly charges by churn status:
Churn
No 61.27
Yes 74.44
Name: MonthlyCharges, dtype: float64

7. Numerical feature correlations with encoded Churn:

```
Contract          -0.397
tenure            -0.352
OnlineSecurity    -0.289
TechSupport       -0.282
TotalCharges      -0.199
OnlineBackup      -0.196
MonthlyCharges     0.193
PaperlessBilling   0.192
DeviceProtection  -0.178
Dependents        -0.164
Name: Churn, dtype: float64

========== TRAIN-TEST SPLIT ==========
Total samples: 7043
Training samples: 5634
Testing samples: 1409
Training percentage: 79.99 %
Testing percentage: 20.01 %

Final processed dataset saved as CustomerChurn_Preprocessed.csv
Final dataset shape: (7043, 21)
```

Explanation of Experiment 3A: Model Planning and Building – Classification

Name: S. Pargavi
Register Number: 241801196
Subject: AD23532 – Principles of Data Science

This experiment uses the preprocessed Customer Churn dataset to train three machine learning classification models: Logistic Regression, Decision Tree, and Random Forest. The models predict whether a customer will leave the company (churn) or continue using its services.

Step-by-step explanation

Step 1: Import the required libraries

Python

Run

None

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, classification_report
)
```

Explanation:

- `pandas` is used to load and manage the dataset.
- `numpy` is used for numerical operations.
- `matplotlib` and `seaborn` create graphs and visualizations.
- `train_test_split` divides the data into training and testing sets.
- `StandardScaler` standardizes numerical features.
- The three classifier libraries create the machine learning models.
- The metrics functions evaluate the models' prediction performance.

Step 2: Load the dataset

Python

Run

```
None
df = pd.read_csv("CustomerChurn_Preprocessed.csv")

print(df.head())
print(df.shape)
print(df.columns)
```

Explanation:

- `read_csv()` loads the preprocessed dataset created in Experiment 2.
- `head()` displays the first five rows.
- `shape` displays the number of rows and columns.
- `columns` displays the names of all features.

The dataset contains customer information, such as tenure, monthly charges, contract type, and churn status.

Step 3: Prepare the dataset

Python

Run

```
None
if "customerID" in df.columns:
    df = df.drop(columns=["customerID"])

for col in df.select_dtypes(include="object").columns:
    df[col] = pd.factorize(df[col])[0]

df = df.replace([np.inf, -np.inf], np.nan)

df = df.fillna(df.median(numeric_only=True))
```

Explanation:

- `customerID` is removed because it identifies customers rather than describing their behavior.
- `pd.factorize()` converts any remaining text categories into numerical values.
- Infinite values are replaced with missing values.
- Missing numerical values are filled with the median of their respective columns.

Machine learning models generally require numerical input and cannot directly process missing values.

Step 4: Separate features and target

Python

Run

```
None
X = df.drop(columns=["Churn"])
y = df["Churn"]
```

Explanation:

- `X` contains the input features used to predict churn.
- `y` contains the target variable, `Churn`, which the models must predict.

For example, tenure and monthly charges can be input features, while churn status is the output.

Step 5: Split the dataset into training and testing data

Python

Run

```
None
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Explanation:

- `test_size=0.2` reserves 20% of the data for testing.
- The remaining 80% is used for training.
- `random_state=42` makes the split reproducible.

- `stratify=y` helps preserve the proportion of churn and non-churn cases in both sets.

The training data teaches the models. The testing data checks how well they predict unseen examples.

Step 6: Create the Logistic Regression model

Python

Run

None

```
log_model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000)
)

log_model.fit(X_train, y_train)
y_pred_log = log_model.predict(X_test)
```

Explanation:

Logistic Regression estimates the probability that a customer belongs to a particular class, such as churn or no churn.

- `StandardScaler()` standardizes the input features.
- `LogisticRegression()` creates the classifier.
- `fit()` trains the model using the training data.
- `predict()` predicts the churn class for the testing data.

This model provides a useful baseline for comparing the other classifiers.

Step 7: Create the Decision Tree model

Python

Run

None

```
dt_model = DecisionTreeClassifier(
    max_depth=10,
    random_state=42
)

dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)
```

Explanation:

A Decision Tree makes predictions using a sequence of feature-based decisions.

For example, it may split customers according to contract type, tenure, or monthly charges.

- `max_depth=10` limits the tree's depth to help control complexity.
- `fit()` trains the tree.
- `predict()` generates churn predictions for the test set.

Decision Trees are relatively easy to interpret because their decisions can be represented as rules.

Step 8: Create the Random Forest model

Python

Run

None

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)
```

```
y_pred_rf = rf_model.predict(X_test)
```

Explanation:

Random Forest combines predictions from multiple Decision Trees.

- `n_estimators=100` creates an ensemble of 100 trees.
- Each tree learns patterns from the training data.
- The trees' predictions are combined to produce the final classification.

Random Forest can capture complex relationships between customer features and churn.

Step 9: Evaluate the models

Python

Run

None

```
for name, y_pred in [
    ("Logistic Regression", y_pred_log),
    ("Decision Tree", y_pred_dt),
    ("Random Forest", y_pred_rf)
]:
    print("\n", name)
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(
        y_test, y_pred, zero_division=0
    ))
    print("Recall:", recall_score(
        y_test, y_pred, zero_division=0
    ))
```

```
    print("F1-score:", f1_score(
        y_test, y_pred, zero_division=0
    ))
```

Explanation:

The code calculates four performance metrics for each model.

Metric	Meaning
Accuracy	Percentage of all predictions that are correct.
Precision	Of the customers predicted to churn, how many actually churned?
Recall	Of the customers who actually churned, how many were correctly identified?
F1-score	Balance between precision and recall.

The confusion matrix shows correct and incorrect predictions for each class.

For churn prediction, recall is especially useful because it measures how many customers who actually leave were identified by the model.

Step 10: Compare all three models

Python

Run

None

```
results = pd.DataFrame({
    "Model": [
        "Logistic Regression",
```

```
        "Decision Tree",
        "Random Forest"
    ],
    "Accuracy": [
        accuracy_score(y_test, y_pred_log),
        accuracy_score(y_test, y_pred_dt),
        accuracy_score(y_test, y_pred_rf)
    ],
    "Precision": [
        precision_score(y_test, y_pred_log, zero_division=0),
        precision_score(y_test, y_pred_dt, zero_division=0),
        precision_score(y_test, y_pred_rf, zero_division=0)
    ],
    "Recall": [
        recall_score(y_test, y_pred_log, zero_division=0),
        recall_score(y_test, y_pred_dt, zero_division=0),
        recall_score(y_test, y_pred_rf, zero_division=0)
    ],
    "F1-score": [
        f1_score(y_test, y_pred_log, zero_division=0),
        f1_score(y_test, y_pred_dt, zero_division=0),
        f1_score(y_test, y_pred_rf, zero_division=0)
    ]
})

print(results)
```

Explanation:

This creates a table containing the evaluation scores of all three models. Comparing the values makes it easier to identify which model performs best.

The results depend on the dataset and its preprocessing, so the best model cannot be confirmed before running the code.

Step 11: Display the comparison bar chart

Python

Run

```
None
results.set_index("Model").plot(
    kind="bar",
    figsize=(10, 6)
)

plt.title("Comparison of Classification Models")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=15)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

Explanation:

The bar chart compares the accuracy, precision, recall, and F1-score of the three models.

- Taller bars represent higher scores.
- Comparing the bars shows which model performs better for each metric.
- The vertical axis ranges from 0 to 1 because these metrics are expressed as proportions.

Step 12: Display the confusion matrices

Python

Run

```
None
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

predictions = [
    ("Logistic Regression", y_pred_log),
    ("Decision Tree", y_pred_dt),
    ("Random Forest", y_pred_rf)
]

for ax, (name, y_pred) in zip(axes, predictions):
    sns.heatmap(
        confusion_matrix(y_test, y_pred),
        annot=True,
        fmt="d",
        cmap="Blues",
        ax=ax
    )
    ax.set_title(name)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

plt.tight_layout()
plt.show()
```

Explanation:

This displays one confusion matrix for each classifier.

Each matrix compares actual churn labels with predicted labels. It helps identify how many customers were correctly classified and how many were incorrectly predicted.

Step 13: Identify the best-performing model

Python

Run

None

```
best_model = results.loc[
    results["F1-score"].idxmax()
]

print("Best Model:", best_model["Model"])
print("Best F1-score:", best_model["F1-score"])
```

Explanation:

- `idxmax()` finds the row containing the highest F1-score.
- `loc[]` retrieves that model's results.
- The code prints the model name and its F1-score.

This step selects the model with the best balance between precision and recall. Depending on the business objective, recall may also be important when the priority is to identify as many customers at risk of leaving as possible.

Step 14: Identify important features

Python

Run

None

```
if hasattr(log_model[-1], "coef_"):
    importance = np.abs(log_model[-1].coef_[0])
elif hasattr(rf_model, "feature_importances_"):
    importance = rf_model.feature_importances_
else:
    importance = dt_model.feature_importances_
```

```
feature_importance = pd.Series(
    importance,
    index=X.columns
).sort_values(ascending=False)

print(feature_importance.head(10))
```

Explanation:

This code displays the ten most important features according to the selected importance method.

- For Logistic Regression, absolute coefficient magnitudes are used.
- For Random Forest or Decision Tree, feature importance values are used.

These methods provide different kinds of importance estimates, so their values should not be treated as directly equivalent. Feature importance indicates predictive usefulness, not necessarily that a feature causes churn.

Step 15: Give business recommendations

The model results can help a company plan customer-retention strategies. Three possible recommendations are:

1. Target customers at risk of leaving: Use model predictions to identify customers who may need additional support or retention offers.
2. Review contracts and payment methods: Examine the churn rates associated with different contract types and payment methods, then develop suitable retention strategies.
3. Improve customer engagement: Monitor customers with shorter tenure and provide onboarding assistance, service guidance, and timely follow-ups.

These are suggested actions; the actual priorities should be based on the dataset's analysis and model results.

Step 16: Save the comparison results

Python

Run

```
None
results.to_csv("Churn_Model_Comparison.csv", index=False)
print("Model comparison saved successfully.")
```

Explanation:

The comparison table is saved as a CSV file named `Churn_Model_Comparison.csv`. You can open it in Excel or use it in your lab report.

Full code:

```
#S.pargavi#241801196

# ============================================================
# EX3A: MODEL PLANNING AND BUILDING - CLASSIFICATION
# ============================================================

# STEP 1: Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    confusion_matrix,
```

```
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

print("Step 1: Libraries imported successfully")


# STEP 2: Load the preprocessed dataset
df = pd.read_csv("CustomerChurn_Preprocessed.csv")

print("\nFirst 5 records:")
print(df.head())

print("\nDataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())


# STEP 3: Prepare features and target
# Remove customerID because it is only an identifier
df = df.drop(columns=["customerID"], errors="ignore")

# Convert any remaining text columns to numeric
for col in df.select_dtypes(include="object").columns:
    df[col] = pd.factorize(df[col])[0]

# Handle any remaining missing or infinite values
df = df.replace([np.inf, -np.inf], np.nan)
df = df.fillna(df.median(numeric_only=True))

X = df.drop(columns=["Churn"])
y = df["Churn"].astype(int)

print("\nFeatures shape:", X.shape)
print("Target shape:", y.shape)
print("\nChurn class distribution:")
print(y.value_counts())
```

```
# STEP 4: Split into training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))


# STEP 5: Build the classification models
models = {
    "Logistic Regression": make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter=1000, random_state=42)
    ),

    "Decision Tree": DecisionTreeClassifier(
        random_state=42,
        max_depth=10
    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        n_jobs=-1
    )
}


# STEP 6: Train, predict, and evaluate each model
results = []
predictions = {}

for name, model in models.items():
```

```
print("\n" + "=" * 55)
print(name)
print("=" * 55)

# Train the model
model.fit(X_train, y_train)

# Predict churn for test data
y_pred = model.predict(X_test)
predictions[name] = y_pred

# Calculate evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(
    y_test, y_pred, zero_division=0
)
recall = recall_score(
    y_test, y_pred, zero_division=0
)
f1 = f1_score(
    y_test, y_pred, zero_division=0
)

# Display confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, zero_division=0
))

print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))
```

```
    results.append({
        "Model": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1
    })


# STEP 7: Compare all models
results_df = pd.DataFrame(results)

print("\n========== MODEL COMPARISON ==========")
print(results_df.round(4).to_string(index=False))


# STEP 8: Plot confusion matrices
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, (name, y_pred) in zip(axes, predictions.items()):
    cm = confusion_matrix(y_test, y_pred)

    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        ax=ax
    )

    ax.set_title(name)
    ax.set_xlabel("Predicted Label")
    ax.set_ylabel("Actual Label")

plt.tight_layout()
plt.show()


# STEP 9: Compare evaluation metrics using a bar chart
plot_df = results_df.set_index("Model")[
```

```
    ["Accuracy", "Precision", "Recall", "F1-Score"]
]

plot_df.plot(
    kind="bar",
    figsize=(12, 6)
)

plt.title("Classification Model Performance Comparison")
plt.ylabel("Score")
plt.xlabel("Classification Model")
plt.ylim(0, 1.05)
plt.xticks(rotation=15)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()


# STEP 10: Identify the best-performing model
best_index = results_df["F1-Score"].idxmax()
best_model_name = results_df.loc[best_index, "Model"]

print("\n========== BEST MODEL ==========")
print("Best-performing model:", best_model_name)
print("Accuracy:",
      round(results_df.loc[best_index, "Accuracy"], 4))
print("Precision:",
      round(results_df.loc[best_index, "Precision"], 4))
print("Recall:",
      round(results_df.loc[best_index, "Recall"], 4))
print("F1-Score:",
      round(results_df.loc[best_index, "F1-Score"], 4))


# STEP 11: Display feature importance when supported
best_model = models[best_model_name]

if best_model_name == "Logistic Regression":
    importance = np.abs(
        best_model.named_steps["logisticregression"].coef_[0]
```

```
    )
elif hasattr(best_model, "feature_importances_"):
    importance = best_model.feature_importances_
else:
    importance = None

if importance is not None:
    feature_importance = pd.DataFrame({
        "Feature": X.columns,
        "Importance": importance
    }).sort_values("Importance", ascending=False)

    print("\nTop 10 important features:")
    print(feature_importance.head(10).to_string(index=False))

    plt.figure(figsize=(10, 6))
    sns.barplot(
        data=feature_importance.head(10),
        x="Importance",
        y="Feature"
    )
    plt.title(f"Top Features - {best_model_name}")
    plt.tight_layout()
    plt.show()


# STEP 12: Business recommendations
print("\n========== BUSINESS RECOMMENDATIONS ==========")
print("1. Identify high-risk customers and offer personalized retention
plans.")
print("2. Review contract and payment patterns to design suitable
incentives.")
print("3. Improve customer support and engagement for customers at high
risk of churn.")


# STEP 13: Save model comparison results
results_df.to_csv("Churn_Model_Comparison.csv", index=False)

print("\nModel comparison saved as Churn_Model_Comparison.csv")
```

```
print("Experiment completed successfully!")
```

Output:

```
Step 1: Libraries imported successfully

First 5 records:
   customerID  gender  SeniorCitizen  Partner  Dependents    tenure  \
0  7590-VHVEG       0              0        1           0 -1.277445
1  5575-GNVDE       1              0        0           0  0.066327
2  3668-QPYBK       1              0        0           0 -1.236724
3  7795-CFOCW       1              0        0           0  0.514251
4  9237-HQITU       0              0        0           0 -1.236724

  PhoneService     MultipleLines  InternetService OnlineSecurity  ...  \
0           No  No phone service                0            No  ...
1          Yes                No                0           Yes  ...
2          Yes                No                0           Yes  ...
3           No  No phone service                0           Yes  ...
4          Yes                No                1            No  ...

  DeviceProtection TechSupport StreamingTV StreamingMovies Contract  \
0               No          No          No              No        0
1              Yes          No          No              No        1
2               No          No          No              No        0
3              Yes         Yes          No              No        1
4               No          No          No              No        0

   PaperlessBilling PaymentMethod  MonthlyCharges  TotalCharges  Churn
0               Yes             2       -1.160323     -0.994242      0
1                No             3       -0.259629     -0.173244      0
2               Yes             3       -0.362660     -0.959674      1
3                No             0       -0.746535     -0.194766      0
4               Yes             2        0.197365     -0.940470      1

[5 rows x 21 columns]

Dataset shape: (7043, 21)

Column names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService',
'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
```

'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn']

Features shape: (7043, 19)
Target shape: (7043,)

Churn class distribution:
Churn
0 5174
1 1869
Name: count, dtype: int64

Training samples: 5634
Testing samples: 1409

==========
Logistic Regression
==========

Confusion Matrix:
[[919 116]
[169 205]]

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.89	0.87	1035
1	0.64	0.55	0.59	374
accuracy			0.80	1409
macro avg	0.74	0.72	0.73	1409
weighted avg	0.79	0.80	0.79	1409

Accuracy : 0.7977
Precision: 0.6386
Recall : 0.5481
F1-Score : 0.5899

==========
Decision Tree
==========

Confusion Matrix:
[[878 157]
[171 203]]

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.85	0.84	1035
1	0.56	0.54	0.55	374
accuracy			0.77	1409
macro avg	0.70	0.70	0.70	1409
weighted avg	0.76	0.77	0.77	1409

Accuracy : 0.7672
Precision: 0.5639
Recall : 0.5428
F1-Score : 0.5531

==========
Random Forest
==========

Confusion Matrix:
[[925 110]
[187 187]]

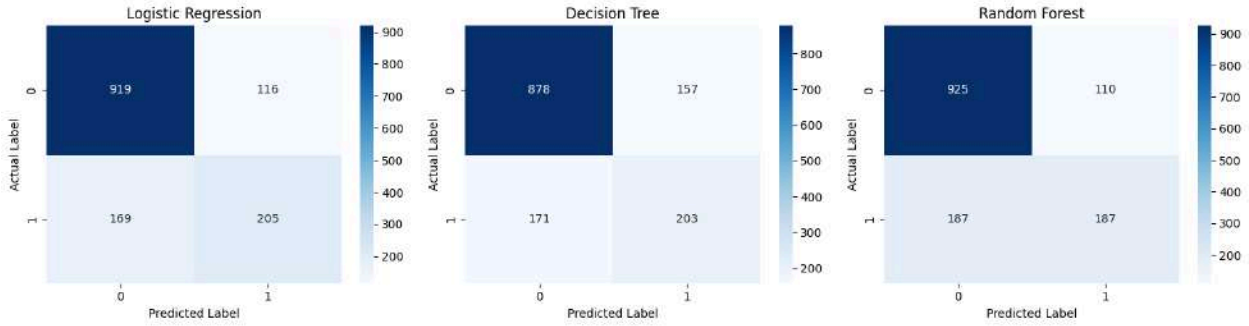
Classification Report:

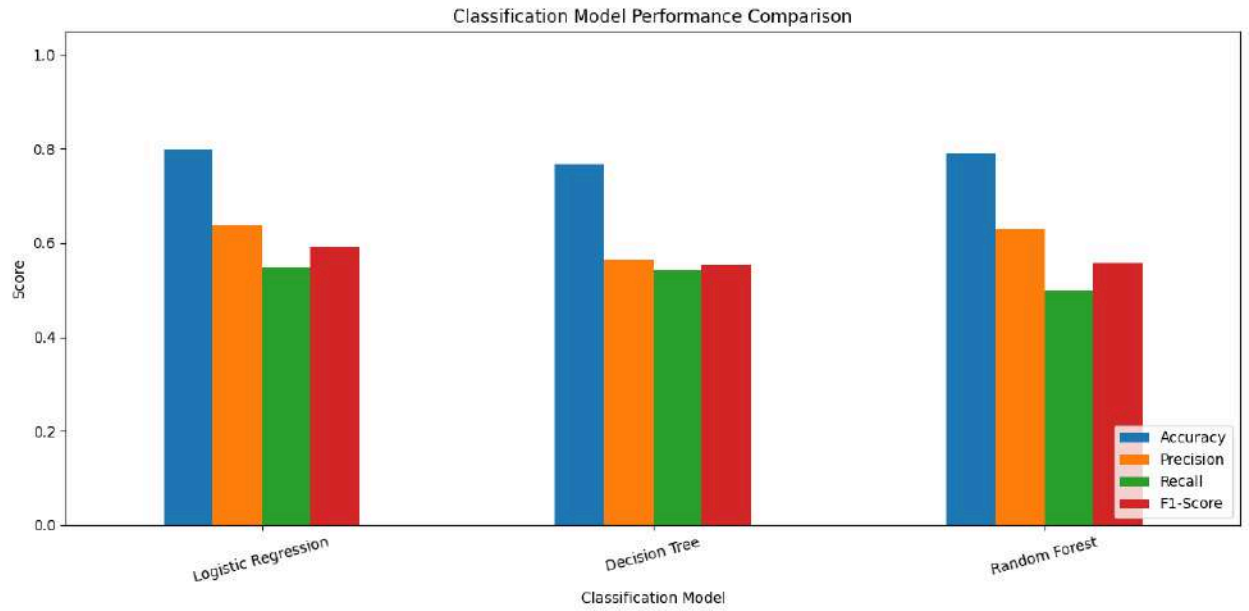
	precision	recall	f1-score	support
0	0.83	0.89	0.86	1035
1	0.63	0.50	0.56	374
accuracy			0.79	1409
macro avg	0.73	0.70	0.71	1409
weighted avg	0.78	0.79	0.78	1409

Accuracy : 0.7892
Precision: 0.6296
Recall : 0.5
F1-Score : 0.5574

========== MODEL COMPARISON ==========

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.7977	0.6386	0.5481	0.5899
Decision Tree	0.7672	0.5639	0.5428	0.5531
Random Forest	0.7892	0.6296	0.5000	0.5574

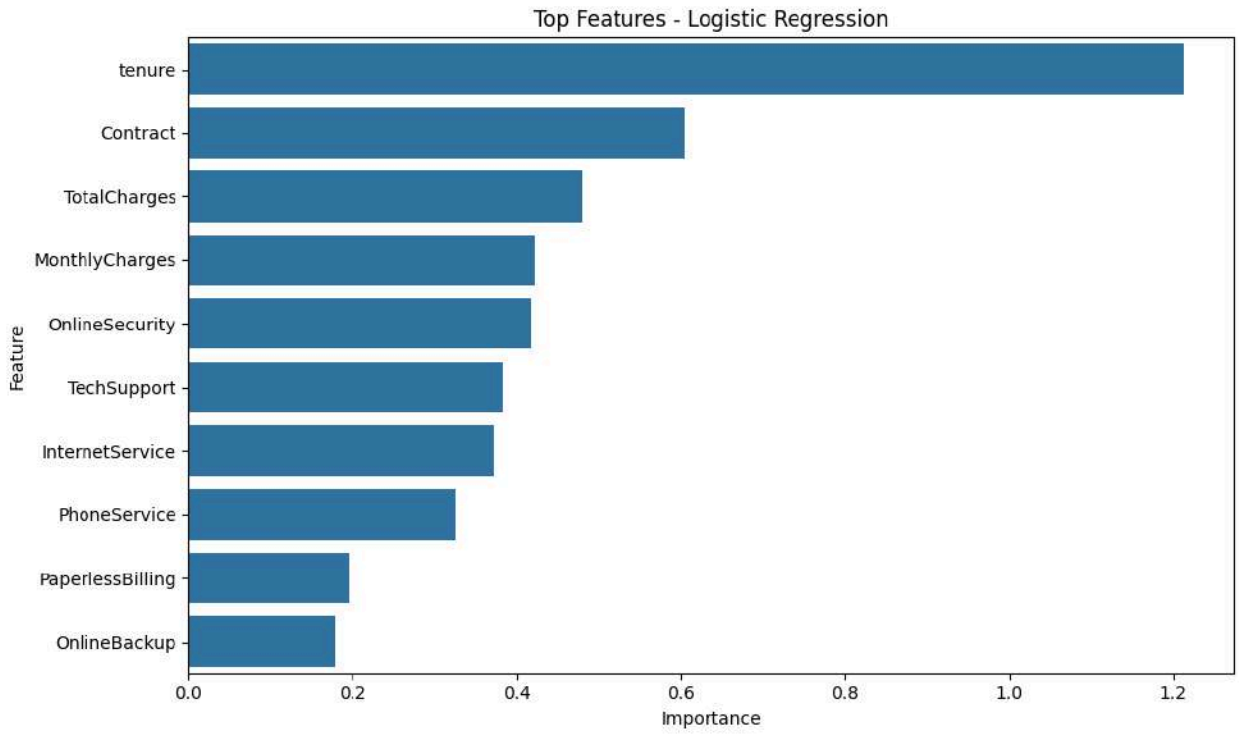




========== BEST MODEL ==========
Best-performing model: Logistic Regression
Accuracy: 0.7977
Precision: 0.6386
Recall: 0.5481
F1-Score: 0.5899

Top 10 important features:

Feature	Importance
tenure	1.212154
Contract	0.604036
TotalCharges	0.479075
MonthlyCharges	0.423346
OnlineSecurity	0.417710
TechSupport	0.382325
InternetService	0.372148
PhoneService	0.325214
PaperlessBilling	0.194448
OnlineBackup	0.178952



========== BUSINESS RECOMMENDATIONS ==========

1. Identify high-risk customers and offer personalized retention plans.
2. Review contract and payment patterns to design suitable incentives.
3. Improve customer support and engagement for customers at high risk of churn.

Model comparison saved as Churn_Model_Comparison.csv

Experiment completed successfully!

Experiment 3B: Model Planning and Building – Regression

Name: S. Pargavi
Register Number: 241801196
Subject: AD23532 – Principles of Data Science

Aim: To build and compare Linear Regression, Polynomial Regression, and Support Vector Regression (SVR) models using the Advertising dataset to predict product sales based on advertising expenditure.

Step-by-step explanation

Step 1: Import the required libraries

The program imports Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn for data processing and visualization. Scikit-learn provides the regression algorithms, data-splitting function, scaling tools, and evaluation metrics.

Step 2: Load and explore the dataset

The Advertising dataset is loaded into a Pandas DataFrame using `pd.read_csv()`.

- `head()` displays the first five records.
- `tail()` displays the last five records.
- `shape` gives the number of rows and columns.
- `dtypes` displays the data types of the columns.
- `isnull().sum()` identifies missing values.
- `duplicated().sum()` counts duplicate records.

- `describe()` displays statistical information such as mean, standard deviation, minimum, and maximum.

This step helps us understand the structure and quality of the dataset.

Step 3: Remove unnecessary columns

The program removes unnecessary index columns such as `Unnamed: 0`, if present.

These columns are not advertising features and are not required for predicting sales.

Step 4: Select input features and target variable

Python

Run

```
None
X = df[["TV", "Radio", "Newspaper"]]
y = df["Sales"]
```

The input features are TV, Radio, and Newspaper advertising expenditures. The target variable is Sales.

The model learns the relationship between advertising expenditure and sales to predict future sales values.

Step 5: Generate a pair plot

The pair plot displays relationships between TV, Radio, Newspaper, and Sales using scatter plots and distributions.

It helps identify possible relationships, trends, and patterns between advertising channels and sales.

Step 6: Plot the correlation heatmap

The correlation heatmap represents the correlation coefficients between numerical variables.

- A positive value indicates that two variables tend to increase together.
- A negative value indicates that one tends to decrease as the other increases.
- A value near zero indicates a weak linear relationship.

The program identifies the advertising feature with the strongest absolute correlation with Sales.

Step 7: Split the dataset

The dataset is divided into training and testing sets using `train_test_split()`.

- 80% training data: Used to train the models.
- 20% testing data: Used to evaluate predictions on unseen data.
- `random_state=42` makes the split reproducible.

The program also displays the total number of samples and the number assigned to each set.

Step 8: Build the Linear Regression model

Linear Regression models the relationship between the advertising features and Sales using a linear equation.

The `fit()` function trains the model, and `predict()` estimates Sales for the testing data.

This model provides a baseline for comparing the other regression methods.

Step 9: Build the Polynomial Regression model

Polynomial Regression uses polynomial features of degree 2 to model nonlinear relationships.

The `PolynomialFeatures(degree=2)` transformation adds squared terms and pairwise interaction terms. Linear Regression then learns from these transformed features.

This model can represent curved relationships that a simple linear model cannot capture.

Step 10: Build the Support Vector Regression model

Support Vector Regression (SVR) uses the Radial Basis Function (RBF) kernel to model potentially nonlinear relationships.

The program first applies `StandardScaler()` to standardize the input features, then trains the SVR model.

Scaling is important because SVR can be sensitive to differences in the numerical ranges of input features.

Step 11: Evaluate the regression models

The program evaluates all three models using four metrics.

Metric	Explanation
MAE	Mean Absolute Error; the average absolute difference between actual and predicted sales.
MSE	Mean Squared Error; the average squared prediction error.
RMSE	Root Mean Squared Error; the square root of MSE, expressed in the target's units.
R^2 Score	Measures how well the model explains variation in Sales compared with predicting the mean.

Interpretation: Lower MAE, MSE, and RMSE generally indicate better predictions. A higher R^2 score generally indicates a better fit.

Step 12: Compare the models using a bar chart

The program creates a bar chart comparing MAE, MSE, RMSE, and R^2 for Linear Regression, Polynomial Regression, and SVR.

This visualization makes it easier to compare the prediction errors and explanatory performance of the models.

Because the metrics use different scales, the bar heights should be interpreted within each metric rather than compared directly across different metrics.

Step 13: Identify the best-performing model

The program selects the model with the lowest RMSE on the testing dataset.

It prints the selected model's name, RMSE, MAE, MSE, and R^2 score.

The actual best model can only be determined after running the code and examining the resulting scores.

Step 14: Plot actual versus predicted Sales

The program plots actual Sales values against predicted Sales values for the selected model.

The red dashed line represents perfect predictions, where predicted Sales equal actual Sales.

- Points close to the line indicate more accurate predictions.
- Points far from the line indicate larger prediction errors.

This graph provides a visual assessment of the selected model's prediction performance.

Step 15: Provide business recommendations

Based on the analysis, the program prints three general recommendations:

1. Allocate more advertising budget to channels that show a strong positive relationship with Sales.
2. Use the best-performing regression model to estimate expected Sales before deciding on advertising expenditure.
3. Regularly compare predicted Sales with actual Sales to assess performance and improve future budget allocation.

These recommendations should be refined using the actual correlation and model results.

Step 16: Save the comparison results

The program saves the evaluation metrics for all three models in Advertising_Model_Comparison.csv.

This file can be opened in Excel or used when preparing the experiment report.

Full code:

```
# S.pargavi#241801196

# Experiment 3B: Model Planning and Building - Regression

# Step 1: Import required libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR

from sklearn.metrics import (
mean_absolute_error,
mean_squared_error,
r2_score
)

# Step 2: Load the Advertising dataset

# Upload Advertising.csv to Google Colab before running this cell.
```

```
df = pd.read_csv("Advertising Budget and Sales.csv")

# Rename columns to match the code expectations
rename_dict = {
    "TV Ad Budget ($)": "TV",
    "Radio Ad Budget ($)": "Radio",
    "Newspaper Ad Budget ($)": "Newspaper",
    "Sales ($)": "Sales"
}
df.rename(columns=rename_dict, inplace=True)

print("FIRST FIVE RECORDS")
print(df.head())

print("\nLAST FIVE RECORDS")
print(df.tail())

print("\nDATASET SHAPE")
print(df.shape)

print("\nDATA TYPES")
print(df.dtypes)

print("\nMISSING VALUES")
print(df.isnull().sum())

print("\nDUPLICATE RECORDS")
print(df.duplicated().sum())

print("\nSUMMARY STATISTICS")
print(df.describe())

# Step 3: Remove unnecessary index columns, if present

df.columns = df.columns.str.strip()

for col in ["Unnamed: 0", "Index", "index"]:
    if col in df.columns:
        df.drop(columns=[col], inplace=True)
```

```
# Step 4: Select input features and target variable

X = df[["TV", "Radio", "Newspaper"]]
y = df["Sales"]

print("\nINPUT FEATURES: TV, Radio, Newspaper")
print(X.head())

print("\nTARGET VARIABLE: Sales")
print(y.head())

# Step 5: Generate a pair plot

sns.pairplot(df[["TV", "Radio", "Newspaper", "Sales"]])
plt.suptitle("Pair Plot of Advertising Dataset", y=1.02)
plt.show()

# Step 6: Plot the correlation heatmap

correlation = df[["TV", "Radio", "Newspaper", "Sales"]].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

# Step 7: Identify the feature most strongly correlated with Sales

sales_correlation = correlation["Sales"].drop("Sales")
strongest_feature = sales_correlation.abs().idxmax()

print("\nCORRELATION WITH SALES")
print(sales_correlation)

print("\nFeature most strongly correlated with Sales:",
strongest_feature)

# Step 8: Split data into training (80%) and testing (20%)
```

```
X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.20,
random_state=42
)

print("\nTotal samples:", len(df))
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 9: Build Linear Regression model

linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

# Step 10: Build Polynomial Regression model (Degree = 2)

polynomial_model = make_pipeline(
PolynomialFeatures(degree=2, include_bias=False),
LinearRegression()
)

polynomial_model.fit(X_train, y_train)
y_pred_poly = polynomial_model.predict(X_test)

# Step 11: Build Support Vector Regression model (RBF kernel)

svr_model = make_pipeline(
StandardScaler(),
SVR(kernel="rbf")
)

svr_model.fit(X_train, y_train)
y_pred_svr = svr_model.predict(X_test)

# Step 12: Evaluate all three models

def evaluate_model(model_name, y_actual, y_predicted):
```

```
    mae = mean_absolute_error(y_actual, y_predicted)
    mse = mean_squared_error(y_actual, y_predicted)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_actual, y_predicted)
    return {
        "Model": model_name,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2 Score": r2
    }

results = pd.DataFrame([
evaluate_model("Linear Regression", y_test, y_pred_linear),
evaluate_model("Polynomial Regression", y_test, y_pred_poly),
evaluate_model("SVR (RBF Kernel)", y_test, y_pred_svr)
])

print("\nMODEL PERFORMANCE COMPARISON")
print(results.round(4).to_string(index=False))

# Step 13: Compare models using a grouped bar chart

metrics = ["MAE", "MSE", "RMSE", "R2 Score"]

results_plot = results.set_index("Model")[metrics]

results_plot.plot(
kind="bar",
figsize=(12, 6)
)

plt.title("Comparison of Regression Models")
plt.ylabel("Metric Value")
plt.xlabel("Regression Model")
plt.xticks(rotation=15)
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```

```
# Step 14: Identify the best-performing model

# Lowest RMSE is used as the primary selection criterion.

best_index = results["RMSE"].idxmin()
best_model_name = results.loc[best_index, "Model"]

predictions = {
"Linear Regression": y_pred_linear,
"Polynomial Regression": y_pred_poly,
"SVR (RBF Kernel)": y_pred_svr
}

best_predictions = predictions[best_model_name]

print("\nBEST-PERFORMING MODEL:", best_model_name)
print("Lowest RMSE:", round(results.loc[best_index, "RMSE"], 4))
print("MAE:", round(results.loc[best_index, "MAE"], 4))
print("MSE:", round(results.loc[best_index, "MSE"], 4))
print("R2 Score:", round(results.loc[best_index, "R2 Score"], 4))

# Step 15: Plot Actual vs Predicted Sales for the best model

plt.figure(figsize=(8, 6))

plt.scatter(
y_test,
best_predictions,
alpha=0.7,
label="Predicted Sales"
)

min_value = min(y_test.min(), best_predictions.min())
max_value = max(y_test.max(), best_predictions.max())

plt.plot(
[min_value, max_value],
[min_value, max_value],
"r--",
label="Perfect Prediction"
```

```
)

plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Actual vs Predicted Sales - " + best_model_name)
plt.legend()
plt.grid(True, linestyle="--", alpha=0.4)
plt.tight_layout()
plt.show()

# Step 16: Display three business recommendations

print("\nBUSINESS RECOMMENDATIONS")

print("1. Allocate more budget to advertising channels that show "
"a strong positive relationship with Sales.")

print("2. Use the best-performing regression model to estimate "
"expected Sales before finalizing advertising expenditure.")

print("3. Review advertising performance regularly and compare "
"predicted Sales with actual Sales to improve budget allocation.")

# Step 17: Save model comparison results

results.to_csv("Advertising_Model_Comparison.csv", index=False)

print("\nModel comparison saved as Advertising_Model_Comparison.csv")
```

Output:

FIRST FIVE RECORDS

	Unnamed: 0	TV	Radio	Newspaper	Sales
0	1	230.1	37.8	69.2	22.1
1	2	44.5	39.3	45.1	10.4
2	3	17.2	45.9	69.3	9.3
3	4	151.5	41.3	58.5	18.5
4	5	180.8	10.8	58.4	12.9

LAST FIVE RECORDS

	Unnamed: 0	TV	Radio	Newspaper	Sales
195	196	38.2	3.7	13.8	7.6
196	197	94.2	4.9	8.1	9.7
197	198	177.0	9.3	6.4	12.8
198	199	283.6	42.0	66.2	25.5
199	200	232.1	8.6	8.7	13.4

DATASET SHAPE

(200, 5)

DATA TYPES

Unnamed: 0 int64
TV float64
Radio float64
Newspaper float64
Sales float64
dtype: object

MISSING VALUES

Unnamed: 0 0
TV 0
Radio 0
Newspaper 0
Sales 0
dtype: int64

DUPLICATE RECORDS

0

SUMMARY STATISTICS

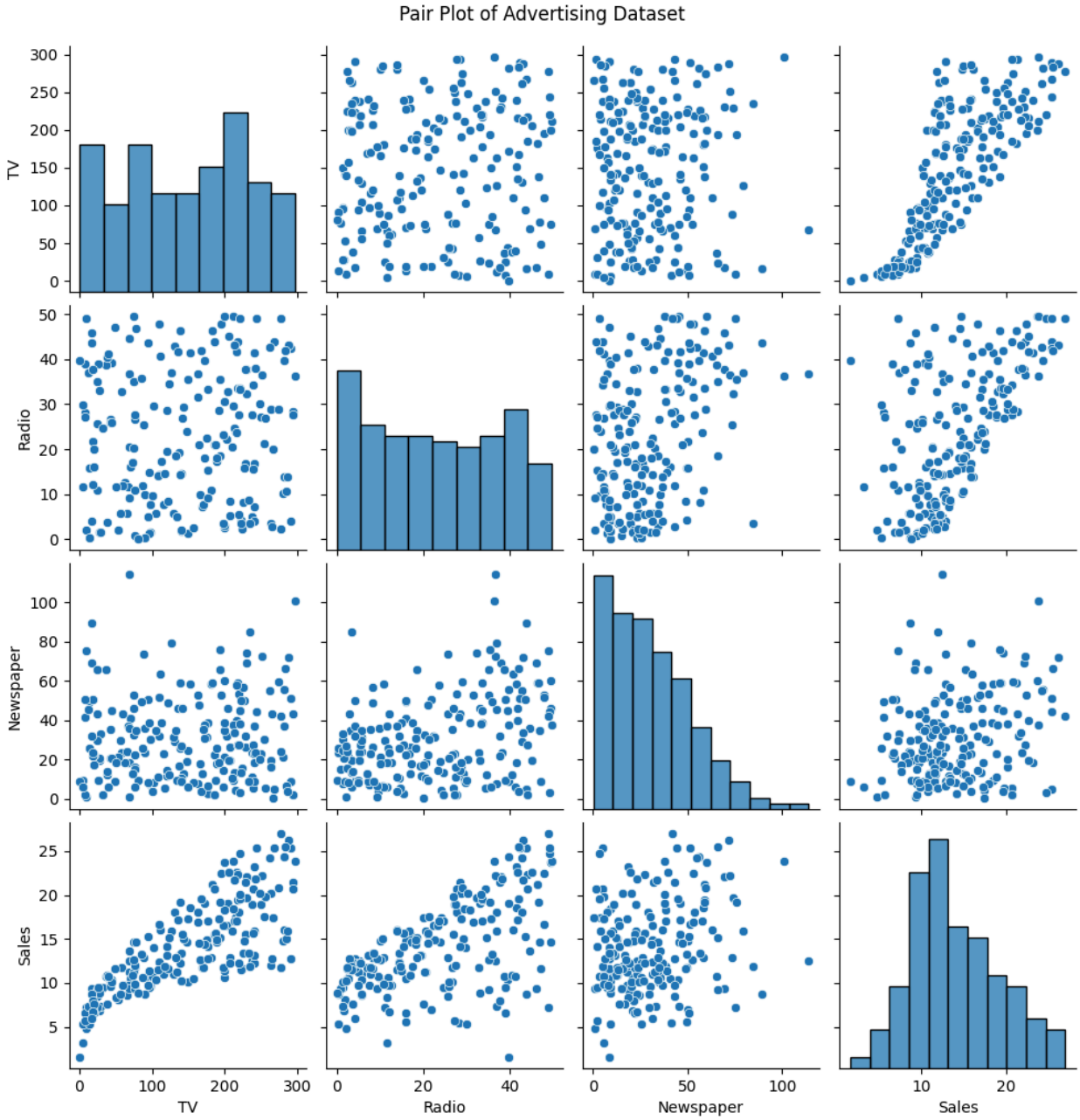
	Unnamed: 0	TV	Radio	Newspaper	Sales
count	200.000000	200.000000	200.000000	200.000000	200.000000
mean	100.500000	147.042500	23.264000	30.554000	14.022500
std	57.879185	85.854236	14.846809	21.778621	5.217457
min	1.000000	0.700000	0.000000	0.300000	1.600000
25%	50.750000	74.375000	9.975000	12.750000	10.375000
50%	100.500000	149.750000	22.900000	25.750000	12.900000
75%	150.250000	218.825000	36.525000	45.100000	17.400000
max	200.000000	296.400000	49.600000	114.000000	27.000000

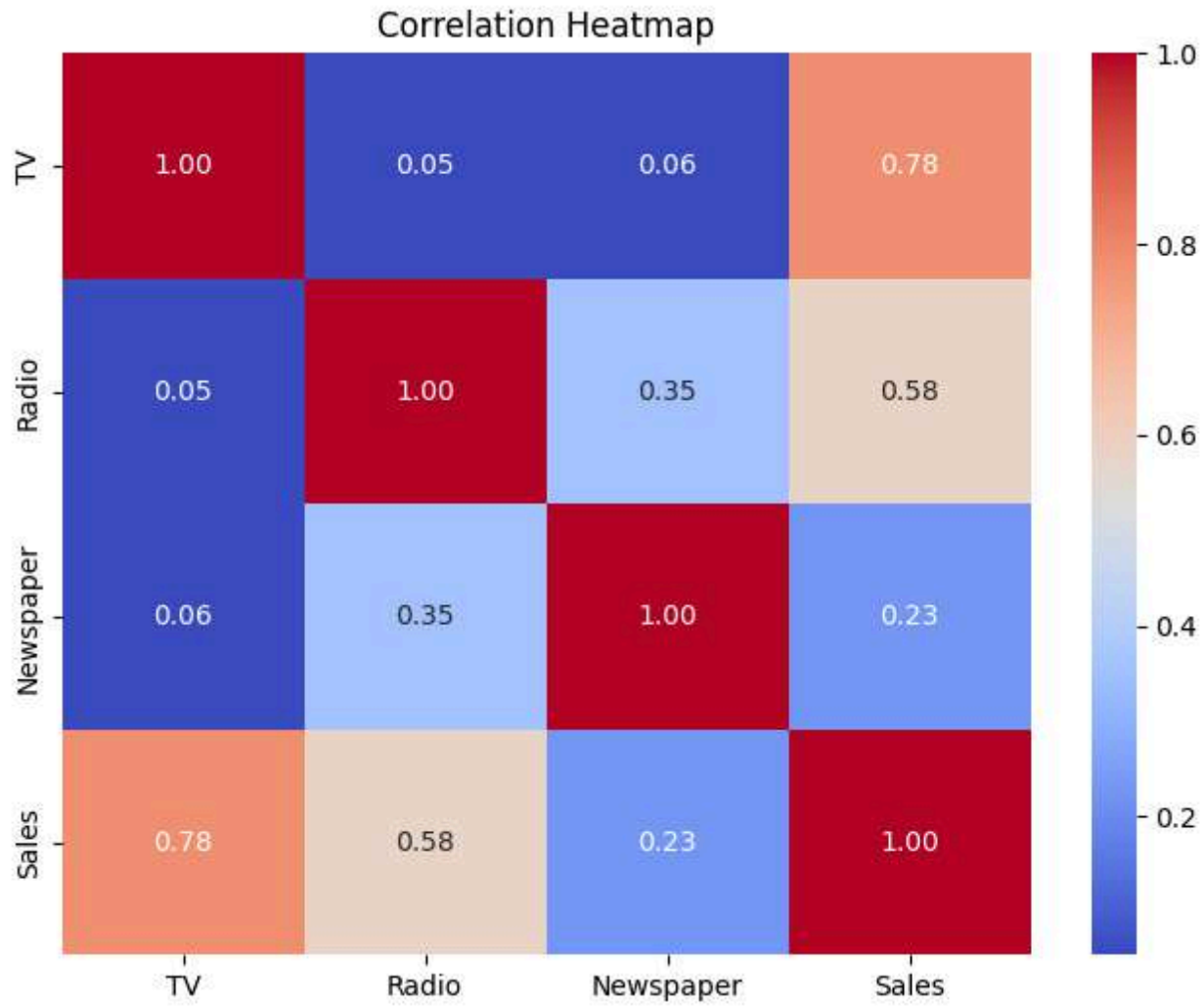
INPUT FEATURES: TV, Radio, Newspaper

	TV	Radio	Newspaper
0	230.1	37.8	69.2
1	44.5	39.3	45.1
2	17.2	45.9	69.3

3 151.5 41.3 58.5
4 180.8 10.8 58.4

TARGET VARIABLE: Sales
0 22.1
1 10.4
2 9.3
3 18.5
4 12.9
Name: Sales, dtype: float64





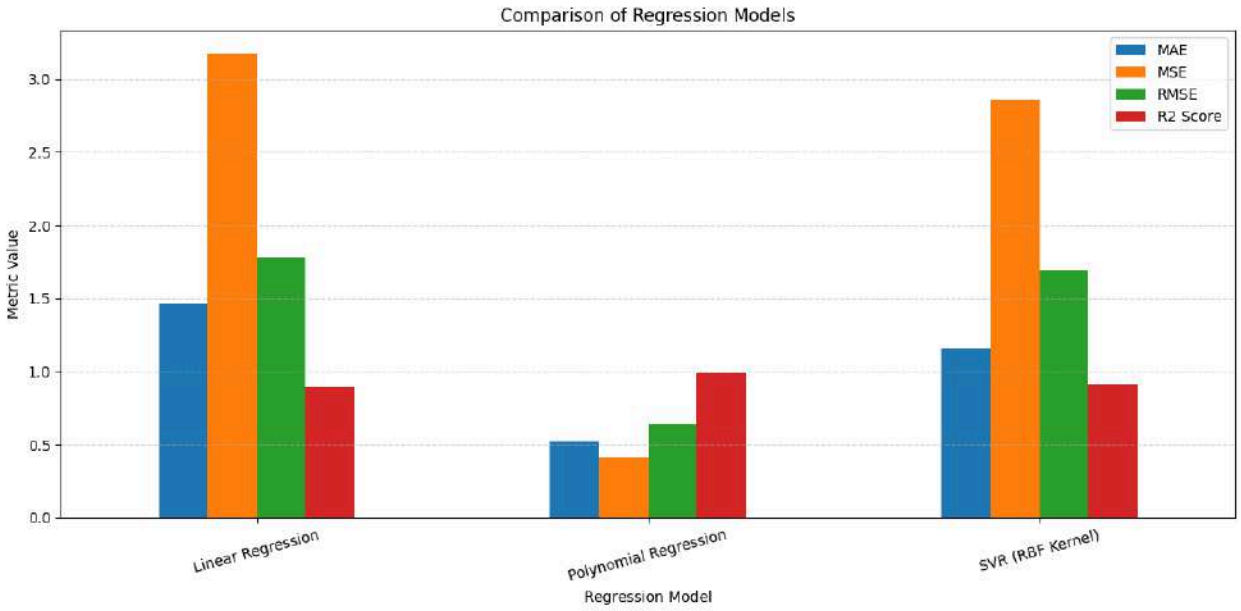
CORRELATION WITH SALES
TV 0.782224
Radio 0.576223
Newspaper 0.228299
Name: Sales, dtype: float64

Feature most strongly correlated with Sales: TV

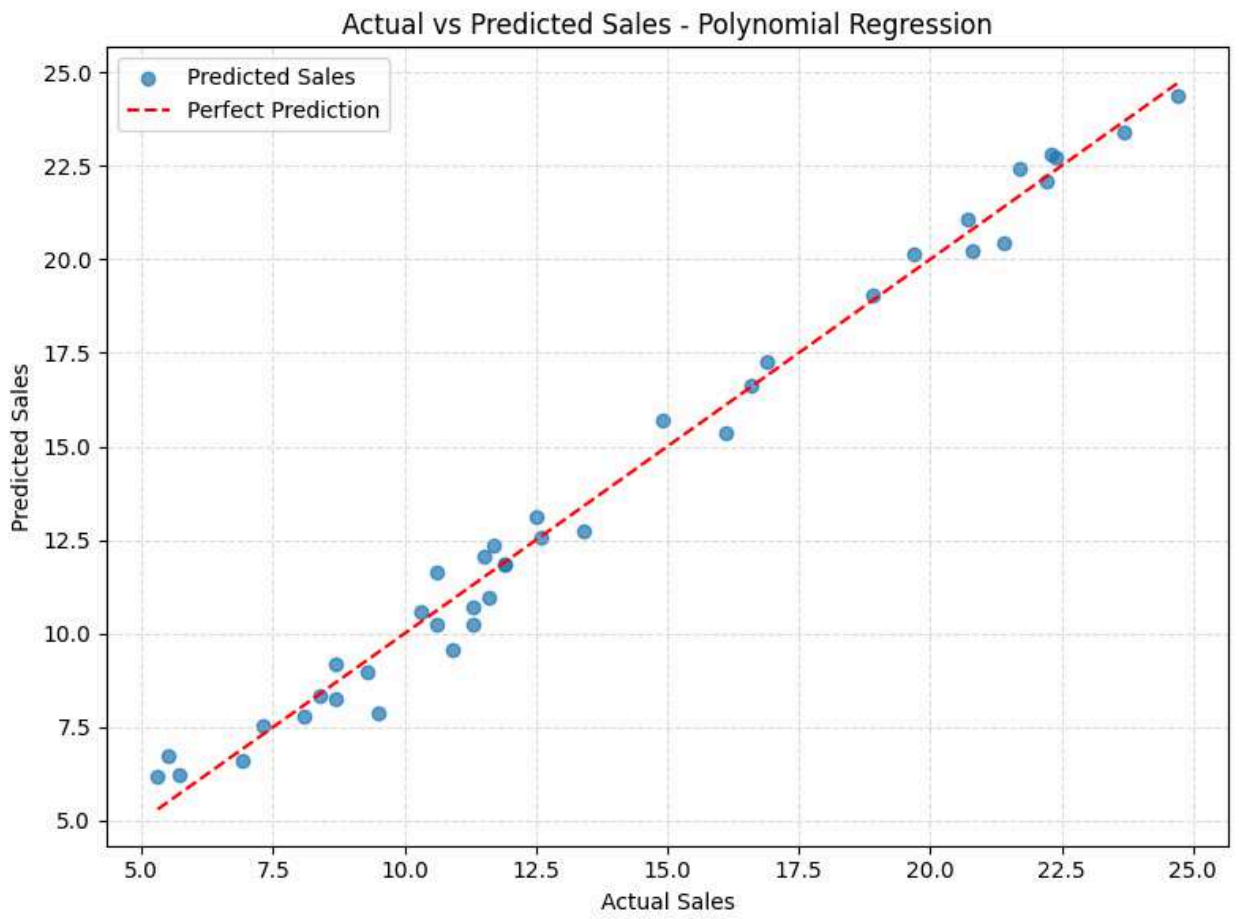
Total samples: 200
Training samples: 160
Testing samples: 40

MODEL PERFORMANCE COMPARISON

Model	MAE	MSE	RMSE	R2 Score
Linear Regression	1.4608	3.1741	1.7816	0.8994
Polynomial Regression	0.5262	0.4129	0.6426	0.9869
SVR (RBF Kernel)	1.1621	2.8621	1.6918	0.9093



BEST-PERFORMING MODEL: Polynomial Regression
Lowest RMSE: 0.6426
MAE: 0.5262
MSE: 0.4129
R2 Score: 0.9869



BUSINESS RECOMMENDATIONS

1. Allocate more budget to advertising channels that show a strong positive relationship with Sales.
2. Use the best-performing regression model to estimate expected Sales before finalizing advertising expenditure.
3. Review advertising performance regularly and compare predicted Sales with actual Sales to improve budget allocation.

Model comparison saved as Advertising_Model_Comparison.csv

S.pargavi

#241801196

Experiment 4: Information Retrieval using NLTK

Experiment 4: Information Retrieval using NLTK

Aim: To develop a basic Information Retrieval (IR) system using NLTK, TF-IDF vectorization, and cosine similarity to retrieve the five most relevant customer reviews for a given search query.

Step-by-step explanation

Step 1: Import the required libraries

The program imports Pandas, NumPy, Regular Expressions (`re`), NLTK, and Scikit-learn.

- `pandas` manages the dataset.
- `numpy` helps identify the reviews with the highest similarity scores.
- `nltk` provides text-processing tools.
- `TfidfVectorizer` converts text into numerical feature vectors.
- `cosine_similarity` measures the similarity between a query and customer reviews.

Step 2: Download NLTK resources

Python

Run

```
None
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
```

These commands download the language resources required for tokenization and stopword removal.

- `punkt` and `punkt_tab` support splitting text into tokens.
- `stopwords` provides common English words, such as the, is, and and, that can be removed during preprocessing.

Step 3: Load the Amazon Fine Food Reviews dataset

Python

Run

```
None
df = pd.read_csv("/content/Reviews.csv")
```

The Amazon Fine Food Reviews dataset is loaded into a Pandas DataFrame.

The program displays the first five records and the dataset shape to understand the dataset's structure and size.

Step 4: Select the review text column

Python

Run

```
None
df = df[["Text"]].copy()
```

The `Text` column contains the customer reviews.

The program selects this column because the goal is to retrieve reviews based on their textual content.

Step 5: Clean the dataset

Python

Run

```
None
df.dropna(subset=["Text"], inplace=True)
df["Text"] = df["Text"].astype(str).str.strip()
df = df[df["Text"] != ""]
df.drop_duplicates(subset=["Text"], inplace=True)
df = df.head(10000).copy()
```

This step prepares the review data for processing.

1. Removes records with missing review text.
2. Converts the text into strings and removes leading or trailing spaces.
3. Removes empty reviews.
4. Removes duplicate reviews.
5. Retains only the first 10,000 reviews to reduce processing time.

Step 6: Load English stopwords

Python

Run

```
None
stop_words = set(stopwords.words("english"))
```

This command loads common English stopwords into a set.

Stopword removal reduces words that often contribute little to matching a query, helping the retrieval system focus on more informative terms.

Step 7: Create the text preprocessing function

Python

Run

```
None
def preprocess_text(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)
    tokens = word_tokenize(text)
    cleaned_tokens = [
        word for word in tokens
        if word not in stop_words
    ]
    return " ".join(cleaned_tokens)
```

This function performs the following operations on each review:

1. Lowercase conversion: Converts all text to lowercase so that words such as `Great` and `great` are treated consistently.
2. Character removal: Removes punctuation, numbers, and special characters.
3. Tokenization: Splits the text into individual words.
4. Stopword removal: Removes common English stopwords.
5. Joining tokens: Combines the remaining words into one cleaned text string.

For example:

- Original review: `Great product! It arrived in 2 days.`
- Cleaned review: `great product arrived days`

Step 8: Apply preprocessing to all reviews

Python

Run

```
None
df["Cleaned_Text"] = df["Text"].apply(preprocess_text)
df = df[df["Cleaned_Text"].str.strip() != ""].reset_index(drop=True)
```

The preprocessing function is applied to every review. The cleaned result is stored in a new column called `Cleaned_Text`.

Reviews that become empty after preprocessing are removed, and the DataFrame index is reset.

The program then displays five original reviews alongside their cleaned versions to demonstrate the preprocessing result.

Step 9: Convert reviews into TF-IDF vectors

Python

Run

```
None
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(df["Cleaned_Text"])
```

TF-IDF stands for Term Frequency–Inverse Document Frequency. It converts text into numerical vectors that represent the importance of words across the collection of reviews.

- Term Frequency (TF): Measures how frequently a term occurs in a document.
- Inverse Document Frequency (IDF): Reduces the relative importance of terms that occur in many documents.
- TF-IDF: Combines these measures to represent terms according to their usefulness for distinguishing documents.

The program displays:

- TF-IDF matrix shape: The number of reviews represented and the number of vocabulary features.

- Vocabulary size: The total number of unique terms learned by the vectorizer.

These numerical vectors allow the system to compare reviews with a search query.

Step 10: Create the information retrieval function

The `retrieve_reviews()` function accepts a user's search query and returns the five most similar reviews.

First, it preprocesses the query using the same function used for the reviews. It then converts the cleaned query into a TF-IDF vector using the already fitted vectorizer.

Next, it computes cosine similarity:

Python

Run

```
None
similarity_scores = cosine_similarity(
    query_vector,
    tfidf_matrix
).flatten()
```

Cosine similarity measures how closely two vectors align. A larger score indicates greater similarity in TF-IDF space.

The program sorts the scores in descending order and selects the top five reviews.

For each retrieved review, it displays:

- Rank
- Similarity score
- Original review
- Cleaned review

The vectorizer is not fitted again for the query, ensuring that the query and review vectors use the same vocabulary.

Step 11: Test the system with sample queries

The program tests these four queries from the experiment:

1. great product with fast shipping
2. disappointed
3. excellent taste
4. poor packaging

For each query, the retrieval function returns the five reviews with the highest cosine similarity scores.

This demonstrates how the system responds to different types of customer feedback searches.

Step 12: Save the cleaned dataset

Python

Run

```
None
df.to_csv("Cleaned_Amazon_Reviews.csv", index=False)
```

This command saves the original review text and the corresponding cleaned text in a CSV file named `Cleaned_Amazon_Reviews.csv`.

The file can be used later for further analysis or information retrieval experiments.

FullCode:

```
# Step 1: Install and import required libraries

!pip install -q nltk scikit-learn pandas
```

```
import pandas as pd
import numpy as np
import re
import string
import nltk

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Step 2: Download required NLTK resources

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

# Step 3: Load the Amazon Fine Food Reviews dataset

# Download Reviews.csv from Kaggle and upload it to Colab first.
# We only need the first 10,000 reviews, so we use nrows=15000 to avoid
reading the corrupted EOF lines at the end of the file.
df = pd.read_csv("/content/Reviews.csv", nrows=15000, on_bad_lines='skip')

print("First five records:")
print(df.head())

print("\nDataset shape:", df.shape)

# Step 4: Select the review text column

print("\nAvailable columns:")
print(df.columns.tolist())

if "Text" not in df.columns:
    raise ValueError("The dataset must contain a column named 'Text'.")

df = df[["Text"]].copy()

# Step 5: Clean the dataset
```

```
df.dropna(subset=["Text"], inplace=True)
df["Text"] = df["Text"].astype(str).str.strip()
df = df[df["Text"] != ""]
df.drop_duplicates(subset=["Text"], inplace=True)

# Retain the first 10,000 reviews

df = df.head(10000).copy()
df.reset_index(drop=True, inplace=True)

print("\nDataset after cleaning:")
print("Number of reviews:", len(df))
print(df.head())

# Step 6: Download and load English stopwords

stop_words = set(stopwords.words("english"))

# Step 7: Create the text preprocessing function

def preprocess_text(text):
    # Convert text to lowercase
    text = text.lower()

    # Remove punctuation, numbers, and special characters
    text = re.sub(r"[^a-z\s]", " ", text)

    # Tokenize text into words
    tokens = word_tokenize(text)

    # Remove English stopwords
    cleaned_tokens = [
        word for word in tokens
        if word not in stop_words
    ]

    # Join tokens into a single string
    return " ".join(cleaned_tokens)
```

```
# Step 8: Apply preprocessing to all reviews

df["Cleaned_Text"] = df["Text"].apply(preprocess_text)

# Remove reviews that became empty after preprocessing

df = df[df["Cleaned_Text"].str.strip() != ""].reset_index(drop=True)

print("\nOriginal and cleaned versions of five reviews:")
for i in range(min(5, len(df))):
    print("\nReview", i + 1)
    print("Original:", df.loc[i, "Text"])
    print("Cleaned:", df.loc[i, "Cleaned_Text"])

# Step 9: Convert reviews into TF-IDF vectors

tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(df["Cleaned_Text"])

print("\nTF-IDF MATRIX INFORMATION")
print("TF-IDF matrix shape:", tfidf_matrix.shape)
print("Vocabulary size:", len(tfidf_vectorizer.vocabulary_))

# Step 10: Create the information retrieval function

def retrieve_reviews(query, top_n=5):
    print("\n" + "=" * 80)
    print("SEARCH QUERY:", query)
    print("=" * 80)

    # Apply the same preprocessing to the query
    cleaned_query = preprocess_text(query)

    if not cleaned_query.strip():
        print("The query contains no searchable words.")
        return

    # Convert the query into a TF-IDF vector
    query_vector = tfidf_vectorizer.transform([cleaned_query])
```

```
    # Compute cosine similarity with all reviews
    similarity_scores = cosine_similarity(
        query_vector,
        tfidf_matrix
    ).flatten()

    # Retrieve the indices of the top similar reviews
    top_indices = np.argsort(similarity_scores)[::-1][:top_n]

    # Display the retrieved reviews
    for rank, index in enumerate(top_indices, start=1):
        print(f"\nRank: {rank}")
        print(f"Similarity Score: {similarity_scores[index]:.4f}")
        print("Original Review:")
        print(df.iloc[index]["Text"])
        print("\nCleaned Review:")
        print(df.iloc[index]["Cleaned_Text"])
        print("-" * 80)

# Step 11: Test the system using the required sample queries

sample_queries = [
    "great product with fast shipping",
    "disappointed",
    "excellent taste",
    "poor packaging"
]

for query in sample_queries:
    retrieve_reviews(query, top_n=5)

# Step 12: Save the processed dataset

df.to_csv("Cleaned_Amazon_Reviews.csv", index=False)
print("\nCleaned dataset saved as Cleaned_Amazon_Reviews.csv")
```

Output:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
First five records:
   Id   ProductId          UserId                      ProfileName  \
0   1  B001E4KFG0  A3SGXH7AUHU8GW                       delmartian
1   2  B00813GRG4  A1D87F6ZCVE5NK                           dll pa
2   3  B000LQOCH0   ABXLMWJIXXAIN  Natalia Corres "Natalia Corres"
3   4  B000UA0QIQ  A395BORC6FGVXV                             Karl
4   5  B006K2ZZ7K  A1UQRSCLF8GW1T    Michael D. Bigham "M. Wassir"

   HelpfulnessNumerator  HelpfulnessDenominator  Score        Time  \
0                     1                       1      5  1303862400
1                     0                       0      1  1346976000
2                     1                       1      4  1219017600
3                     3                       3      2  1307923200
4                     0                       0      5  1350777600

                Summary
Text
0  Good Quality Dog Food  I have bought several of the Vitality canned
d...
1      Not as Advertised  Product arrived labeled as Jumbo Salted
Peanut...
2  "Delight" says it all  This is a confection that has been around a
fe...
3         Cough Medicine  If you are looking for the secret ingredient
i...
4            Great taffy  Great taffy at a great price.  There was a
wid...

Dataset shape: (15000, 10)

Available columns:
['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerator',
'HelpfulnessDenominator', 'Score', 'Time', 'Summary', 'Text']

Dataset after cleaning:
Number of reviews: 10000
                                                Text
0  I have bought several of the Vitality canned d...
1  Product arrived labeled as Jumbo Salted Peanut...
```

2 This is a confection that has been around a fe...
3 If you are looking for the secret ingredient i...
4 Great taffy at a great price. There was a wid...

Original and cleaned versions of five reviews:

Review 1
Original: I have bought several of the Vitality canned dog food products and have found them all to be of good quality. The product looks more like a stew than a processed meat and it smells better. My Labrador is finicky and she appreciates this product better than most.
Cleaned: bought several vitality canned dog food products found good quality product looks like stew processed meat smells better labrador finicky appreciates product better

Review 2
Original: Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized unsalted. Not sure if this was an error or if the vendor intended to represent the product as "Jumbo".
Cleaned: product arrived labeled jumbo salted peanuts peanuts actually small sized unsalted sure error vendor intended represent product jumbo

Review 3
Original: This is a confection that has been around a few centuries. It is a light, pillowy citrus gelatin with nuts - in this case Filberts. And it is cut into tiny squares and then liberally coated with powdered sugar. And it is a tiny mouthful of heaven. Not too chewy, and very flavorful. I highly recommend this yummy treat. If you are familiar with the story of C.S. Lewis' "The Lion, The Witch, and The Wardrobe" - this is the treat that seduces Edmund into selling out his Brother and Sisters to the Witch.
Cleaned: confection around centuries light pillowy citrus gelatin nuts case filberts cut tiny squares liberally coated powdered sugar tiny mouthful heaven chewy flavorful highly recommend yummy treat familiar story c lewis lion witch wardrobe treat seduces edmund selling brother sisters witch

Review 4
Original: If you are looking for the secret ingredient in Robitussin I believe I have found it. I got this in addition to the Root Beer Extract I ordered (which was good) and made some cherry soda. The flavor is very medicinal.
Cleaned: looking secret ingredient robitussin believe found got addition root beer extract ordered good made cherry soda flavor medicinal

Review 5

Original: Great taffy at a great price. There was a wide assortment of yummy taffy. Delivery was very quick. If your a taffy lover, this is a deal.
Cleaned: great taffy great price wide assortment yummy taffy delivery quick taffy lover deal

TF-IDF MATRIX INFORMATION
TF-IDF matrix shape: (10000, 18521)
Vocabulary size: 18521

==========
SEARCH QUERY: great product with fast shipping
==========

Rank: 1
Similarity Score: 0.5103
Original Review:
Enjoyed the product and they also provided very fast shipping. I'm about out and need to order more.

Cleaned Review:
enjoyed product also provided fast shipping need order

Rank: 2
Similarity Score: 0.4925
Original Review:
The tea is good and fresh. We enjoy it. The shipping is fast and cost is reasonable.
I recommend this product.

Cleaned Review:
tea good fresh enjoy shipping fast cost reasonable br recommend product
----

Rank: 3
Similarity Score: 0.4001
Original Review:
These are very good, they were a great price and fast-free shipping when you purchase enough food. The only thing is that the box was smaller than I guessed it would be.

Cleaned Review:

good great price fast free shipping purchase enough food thing box smaller guessed would
---

Rank: 4
Similarity Score: 0.3996
Original Review:
The energy drink is a great product. The shipping price is crazy tho. I paid more for the shipping then both products together, so not a great deal.

Cleaned Review:
energy drink great product shipping price crazy tho paid shipping products together great deal
---

Rank: 5
Similarity Score: 0.3936
Original Review:
Was exactly what I was looking for. Super fast shipping! A bit on the spicy side,but great taste!!!!
Would definately purchase them again!

Cleaned Review:
exactly looking super fast shipping bit spicy side great taste br would definately purchase
---

==========
SEARCH QUERY: disappointed
==========

Rank: 1
Similarity Score: 0.4846
Original Review:
I am a bit disappointed. The flavor was not what I wanted or expected.

Cleaned Review:
bit disappointed flavor wanted expected

Rank: 2

Similarity Score: 0.3582
Original Review:
The product is very good. Way too expensive and at almost 5.00 a box to get pancakes or biscuits I was disappointed

Cleaned Review:
product good way expensive almost box get pancakes biscuits disappointed
---

Rank: 3
Similarity Score: 0.3216
Original Review:
I was disappointed in this product because I thought it would be bigger.
Also, it did not come with enough icing. I had to use my own.

Cleaned Review:
disappointed product thought would bigger br also come enough icing use
---

Rank: 4
Similarity Score: 0.3195
Original Review:
Disappointed. The big boxes had a very different flavor than the smaller boxes. I am not sure why but not good.

Cleaned Review:
disappointed big boxes different flavor smaller boxes sure good
---

Rank: 5
Similarity Score: 0.3117
Original Review:
Just plain nasty!!! This item tasted like cardboard, very watered down and no flavor to it at all. Very disappointed

Cleaned Review:
plain nasty item tasted like cardboard watered flavor disappointed
---

==========
SEARCH QUERY: excellent taste

==========
======

Rank: 1
Similarity Score: 0.5020
Original Review:
It is very very good sweets
Amazon always is excellent
Amazon always is excellent
Amazon always is excellent
Amazon always is excellent

Cleaned Review:
good sweets br amazon always excellent br amazon always excellent br amazon always excellent br amazon always excellent
---

Rank: 2
Similarity Score: 0.4605
Original Review:
The Popchips were excellent - had excellent flavor and low in calories. When you eat these - don't need another potato chip!

Cleaned Review:
popchips excellent excellent flavor low calories eat need another potato chip
---

Rank: 3
Similarity Score: 0.4054
Original Review:
Excellent product. Exactly what they said it would be. Superior chamomile from what I have ordered before from other companies. Excellent value for the price.

Cleaned Review:
excellent product exactly said would superior chamomile ordered companies excellent value price
---

Rank: 4
Similarity Score: 0.3856
Original Review:
We recieved the product on time and in excellent shape. It tastes good and meets our expectations. Thanks for the excellent service.

Cleaned Review:
recieved product time excellent shape tastes good meets expectations thanks excellent service
---

Rank: 5
Similarity Score: 0.3840
Original Review:
Title says it all, a Excellent quality product and Excellent service at a fair price. Product came ahead of time and as expected.

Cleaned Review:
title says excellent quality product excellent service fair price product came ahead time expected
---

==========
SEARCH QUERY: poor packaging
==========

Rank: 1
Similarity Score: 0.3246
Original Review:
I was delighted to find this product online, because I cannot always find it in the stores. However, each package I have opened thus far is at least half reduced to crumbs because of poor packaging for shipping. I doubt that I will order more using this method.

Cleaned Review:
delighted find product online always find stores however package opened thus far least half reduced crumbs poor packaging shipping doubt order using method
---

Rank: 2
Similarity Score: 0.3085
Original Review:
This review is for the packaging and not the actual mix, since the mix is a very subjective thing and my taste my not match others and, some wrote about how poor the packaging was. All I can say is with mine there is more than enough labeling, on the bags themselves and on the box there are instructions on how to. Granted the box is nothing heavy duty or special

and the mix is in plain white bags, there are more than enough instructions to go by.

Cleaned Review:
review packaging actual mix since mix subjective thing taste match others wrote poor packaging say mine enough labeling bags box instructions granted box nothing heavy duty special mix plain white bags enough instructions go
---

Rank: 3
Similarity Score: 0.3084
Original Review:
The soup is rich and tasty, but DON'T buy it from Amazon. The packaging is so poor that the cans arive dented as did the replacement shipment. I got my money back but now have to pay more to buy this wonderful soup from my local grocer.

Cleaned Review:
soup rich tasty buy amazon packaging poor cans arive dented replacement shipment got money back pay buy wonderful soup local grocer
---

Rank: 4
Similarity Score: 0.2920
Original Review:
Didnt really enjoy these after wanting to try something other than aunt jemima's from Dominicks. The pancakes tasted strange and sometimes forgettable, but it might have been due to the poor packaging they came in along with the syrup.

Cleaned Review:
didnt really enjoy wanting try something aunt jemima dominicks pancakes tasted strange sometimes forgettable might due poor packaging came along syrup
---

Rank: 5
Similarity Score: 0.2811
Original Review:
was really dissappointed in product... too salty and poor flavor....Convenience is what makes product worth purchasing if you like pork rinds...

```
Cleaned Review:
really dissappointed product salty poor flavor convenience makes product
worth purchasing like pork rinds
--------------------------------------------------------------------------
------


Cleaned dataset saved as Cleaned_Amazon_Reviews.csv
```

S.pargavi

#241801196

Experiment 5: Information Extraction using spaCy

Experiment 5: Information Extraction using spaCy

Aim: To implement an information extraction system using spaCy to process customer reviews, identify linguistic features, extract named entities, and organize the extracted information into a structured format.

Step-by-step explanation

Step 1: Install and import the required libraries

The program installs and imports spaCy and Pandas.

- spaCy is a Natural Language Processing (NLP) library used for text analysis.
- Pandas is used to load, process, and organize the dataset.
- Counter counts the frequency of different Part-of-Speech (POS) categories.

Step 2: Download the spaCy English language model

Python

Run

None

```
!python -m spacy download en_core_web_sm
```

This command downloads the small English language model used to analyze text.

The model supports tokenization, lemmatization, POS tagging, and Named Entity Recognition (NER).

Step 3: Load the spaCy model

Python

Run

```
None
nlp = spacy.load("en_core_web_sm")
```

The downloaded model is loaded into the variable `nlp`.

When text is passed to `nlp`, spaCy processes it and identifies words, their grammatical roles, and recognized named entities.

Step 4: Load the Amazon Fine Food Reviews dataset

Python

Run

```
None
df = pd.read_csv("/content/Reviews.csv")
```

The Amazon Fine Food Reviews dataset is loaded into a Pandas DataFrame.

The program displays the first five records and the dataset shape to understand the contents and size of the dataset.

Step 5: Clean the dataset

The program selects the `Text` column, removes missing or empty reviews, eliminates duplicate reviews, and retains the first 10,000 reviews.

Explanation:

- Missing reviews cannot provide useful text for extraction.
- Duplicate reviews may cause the same text to be analyzed repeatedly.
- Limiting the dataset to 10,000 reviews reduces processing time.

Step 6: Extract tokens, lemmas, and POS tags

The program applies the spaCy pipeline to five reviews and displays their original text, tokens, lemmas, and POS tags.

- Tokens: Individual words or punctuation units identified in the text.
- Lemmas: The base forms of words. For example, running may become run.
- POS tags: Grammatical categories assigned to tokens, such as nouns, verbs, adjectives, and adverbs.

This step demonstrates how spaCy breaks down and analyzes natural language.

Step 7: Identify major POS categories

The program identifies four major grammatical categories.

POS category	Meaning	Example
Nouns	Name objects, people, or concepts	product, coffee
Verbs	Express actions or states	bought, tasted
Adjectives	Describe nouns	excellent, good
Adverbs	Modify verbs, adjectives, or other adverbs	quickly, very

The code collects words belonging to these categories from the first five reviews and displays them.

In the implementation, proper nouns are grouped with nouns, and auxiliary verbs are grouped with verbs.

Step 8: Count POS category frequencies

The program processes the reviews using `nlp.pipe()` and counts the occurrences of nouns, verbs, adjectives, and adverbs.

The resulting frequency table shows how often each category appears in the dataset.

This helps analyze the linguistic structure of customer feedback and understand which types of words are frequently used.

Step 9: Perform Named Entity Recognition (NER)

Named Entity Recognition identifies named entities in the review text.

The program extracts recognized entities with the following labels:

- `PERSON` — names of people.
- `ORG` — organizations.
- `DATE` — dates or date expressions.
- `MONEY` — monetary values.
- `QUANTITY` — quantities.
- `PRODUCT` — product names, when recognized by the model.

The extracted entities are stored in a structured DataFrame containing the review ID, original review, entity text, and entity label.

Note: The spaCy model may not recognize every entity or every requested label. The code displays only entities that the model actually identifies.

Step 10: Create the information extraction function

The function `extract_information(review)` accepts a customer review as input and processes it using spaCy.

It returns:

1. Original review.

2. Tokens.
3. Lemmas.
4. POS tags.
5. Named entities.
6. Entity labels.

This function makes the extraction process reusable for any new customer review.

Step 11: Test the function with three new reviews

The program tests the extraction function using three sample customer reviews about a product, chocolate, and coffee.

For each review, it displays the original text, tokens, lemmas, POS tags, named entities, and entity labels.

This demonstrates that the same information extraction pipeline can process text beyond the original dataset.

Step 12: Display the extracted information in a structured format

The results from the three new reviews are organized into a Pandas DataFrame.

Each row represents a review, while the columns contain its tokens, lemmas, POS tags, named entities, and entity labels.

A structured format makes the results easier to inspect, analyze, and reuse in other applications.

Step 13: Save the output files

The program saves four CSV files:

- `Cleaned_Amazon_Reviews.csv` — cleaned review text.
- `POS_Frequency.csv` — frequency of POS categories.
- `Extracted_Named_Entities.csv` — entities extracted from the original reviews.

- Information_Extraction_Results.csv — structured results for the three new reviews.

These files can be opened in Excel or used for further customer feedback analysis.

Fullcode:

```
# Step 1: Install and import required libraries

!pip install -q spacy pandas

import pandas as pd
import spacy
from collections import Counter

# Step 2: Download the spaCy English language model

!python -m spacy download en_core_web_sm

# Step 3: Load the spaCy model

nlp = spacy.load("en_core_web_sm")

# Step 4: Load the Amazon Fine Food Reviews dataset

# Upload Reviews.csv to Google Colab before running.

df = pd.read_csv("/content/Reviews.csv")

print("FIRST FIVE RECORDS")
print(df.head())

print("\nDATASET SHAPE")
print(df.shape)

# Step 5: Clean the dataset

if "Text" not in df.columns:
```

```
    raise ValueError("Dataset must contain a 'Text' column.")

df = df[["Text"]].copy()
df.dropna(subset=["Text"], inplace=True)

df["Text"] = df["Text"].astype(str).str.strip()
df = df[df["Text"] != ""]

df.drop_duplicates(subset=["Text"], inplace=True)

# Retain the first 10,000 reviews

df = df.head(10000).reset_index(drop=True)

print("\nCLEANED DATASET")
print("Total reviews:", len(df))
print(df.head())

# Step 6: Apply spaCy to five reviews

print("\nTOKENS, LEMMAS, AND POS TAGS FOR FIVE REVIEWS")

for i, review in enumerate(df["Text"].head(5), start=1):
    doc = nlp(review)
    print(f"\nREVIEW {i}")
    print("Original Review:", review)
    print("Tokens:", [token.text for token in doc])
    print("Lemmas:", [token.lemma_ for token in doc])
    print("POS Tags:", [(token.text, token.pos_) for token in doc])

# Step 7: Identify nouns, verbs, adjectives, and adverbs

print("\nMAJOR POS CATEGORIES")

pos_categories = {
    "Nouns": ["NOUN", "PROPN"],
    "Verbs": ["VERB", "AUX"],
    "Adjectives": ["ADJ"],
    "Adverbs": ["ADV"]
}
```

```
for category, tags in pos_categories.items():
    words = []
    for review in df["Text"].head(5):
        doc = nlp(review)
        words.extend([
            token.text for token in doc
            if token.pos_ in tags
        ])

    print(f"\n{category}:")
    print(words)

# Step 8: Count the frequency of major POS categories

pos_counts = Counter()

for doc in nlp.pipe(df["Text"], batch_size=100):
    for token in doc:
        if token.pos_ in ["NOUN", "PROPN"]:
            pos_counts["Nouns"] += 1
        elif token.pos_ in ["VERB", "AUX"]:
            pos_counts["Verbs"] += 1
        elif token.pos_ == "ADJ":
            pos_counts["Adjectives"] += 1
        elif token.pos_ == "ADV":
            pos_counts["Adverbs"] += 1

pos_frequency = pd.DataFrame(
    pos_counts.items(),
    columns=["POS Category", "Frequency"]
)

print("\nPOS CATEGORY FREQUENCIES")
print(pos_frequency)

# Step 9: Perform Named Entity Recognition (NER)

# Entity types depend on what the model recognizes.
```

```
allowed_entities = {
    "PERSON", "ORG", "DATE", "MONEY",
    "QUANTITY", "PRODUCT"
}

entity_records = []

for review_id, review in enumerate(df["Text"], start=1):
    doc = nlp(review)
    for ent in doc.ents:
        if ent.label_ in allowed_entities:
            entity_records.append({
                "Review_ID": review_id,
                "Review": review,
                "Entity": ent.text,
                "Entity_Label": ent.label_
            })

entity_df = pd.DataFrame(
    entity_records,
    columns=["Review_ID", "Review", "Entity", "Entity_Label"]
)

print("\nEXTRACTED NAMED ENTITIES")
print(entity_df.head(20))

# Step 10: Create an information extraction function

def extract_information(review):
    doc = nlp(review)
    return {
        "Original Review": review,
        "Tokens": [token.text for token in doc],
        "Lemmas": [token.lemma_ for token in doc],
        "POS Tags": [
            (token.text, token.pos_) for token in doc
        ],
        "Named Entities": [
            ent.text for ent in doc.ents
        ],
```

```
        "Entity Labels": [
            ent.label_ for ent in doc.ents
        ]
    }

# Step 11: Test using three new customer reviews

new_reviews = [
    "I bought this product from Amazon and it arrived in two days.",
    "John said the chocolate tasted excellent and cost 15 dollars.",
    "The coffee brand delivered a great product to New York yesterday."
]

extraction_results = []

for i, review in enumerate(new_reviews, start=1):
    result = extract_information(review)
    print(f"\nNEW REVIEW {i}")
    print("Original Review:", result["Original Review"])
    print("Tokens:", result["Tokens"])
    print("Lemmas:", result["Lemmas"])
    print("POS Tags:", result["POS Tags"])
    print("Named Entities:", result["Named Entities"])
    print("Entity Labels:", result["Entity Labels"])

    extraction_results.append({
        "Review_ID": i,
        "Original_Review": result["Original Review"],
        "Tokens": ", ".join(result["Tokens"]),
        "Lemmas": ", ".join(result["Lemmas"]),
        "POS_Tags": str(result["POS Tags"]),
        "Named_Entities": ", ".join(result["Named Entities"]),
        "Entity_Labels": ", ".join(result["Entity Labels"])
    })

# Step 12: Display extracted information in structured format

extraction_df = pd.DataFrame(extraction_results)

print("\nSTRUCTURED INFORMATION EXTRACTION RESULTS")
```

```
print(extraction_df.to_string(index=False))

# Step 13: Save the output files

df.to_csv("Cleaned_Amazon_Reviews.csv", index=False)
pos_frequency.to_csv("POS_Frequency.csv", index=False)
entity_df.to_csv("Extracted_Named_Entities.csv", index=False)
extraction_df.to_csv("Information_Extraction_Results.csv", index=False)

print("\nFiles saved successfully:")
print("1. Cleaned_Amazon_Reviews.csv")
print("2. POS_Frequency.csv")
print("3. Extracted_Named_Entities.csv")
print("4. Information_Extraction_Results.csv")
```

Output:

```
Collecting en-core-web-sm==3.8.0
  Downloading
https://github.com/explosion/spacy-models/releases/download/en_core_web_sm
-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
     ---------------------------------------- 12.8/12.8 MB 32.6 MB/s eta 0:00:00
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python
in
order to load all the package's dependencies. You can do this by selecting
the
'Restart kernel' or 'Restart runtime' option.
FIRST FIVE RECORDS
   Id   ProductId          UserId                      ProfileName  \
0   1  B001E4KFG0  A3SGXH7AUHU8GW                       delmartian
1   2  B00813GRG4  A1D87F6ZCVE5NK                           dll pa
2   3  B000LQOCH0   ABXLMWJIXXAIN  Natalia Corres "Natalia Corres"
3   4  B000UA0QIQ  A395BORC6FGVXV                             Karl
4   5  B006K2ZZ7K  A1UQRSCLF8GW1T    Michael D. Bigham "M. Wassir"

   HelpfulnessNumerator  HelpfulnessDenominator  Score        Time  \
0                     1                       1      5  1303862400
1                     0                       0      1  1346976000
```

```
2                     1                       1      4  1219017600
3                     3                       3      2  1307923200
4                     0                       0      5  1350777600

                 Summary
Text
0  Good Quality Dog Food  I have bought several of the Vitality canned
d...
1      Not as Advertised  Product arrived labeled as Jumbo Salted
Peanut...
2  "Delight" says it all  This is a confection that has been around a
fe...
3         Cough Medicine  If you are looking for the secret ingredient
i...
4            Great taffy  Great taffy at a great price.  There was a
wid...

DATASET SHAPE
(568454, 10)

CLEANED DATASET
Total reviews: 10000
                                                Text
0  I have bought several of the Vitality canned d...
1  Product arrived labeled as Jumbo Salted Peanut...
2  This is a confection that has been around a fe...
3  If you are looking for the secret ingredient i...
4  Great taffy at a great price.  There was a wid...

TOKENS, LEMMAS, AND POS TAGS FOR FIVE REVIEWS

REVIEW 1
Original Review: I have bought several of the Vitality canned dog food
products and have found them all to be of good quality. The product looks
more like a stew than a processed meat and it smells better. My Labrador
is finicky and she appreciates this product better than  most.
Tokens: ['I', 'have', 'bought', 'several', 'of', 'the', 'Vitality',
'canned', 'dog', 'food', 'products', 'and', 'have', 'found', 'them',
'all', 'to', 'be', 'of', 'good', 'quality', '.', 'The', 'product',
'looks', 'more', 'like', 'a', 'stew', 'than', 'a', 'processed', 'meat',
'and', 'it', 'smells', 'better', '.', 'My', 'Labrador', 'is', 'finicky',
'and', 'she', 'appreciates', 'this', 'product', 'better', 'than', ' ',
'most', '.']
Lemmas: ['I', 'have', 'buy', 'several', 'of', 'the', 'Vitality', 'can',
'dog', 'food', 'product', 'and', 'have', 'find', 'they', 'all', 'to',
'be', 'of', 'good', 'quality', '.', 'the', 'product', 'look', 'more',
'like', 'a', 'stew', 'than', 'a', 'process', 'meat', 'and', 'it', 'smell',
```

'well', '.', 'my', 'Labrador', 'be', 'finicky', 'and', 'she',
'appreciate', 'this', 'product', 'well', 'than', ' ', 'most', '.']
POS Tags: [('I', 'PRON'), ('have', 'AUX'), ('bought', 'VERB'), ('several',
'ADJ'), ('of', 'ADP'), ('the', 'DET'), ('Vitality', 'PROPN'), ('canned',
'VERB'), ('dog', 'NOUN'), ('food', 'NOUN'), ('products', 'NOUN'), ('and',
'CCONJ'), ('have', 'AUX'), ('found', 'VERB'), ('them', 'PRON'), ('all',
'PRON'), ('to', 'PART'), ('be', 'AUX'), ('of', 'ADP'), ('good', 'ADJ'),
('quality', 'NOUN'), ('.', 'PUNCT'), ('The', 'DET'), ('product', 'NOUN'),
('looks', 'VERB'), ('more', 'ADV'), ('like', 'ADP'), ('a', 'DET'),
('stew', 'NOUN'), ('than', 'ADP'), ('a', 'DET'), ('processed', 'VERB'),
('meat', 'NOUN'), ('and', 'CCONJ'), ('it', 'PRON'), ('smells', 'VERB'),
('better', 'ADV'), ('.', 'PUNCT'), ('My', 'PRON'), ('Labrador', 'PROPN'),
('is', 'AUX'), ('finicky', 'ADJ'), ('and', 'CCONJ'), ('she', 'PRON'),
('appreciates', 'VERB'), ('this', 'DET'), ('product', 'NOUN'), ('better',
'ADV'), ('than', 'ADP'), (' ', 'SPACE'), ('most', 'ADJ'), ('.', 'PUNCT')]

REVIEW 2
Original Review: Product arrived labeled as Jumbo Salted Peanuts...the
peanuts were actually small sized unsalted. Not sure if this was an error
or if the vendor intended to represent the product as "Jumbo".
Tokens: ['Product', 'arrived', 'labeled', 'as', 'Jumbo', 'Salted',
'Peanuts', '...', 'the', 'peanuts', 'were', 'actually', 'small', 'sized',
'unsalted', '.', 'Not', 'sure', 'if', 'this', 'was', 'an', 'error', 'or',
'if', 'the', 'vendor', 'intended', 'to', 'represent', 'the', 'product',
'as', '"', 'Jumbo', '"', '.']
Lemmas: ['product', 'arrived', 'label', 'as', 'Jumbo', 'Salted',
'Peanuts', '...', 'the', 'peanut', 'be', 'actually', 'small', 'sized',
'unsalte', '.', 'not', 'sure', 'if', 'this', 'be', 'an', 'error', 'or',
'if', 'the', 'vendor', 'intend', 'to', 'represent', 'the', 'product',
'as', '"', 'Jumbo', '"', '.']
POS Tags: [('Product', 'NOUN'), ('arrived', 'AUX'), ('labeled', 'VERB'),
('as', 'ADP'), ('Jumbo', 'PROPN'), ('Salted', 'PROPN'), ('Peanuts',
'PROPN'), ('...', 'PUNCT'), ('the', 'DET'), ('peanuts', 'NOUN'), ('were',
'AUX'), ('actually', 'ADV'), ('small', 'ADJ'), ('sized', 'ADJ'),
('unsalted', 'VERB'), ('.', 'PUNCT'), ('Not', 'PART'), ('sure', 'ADJ'),
('if', 'SCONJ'), ('this', 'PRON'), ('was', 'AUX'), ('an', 'DET'),
('error', 'NOUN'), ('or', 'CCONJ'), ('if', 'SCONJ'), ('the', 'DET'),
('vendor', 'NOUN'), ('intended', 'VERB'), ('to', 'PART'), ('represent',
'VERB'), ('the', 'DET'), ('product', 'NOUN'), ('as', 'ADP'), ('"',
'PUNCT'), ('Jumbo', 'PROPN'), ('"', 'PUNCT'), ('.', 'PUNCT')]

REVIEW 3
Original Review: This is a confection that has been around a few
centuries. It is a light, pillowy citrus gelatin with nuts - in this case
Filberts. And it is cut into tiny squares and then liberally coated with
powdered sugar. And it is a tiny mouthful of heaven. Not too chewy, and
very flavorful. I highly recommend this yummy treat. If you are familiar

with the story of C.S. Lewis' "The Lion, The Witch, and The Wardrobe" - this is the treat that seduces Edmund into selling out his Brother and Sisters to the Witch.
Tokens: ['This', 'is', 'a', 'confection', 'that', 'has', 'been', 'around', 'a', 'few', 'centuries', '.', ' ', 'It', 'is', 'a', 'light', ',', 'pillowy', 'citrus', 'gelatin', 'with', 'nuts', '-', 'in', 'this', 'case', 'Filberts', '.', 'And', 'it', 'is', 'cut', 'into', 'tiny', 'squares', 'and', 'then', 'liberally', 'coated', 'with', 'powdered', 'sugar', '.', ' ', 'And', 'it', 'is', 'a', 'tiny', 'mouthful', 'of', 'heaven', '.', ' ', 'Not', 'too', 'chewy', ',', 'and', 'very', 'flavorful', '.', ' ', 'I', 'highly', 'recommend', 'this', 'yummy', 'treat', '.', ' ', 'If', 'you', 'are', 'familiar', 'with', 'the', 'story', 'of', 'C.S.', 'Lewis', "'", '"', 'The', 'Lion', ',', 'The', 'Witch', ',', 'and', 'The', 'Wardrobe', '"', '-', 'this', 'is', 'the', 'treat', 'that', 'seduces', 'Edmund', 'into', 'selling', 'out', 'his', 'Brother', 'and', 'Sisters', 'to', 'the', 'Witch', '.']
Lemmas: ['this', 'be', 'a', 'confection', 'that', 'have', 'be', 'around', 'a', 'few', 'century', '.', ' ', 'it', 'be', 'a', 'light', ',', 'pillowy', 'citrus', 'gelatin', 'with', 'nut', '-', 'in', 'this', 'case', 'filbert', '.', 'and', 'it', 'be', 'cut', 'into', 'tiny', 'square', 'and', 'then', 'liberally', 'coat', 'with', 'powdered', 'sugar', '.', ' ', 'and', 'it', 'be', 'a', 'tiny', 'mouthful', 'of', 'heaven', '.', ' ', 'not', 'too', 'chewy', ',', 'and', 'very', 'flavorful', '.', ' ', 'I', 'highly', 'recommend', 'this', 'yummy', 'treat', '.', ' ', 'if', 'you', 'be', 'familiar', 'with', 'the', 'story', 'of', 'C.S.', 'Lewis', "'", '"', 'the', 'Lion', ',', 'the', 'Witch', ',', 'and', 'the', 'Wardrobe', '"', '-', 'this', 'be', 'the', 'treat', 'that', 'seduce', 'Edmund', 'into', 'sell', 'out', 'his', 'brother', 'and', 'Sisters', 'to', 'the', 'Witch', '.']
POS Tags: [('This', 'PRON'), ('is', 'AUX'), ('a', 'DET'), ('confection', 'NOUN'), ('that', 'PRON'), ('has', 'AUX'), ('been', 'AUX'), ('around', 'ADP'), ('a', 'DET'), ('few', 'ADJ'), ('centuries', 'NOUN'), ('.', 'PUNCT'), (' ', 'SPACE'), ('It', 'PRON'), ('is', 'AUX'), ('a', 'DET'), ('light', 'ADJ'), (',', 'PUNCT'), ('pillowy', 'ADJ'), ('citrus', 'NOUN'), ('gelatin', 'VERB'), ('with', 'ADP'), ('nuts', 'NOUN'), ('-', 'PUNCT'), ('in', 'ADP'), ('this', 'DET'), ('case', 'NOUN'), ('Filberts', 'NOUN'), ('.', 'PUNCT'), ('And', 'CCONJ'), ('it', 'PRON'), ('is', 'AUX'), ('cut', 'VERB'), ('into', 'ADP'), ('tiny', 'ADJ'), ('squares', 'NOUN'), ('and', 'CCONJ'), ('then', 'ADV'), ('liberally', 'ADV'), ('coated', 'VERB'), ('with', 'ADP'), ('powdered', 'ADJ'), ('sugar', 'NOUN'), ('.', 'PUNCT'), (' ', 'SPACE'), ('And', 'CCONJ'), ('it', 'PRON'), ('is', 'AUX'), ('a', 'DET'), ('tiny', 'ADJ'), ('mouthful', 'ADJ'), ('of', 'ADP'), ('heaven', 'PROPN'), ('.', 'PUNCT'), (' ', 'SPACE'), ('Not', 'PART'), ('too', 'ADV'), ('chewy', 'NOUN'), (',', 'PUNCT'), ('and', 'CCONJ'), ('very', 'ADV'), ('flavorful', 'ADJ'), ('.', 'PUNCT'), (' ', 'SPACE'), ('I', 'PRON'), ('highly', 'ADV'), ('recommend', 'VERB'), ('this', 'DET'), ('yummy', 'ADJ'), ('treat', 'NOUN'), ('.', 'PUNCT'), (' ', 'SPACE'), ('If',

'SCONJ'), ('you', 'PRON'), ('are', 'AUX'), ('familiar', 'ADJ'), ('with', 'ADP'), ('the', 'DET'), ('story', 'NOUN'), ('of', 'ADP'), ('C.S.', 'PROPN'), ('Lewis', 'PROPN'), ("'", 'PUNCT'), ('"', 'PUNCT'), ('The', 'DET'), ('Lion', 'PROPN'), (',', 'PUNCT'), ('The', 'DET'), ('Witch', 'PROPN'), (',', 'PUNCT'), ('and', 'CCONJ'), ('The', 'DET'), ('Wardrobe', 'PROPN'), ('"', 'PUNCT'), ('-', 'PUNCT'), ('this', 'PRON'), ('is', 'AUX'), ('the', 'DET'), ('treat', 'NOUN'), ('that', 'PRON'), ('seduces', 'VERB'), ('Edmund', 'PROPN'), ('into', 'ADP'), ('selling', 'VERB'), ('out', 'ADP'), ('his', 'PRON'), ('Brother', 'NOUN'), ('and', 'CCONJ'), ('Sisters', 'PROPN'), ('to', 'ADP'), ('the', 'DET'), ('Witch', 'PROPN'), ('.', 'PUNCT')]

REVIEW 4
Original Review: If you are looking for the secret ingredient in Robitussin I believe I have found it. I got this in addition to the Root Beer Extract I ordered (which was good) and made some cherry soda. The flavor is very medicinal.
Tokens: ['If', 'you', 'are', 'looking', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'have', 'found', 'it', '.', ' ', 'I', 'got', 'this', 'in', 'addition', 'to', 'the', 'Root', 'Beer', 'Extract', 'I', 'ordered', '(', 'which', 'was', 'good', ')', 'and', 'made', 'some', 'cherry', 'soda', '.', ' ', 'The', 'flavor', 'is', 'very', 'medicinal', '.']
Lemmas: ['if', 'you', 'be', 'look', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'have', 'find', 'it', '.', ' ', 'I', 'get', 'this', 'in', 'addition', 'to', 'the', 'Root', 'Beer', 'Extract', 'I', 'order', '(', 'which', 'be', 'good', ')', 'and', 'make', 'some', 'cherry', 'soda', '.', ' ', 'the', 'flavor', 'be', 'very', 'medicinal', '.']
POS Tags: [('If', 'SCONJ'), ('you', 'PRON'), ('are', 'AUX'), ('looking', 'VERB'), ('for', 'ADP'), ('the', 'DET'), ('secret', 'ADJ'), ('ingredient', 'NOUN'), ('in', 'ADP'), ('Robitussin', 'PROPN'), ('I', 'PRON'), ('believe', 'VERB'), ('I', 'PRON'), ('have', 'AUX'), ('found', 'VERB'), ('it', 'PRON'), ('.', 'PUNCT'), (' ', 'SPACE'), ('I', 'PRON'), ('got', 'VERB'), ('this', 'PRON'), ('in', 'ADP'), ('addition', 'NOUN'), ('to', 'ADP'), ('the', 'DET'), ('Root', 'PROPN'), ('Beer', 'PROPN'), ('Extract', 'PROPN'), ('I', 'PRON'), ('ordered', 'VERB'), ('(', 'PUNCT'), ('which', 'PRON'), ('was', 'AUX'), ('good', 'ADJ'), (')', 'PUNCT'), ('and', 'CCONJ'), ('made', 'VERB'), ('some', 'DET'), ('cherry', 'NOUN'), ('soda', 'NOUN'), ('.', 'PUNCT'), (' ', 'SPACE'), ('The', 'DET'), ('flavor', 'NOUN'), ('is', 'AUX'), ('very', 'ADV'), ('medicinal', 'ADJ'), ('.', 'PUNCT')]

REVIEW 5
Original Review: Great taffy at a great price. There was a wide assortment of yummy taffy. Delivery was very quick. If your a taffy lover, this is a deal.

Tokens: ['Great', 'taffy', 'at', 'a', 'great', 'price', '.', ' ', 'There', 'was', 'a', 'wide', 'assortment', 'of', 'yummy', 'taffy', '.', ' ', 'Delivery', 'was', 'very', 'quick', '.', ' ', 'If', 'your', 'a', 'taffy', 'lover', ',', 'this', 'is', 'a', 'deal', '.']
Lemmas: ['great', 'taffy', 'at', 'a', 'great', 'price', '.', ' ', 'there', 'be', 'a', 'wide', 'assortment', 'of', 'yummy', 'taffy', '.', ' ', 'Delivery', 'be', 'very', 'quick', '.', ' ', 'if', 'your', 'a', 'taffy', 'lover', ',', 'this', 'be', 'a', 'deal', '.']
POS Tags: [('Great', 'ADJ'), ('taffy', 'NOUN'), ('at', 'ADP'), ('a', 'DET'), ('great', 'ADJ'), ('price', 'NOUN'), ('.', 'PUNCT'), (' ', 'SPACE'), ('There', 'PRON'), ('was', 'VERB'), ('a', 'DET'), ('wide', 'ADJ'), ('assortment', 'NOUN'), ('of', 'ADP'), ('yummy', 'ADJ'), ('taffy', 'NOUN'), ('.', 'PUNCT'), (' ', 'SPACE'), ('Delivery', 'PROPN'), ('was', 'AUX'), ('very', 'ADV'), ('quick', 'ADJ'), ('.', 'PUNCT'), (' ', 'SPACE'), ('If', 'SCONJ'), ('your', 'PRON'), ('a', 'DET'), ('taffy', 'PROPN'), ('lover', 'NOUN'), (',', 'PUNCT'), ('this', 'PRON'), ('is', 'AUX'), ('a', 'DET'), ('deal', 'NOUN'), ('.', 'PUNCT')]

MAJOR POS CATEGORIES

Nouns:
['Vitality', 'dog', 'food', 'products', 'quality', 'product', 'stew', 'meat', 'Labrador', 'product', 'Product', 'Jumbo', 'Salted', 'Peanuts', 'peanuts', 'error', 'vendor', 'product', 'Jumbo', 'confection', 'centuries', 'citrus', 'nuts', 'case', 'Filberts', 'squares', 'sugar', 'heaven', 'chewy', 'treat', 'story', 'C.S.', 'Lewis', 'Lion', 'Witch', 'Wardrobe', 'treat', 'Edmund', 'Brother', 'Sisters', 'Witch', 'ingredient', 'Robitussin', 'addition', 'Root', 'Beer', 'Extract', 'cherry', 'soda', 'flavor', 'taffy', 'price', 'assortment', 'taffy', 'Delivery', 'taffy', 'lover', 'deal']

Verbs:
['have', 'bought', 'canned', 'have', 'found', 'be', 'looks', 'processed', 'smells', 'is', 'appreciates', 'arrived', 'labeled', 'were', 'unsalted', 'was', 'intended', 'represent', 'is', 'has', 'been', 'is', 'gelatin', 'is', 'cut', 'coated', 'is', 'recommend', 'are', 'is', 'seduces', 'selling', 'are', 'looking', 'believe', 'have', 'found', 'got', 'ordered', 'was', 'made', 'is', 'was', 'was', 'is']

Adjectives:
['several', 'good', 'finicky', 'most', 'small', 'sized', 'sure', 'few', 'light', 'pillowy', 'tiny', 'powdered', 'tiny', 'mouthful', 'flavorful', 'yummy', 'familiar', 'secret', 'good', 'medicinal', 'Great', 'great', 'wide', 'yummy', 'quick']

Adverbs:

```
['more', 'better', 'better', 'actually', 'then', 'liberally', 'too',
'very', 'highly', 'very', 'very']

POS CATEGORY FREQUENCIES
  POS Category  Frequency
0        Verbs     152765
1   Adjectives      71482
2        Nouns     181239
3      Adverbs      50615

EXTRACTED NAMED ENTITIES
    Review_ID                                             Review  \
0           1  I have bought several of the Vitality canned d...
1           1  I have bought several of the Vitality canned d...
2           2  Product arrived labeled as Jumbo Salted Peanut...
3           3  This is a confection that has been around a fe...
4           3  This is a confection that has been around a fe...
5           3  This is a confection that has been around a fe...
6           3  This is a confection that has been around a fe...
7           3  This is a confection that has been around a fe...
8           3  This is a confection that has been around a fe...
9           5  Great taffy at a great price.  There was a wid...
10          6  I got a wild hair for taffy and ordered this f...
11          6  I got a wild hair for taffy and ordered this f...
12          7  This saltwater taffy had great flavors and was...
13          9  Right now I'm mostly just sprouting this so my...
14         11  I don't know if it's the cactus or the tequila...
15         13  My cats have been happily eating Felidae Plati...
16         13  My cats have been happily eating Felidae Plati...
17         14  good flavor! these came securely packed... the...
18         15  The Strawberry Twizzlers are my guilty pleasur...
19         15  The Strawberry Twizzlers are my guilty pleasur...

                            Entity Entity_Label
0                         Vitality          ORG
1                      My Labrador       PERSON
2                     Jumbo Salted          ORG
3             around a few centuries         DATE
4             pillowy citrus gelatin          ORG
5                      C.S. Lewis'          ORG
6                           Edmund      PRODUCT
7                         Sisters       PERSON
8                           Witch          ORG
9                        Delivery       PERSON
10                       five pound     QUANTITY
11                  only two weeks         DATE
12                     Fralinger's          ORG
```

```
13                          Wheatgrass       PERSON
14  Tequila Picante Gourmet de Inclan          ORG
15                    Felidae Platinum          ORG
16                more than two years         DATE
17                            Twizzlers          ORG
18           The Strawberry Twizzlers          ORG
19                          Six pounds     QUANTITY

NEW REVIEW 1
Original Review: I bought this product from Amazon and it arrived in two
days.
Tokens: ['I', 'bought', 'this', 'product', 'from', 'Amazon', 'and', 'it',
'arrived', 'in', 'two', 'days', '.']
Lemmas: ['I', 'buy', 'this', 'product', 'from', 'Amazon', 'and', 'it',
'arrive', 'in', 'two', 'day', '.']
POS Tags: [('I', 'PRON'), ('bought', 'VERB'), ('this', 'DET'), ('product',
'NOUN'), ('from', 'ADP'), ('Amazon', 'PROPN'), ('and', 'CCONJ'), ('it',
'PRON'), ('arrived', 'VERB'), ('in', 'ADP'), ('two', 'NUM'), ('days',
'NOUN'), ('.', 'PUNCT')]
Named Entities: ['Amazon', 'two days']
Entity Labels: ['ORG', 'DATE']

NEW REVIEW 2
Original Review: John said the chocolate tasted excellent and cost 15
dollars.
Tokens: ['John', 'said', 'the', 'chocolate', 'tasted', 'excellent', 'and',
'cost', '15', 'dollars', '.']
Lemmas: ['John', 'say', 'the', 'chocolate', 'taste', 'excellent', 'and',
'cost', '15', 'dollar', '.']
POS Tags: [('John', 'PROPN'), ('said', 'VERB'), ('the', 'DET'),
('chocolate', 'NOUN'), ('tasted', 'VERB'), ('excellent', 'ADJ'), ('and',
'CCONJ'), ('cost', 'VERB'), ('15', 'NUM'), ('dollars', 'NOUN'), ('.',
'PUNCT')]
Named Entities: ['John', '15 dollars']
Entity Labels: ['PERSON', 'MONEY']

NEW REVIEW 3
Original Review: The coffee brand delivered a great product to New York
yesterday.
Tokens: ['The', 'coffee', 'brand', 'delivered', 'a', 'great', 'product',
'to', 'New', 'York', 'yesterday', '.']
Lemmas: ['the', 'coffee', 'brand', 'deliver', 'a', 'great', 'product',
'to', 'New', 'York', 'yesterday', '.']
POS Tags: [('The', 'DET'), ('coffee', 'NOUN'), ('brand', 'NOUN'),
('delivered', 'VERB'), ('a', 'DET'), ('great', 'ADJ'), ('product',
'NOUN'), ('to', 'ADP'), ('New', 'PROPN'), ('York', 'PROPN'), ('yesterday',
'NOUN'), ('.', 'PUNCT')]
```

Named Entities: ['New York', 'yesterday']
Entity Labels: ['GPE', 'DATE']

STRUCTURED INFORMATION EXTRACTION RESULTS
 Review_ID
Original_Review
Tokens
Lemmas
POS_Tags   Named_Entities Entity_Labels
         1   I bought this product from Amazon and it arrived in two days.   I, bought, this, product, from, Amazon, and, it, arrived, in, two, days, .   I, buy, this, product, from, Amazon, and, it, arrive, in, two, day, . [('I', 'PRON'), ('bought', 'VERB'), ('this', 'DET'), ('product', 'NOUN'), ('from', 'ADP'), ('Amazon', 'PROPN'), ('and', 'CCONJ'), ('it', 'PRON'), ('arrived', 'VERB'), ('in', 'ADP'), ('two', 'NUM'), ('days', 'NOUN'), ('.', 'PUNCT')]   Amazon, two days   ORG, DATE
         2   John said the chocolate tasted excellent and cost 15 dollars.   John, said, the, chocolate, tasted, excellent, and, cost, 15, dollars, .   John, say, the, chocolate, taste, excellent, and, cost, 15, dollar, .   [('John', 'PROPN'), ('said', 'VERB'), ('the', 'DET'), ('chocolate', 'NOUN'), ('tasted', 'VERB'), ('excellent', 'ADJ'), ('and', 'CCONJ'), ('cost', 'VERB'), ('15', 'NUM'), ('dollars', 'NOUN'), ('.', 'PUNCT')]   John, 15 dollars PERSON, MONEY
         3 The coffee brand delivered a great product to New York yesterday. The, coffee, brand, delivered, a, great, product, to, New, York, yesterday, . the, coffee, brand, deliver, a, great, product, to, New, York, yesterday, .   [('The', 'DET'), ('coffee', 'NOUN'), ('brand', 'NOUN'), ('delivered', 'VERB'), ('a', 'DET'), ('great', 'ADJ'), ('product', 'NOUN'), ('to', 'ADP'), ('New', 'PROPN'), ('York', 'PROPN'), ('yesterday', 'NOUN'), ('.', 'PUNCT')] New York, yesterday   GPE, DATE

Files saved successfully:
1. Cleaned_Amazon_Reviews.csv
2. POS_Frequency.csv
3. Extracted_Named_Entities.csv
4. Information_Extraction_Results.csv

S.pargavi

#241801196

Ex6b: Time Series Data Analysis and Forecast

Ex6b: Time Series Data Analysis and Forecast — Explanation

Objective: To analyze monthly airline passenger data, identify trends and seasonal patterns, check stationarity using the Augmented Dickey-Fuller (ADF) test, build an ARIMA forecasting model, and evaluate its prediction accuracy using MSE, MAE, and RMSE.

Step-by-step explanation

Step 1: Import the required libraries

Python

Run

None

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error,
mean_absolute_error
```

Explanation:

- `pandas` is used to load and manipulate time series data.
- `numpy` performs numerical calculations, including square roots and logarithms.
- `matplotlib.pyplot` creates graphs.
- `adfuller` performs the Augmented Dickey-Fuller stationarity test.
- `ARIMA` builds the time series forecasting model.
- `mean_squared_error` and `mean_absolute_error` calculate forecast errors.

The `pip install` command in the program installs the required libraries in Google Colab.

Step 2: Load the airline passenger dataset

Python

Run

```
None
df = pd.read_csv(
    url,
    parse_dates=["Month"],
    index_col="Month"
)
```

Explanation:

- `read_csv()` reads the airline passenger CSV file.
- `parse_dates=["Month"]` converts the Month column into date values.

- `index_col="Month"` sets the month as the DataFrame index.
- This creates a time-indexed dataset that can be analyzed chronologically.

The dataset contains monthly passenger counts from 1949 to 1960.

Step 3: Explore the dataset

Python

Run

```
None
print(df.head())
print(df.tail())
print(df.shape)
print(df.dtypes)
print(df.isnull().sum())
print(df.describe())
```

Explanation:

- `head()` displays the first five observations.
- `tail()` displays the last five observations.
- `shape` shows the number of rows and columns.
- `dtypes` displays the data types.
- `isnull().sum()` checks for missing values.
- `describe()` displays statistics such as mean, standard deviation, minimum, and maximum.

Purpose: To understand the dataset before performing forecasting.

Step 4: Plot the original time series

Python

Run

```
None
plt.figure(figsize=(12, 5))
plt.plot(df.index, df["Passengers"], marker="o")
plt.title("Monthly Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Number of Passengers (Thousands)")
plt.grid(True)
plt.show()
```

Explanation:

- `plt.figure()` sets the graph size.
- `plt.plot()` plots passenger counts against dates.
- `marker="o"` marks the observations.
- `title()`, `xlabel()`, and `ylabel()` add the graph title and axis labels.
- `grid(True)` adds grid lines.

Purpose: To observe the trend, possible seasonal patterns, and unusual changes in passenger numbers.

The original series generally increases over time, so it is unlikely to have a constant mean.

Step 5: Check stationarity using the ADF test

Python

Run

None

```
result = adfuller(df["Passengers"])

print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

Explanation:

The Augmented Dickey-Fuller test checks whether a time series is stationary.

A stationary series has statistical properties, such as its mean, that remain reasonably stable over time.

The test uses the following hypotheses:

- Null hypothesis (H_0): The series has a unit root and is non-stationary.
- Alternative hypothesis (H_1): The series is stationary.

Interpret the p-value as follows:

- If `p-value < 0.05`, reject H_0; the test provides evidence of stationarity.
- If `p-value >= 0.05`, there is insufficient evidence to reject H_0.

The code's `adf_test()` function prints the ADF statistic, p-value, critical values, and interpretation.

Step 6: Apply the logarithmic transformation

Python

Run

```
None
log_series = np.log(df["Passengers"])
```

Explanation:

- `np.log()` calculates the natural logarithm of each passenger count.
- A log transformation can reduce the effect of increasing variance as passenger counts grow.
- It may make the series easier to model.

The transformed series is plotted to compare its pattern with the original series.

Purpose: To stabilize the variability before differencing and forecasting.

Step 7: Difference the series

Python

Run

```
None
first_diff = log_series.diff().dropna()
```

Explanation:

- `diff()` subtracts each previous observation from the current observation.
- `dropna()` removes the first missing value created by differencing.

Mathematically, first differencing is:

$\Delta y_t = y_t - y_{t-1}$

For the log-transformed series:

$\Delta \log(y_t) = \log(y_t) - \log(y_{t-1})$

Differencing helps remove a trend and may make the series stationary.

The code plots the differenced series and applies the ADF test again.

Step 8: Select the differencing order

Python

Run

```
None
second_diff = log_series.diff().diff().dropna()
```

Explanation:

If the first-differenced log series is stationary, the program selects `d = 1`. Otherwise, it tests a second difference and selects `d = 2` if that test indicates stationarity.

The ARIMA model is written as:

$ARIMA(p,d,q)$

- `p`: order of the autoregressive component.
- `d`: number of differences applied.
- `q`: order of the moving-average component.

The code uses `p = 2` and `q = 2`, while selecting `d` using the ADF test.

Note: The code uses `d = 2` as a fallback if neither differenced series passes the ADF test. This does not guarantee stationarity; it is a practical choice for the experiment. Also, the test is applied to the full log series before splitting, so the differencing choice is not strictly training-only.

Step 9: Split the data into training and testing sets

Python

Run

```
None
split_point = int(len(df) * 0.80)

train = df["Passengers"].iloc[:split_point]
test = df["Passengers"].iloc[split_point:]
```

Explanation:

- `len(df)` returns the total number of observations.
- Multiplying by `0.80` selects approximately 80% for training.
- `iloc[:split_point]` selects the earlier observations.
- `iloc[split_point:]` selects the remaining observations for testing.

For the 144-observation dataset:

Dataset	Number of observations
Total	144

Training (80%)	115
Testing (20%)	29

The training set is used to fit the model. The test set is held back to evaluate forecasts on later months.

Step 10: Fit the ARIMA model

Python

Run

None

```
log_train = np.log(train)

model = ARIMA(
    log_train,
    order=(2, d, 2)
)

model_fit = model.fit()

print(model_fit.summary())
```

Explanation:

- `np.log(train)` transforms the training passenger counts.
- `ARIMA(..., order=(2, d, 2))` defines the forecasting model.

- `model.fit()` estimates the model parameters from the training data.
- `summary()` displays the fitted model's statistical details.

The model learns patterns in historical passenger counts that can be used to forecast future values.

Step 11: Forecast the test period

Python

Run

```
None
log_predictions = model_fit.forecast(steps=len(test))
predictions = np.exp(np.asarray(log_predictions))
```

Explanation:

- `forecast(steps=len(test))` forecasts as many months as there are test observations.
- The forecasts are initially on the logarithmic scale because the model was trained on logged values.
- `np.exp()` converts the predictions back to the original passenger-count scale.

The code then creates a DataFrame containing actual and predicted passenger counts so they can be compared month by month.

Step 12: Calculate forecast accuracy

Python

Run

None

```
mse = mean_squared_error(test, predictions)
mae = mean_absolute_error(test, predictions)
rmse = np.sqrt(mse)
```

Explanation:

These three metrics measure how far the predictions are from the actual passenger counts.

Mean Squared Error (MSE)

MSE=1n∑i=1n(yi−y^i)2MSE=\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat y_i)^2MSE=n1i=1∑n(yi−y^i)2

It squares the errors before averaging, giving greater weight to large errors.

Mean Absolute Error (MAE)

MAE=1n∑i=1n|yi−y^i|MAE=\frac{1}{n}\sum_{i=1}^{n}|y_i-\hat y_i|MAE=n1i=1∑n|yi−y^i|

It calculates the average absolute prediction error.

Root Mean Squared Error (RMSE)

RMSE=MSERMSE=\sqrt{MSE}RMSE=MSE

It is expressed in the same units as passenger counts.

Interpretation: Lower values generally indicate better forecasts when comparing models evaluated on the same data.

Step 13: Plot training, testing, and forecast results

Python

Run

```
None
plt.figure(figsize=(14, 6))

plt.plot(train.index, train, label="Training Data")
plt.plot(test.index, test, label="Actual Test Data")
plt.plot(
    predictions.index,
    predictions,
    label="ARIMA Forecast",
    linestyle="--",
    marker="o"
)

plt.title("ARIMA Forecast: Airline Passenger Data")
plt.xlabel("Year")
plt.ylabel("Number of Passengers (Thousands)")
plt.legend()
plt.grid(True)
plt.show()
```

Explanation:

- The first line displays the training data.
- The second line displays actual passenger counts in the test period.
- The third line displays the model's forecast.
- `legend()` identifies each line.
- `grid(True)` improves readability.

Purpose: To visually compare the predicted passenger counts with actual observations and assess how closely the forecasts follow the test data.

Step 14: Save the forecast results

Python

Run

```
None
results.to_csv("Airline_Passenger_Forecast.csv")
```

Explanation:

- `results` contains actual and predicted passenger counts.
- `to_csv()` saves the table to a CSV file.
- The file can be downloaded from Google Colab's Files panel for your assignment.

Fullcode:

```
# Step 1: Import required libraries
!pip install -q pandas numpy matplotlib statsmodels scikit-learn

import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error

# ------------------------------------------------
# STEP 2: LOAD THE DATASET
# ------------------------------------------------

url =
"https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passe
ngers.csv"

df = pd.read_csv(
    url,
    parse_dates=["Month"],
    index_col="Month"
)

df.index = pd.DatetimeIndex(df.index, freq="MS")
df.columns = ["Passengers"]

print("FIRST FIVE ROWS")
print(df.head())

print("\nLAST FIVE ROWS")
print(df.tail())

print("\nDATASET SHAPE:", df.shape)

print("\nDATA TYPES")
print(df.dtypes)

print("\nMISSING VALUES")
print(df.isnull().sum())

print("\nSTATISTICAL SUMMARY")
print(df.describe())

# ------------------------------------------------
# STEP 3: PLOT THE ORIGINAL TIME SERIES
# ------------------------------------------------
```

```
plt.figure(figsize=(12, 5))
plt.plot(df.index, df["Passengers"], marker="o")
plt.title("Monthly Airline Passengers (1949-1960)")
plt.xlabel("Year")
plt.ylabel("Number of Passengers (Thousands)")
plt.grid(True)
plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 4: CHECK STATIONARITY USING ADF TEST
# ------------------------------------------------

def adf_test(series, name):
    series = series.dropna()
    result = adfuller(series)

    print(f"\nADF TEST: {name}")
    print("ADF Statistic:", round(result[0], 4))
    print("p-value:", round(result[1], 4))
    print("Critical Values:")

    for key, value in result[4].items():
        print(f"{key}: {value:.4f}")

    if result[1] < 0.05:
        print("Result: Series is stationary (reject H0).")
        return True
    else:
        print("Result: Series is non-stationary (fail to reject H0).")
        return False

adf_test(df["Passengers"], "Original Series")

# ------------------------------------------------
# STEP 5: APPLY LOG TRANSFORMATION
# ------------------------------------------------

# Log transformation helps stabilize changing variance.
log_series = np.log(df["Passengers"])
```

```
plt.figure(figsize=(12, 5))
plt.plot(log_series.index, log_series)
plt.title("Log-Transformed Airline Passenger Series")
plt.xlabel("Year")
plt.ylabel("Log(Passengers)")
plt.grid(True)
plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 6: DIFFERENCE THE SERIES AND CHECK STATIONARITY
# ------------------------------------------------

first_diff = log_series.diff().dropna()

plt.figure(figsize=(12, 5))
plt.plot(first_diff.index, first_diff)
plt.title("First Difference of Log Passenger Series")
plt.xlabel("Year")
plt.ylabel("Differenced Log Passengers")
plt.grid(True)
plt.tight_layout()
plt.show()

first_stationary = adf_test(
    first_diff,
    "First Difference of Log Series"
)

# ------------------------------------------------
# STEP 7: USE SECOND DIFFERENCING IF REQUIRED
# ------------------------------------------------

second_diff = log_series.diff().diff().dropna()

if first_stationary:
    d = 1
    differenced_series = first_diff
    print("\nSelected differencing order d = 1")
```

```
else:
    second_stationary = adf_test(
        second_diff,
        "Second Difference of Log Series"
    )

    if second_stationary:
        d = 2
        differenced_series = second_diff
        print("\nSelected differencing order d = 2")
    else:
        # Use d=2 as a practical fallback for this dataset.
        d = 2
        differenced_series = second_diff
        print("\nADF did not confirm stationarity after d=2.")
        print("Using d=2 for the ARIMA experiment.")

print("\nSelected ARIMA differencing order:", d)

print("\nDIFFERENCED DATAFRAME")
display(differenced_series.to_frame(name="Differenced_Log_Passengers").hea
d(10))

# ------------------------------------------------
# STEP 8: SPLIT ORIGINAL DATA INTO TRAIN AND TEST
# ------------------------------------------------

split_point = int(len(df) * 0.80)

train = df["Passengers"].iloc[:split_point]
test = df["Passengers"].iloc[split_point:]

print("\nTOTAL SAMPLES:", len(df))
print("TRAINING SAMPLES:", len(train))
print("TESTING SAMPLES:", len(test))

print("\nTRAINING PERIOD:", train.index.min(), "to", train.index.max())
print("TESTING PERIOD:", test.index.min(), "to", test.index.max())

# ------------------------------------------------
```

```
# STEP 9: FIT ARIMA MODEL ON TRAINING DATA
# ------------------------------------------------

# The model uses log-transformed passenger counts.
log_train = np.log(train)

# ARIMA order = (p, d, q)
# p = 2 autoregressive terms
# d = differencing order selected by ADF testing
# q = 2 moving-average terms

model = ARIMA(
    log_train,
    order=(2, d, 2)
)

model_fit = model.fit()

print("\nARIMA MODEL SUMMARY")
print(model_fit.summary())

# ------------------------------------------------
# STEP 10: FORECAST THE TEST PERIOD
# ------------------------------------------------

log_predictions = model_fit.forecast(steps=len(test))

# Convert log predictions back to passenger counts.
predictions = np.exp(np.asarray(log_predictions))

predictions = pd.Series(
    predictions,
    index=test.index,
    name="Predicted_Passengers"
)

print("\nACTUAL VS PREDICTED PASSENGERS")

results = pd.DataFrame({
    "Actual Passengers": test,
```

```
    "Predicted Passengers": predictions
})

display(results.round(2))

# ------------------------------------------------
# STEP 11: EVALUATE FORECAST ACCURACY
# ------------------------------------------------

mse = mean_squared_error(test, predictions)
mae = mean_absolute_error(test, predictions)
rmse = np.sqrt(mse)

print("\nFORECAST EVALUATION METRICS")
print("MSE:", round(mse, 4))
print("MAE:", round(mae, 4))
print("RMSE:", round(rmse, 4))

# ------------------------------------------------
# STEP 12: PLOT TRAIN, TEST AND PREDICTIONS
# ------------------------------------------------

plt.figure(figsize=(14, 6))

plt.plot(
    train.index, train,
    label="Training Data"
)

plt.plot(
    test.index, test,
    label="Actual Test Data"
)

plt.plot(
    predictions.index, predictions,
    label="ARIMA Forecast",
    linestyle="--",
    marker="o"
)
```

```
plt.title("ARIMA Forecast: Airline Passenger Data")
plt.xlabel("Year")
plt.ylabel("Number of Passengers (Thousands)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 13: SAVE FORECAST RESULTS
# ------------------------------------------------

results.to_csv("Airline_Passenger_Forecast.csv")

print("\nFORECAST RESULTS SAVED")
print("File: Airline_Passenger_Forecast.csv")
print("\nTIME SERIES ANALYSIS COMPLETED")
```

Output:

FIRST FIVE ROWS

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

LAST FIVE ROWS

Month	Passengers
1960-08-01	606
1960-09-01	508
1960-10-01	461
1960-11-01	390
1960-12-01	432

DATASET SHAPE: (144, 1)

DATA TYPES
Passengers int64
dtype: object

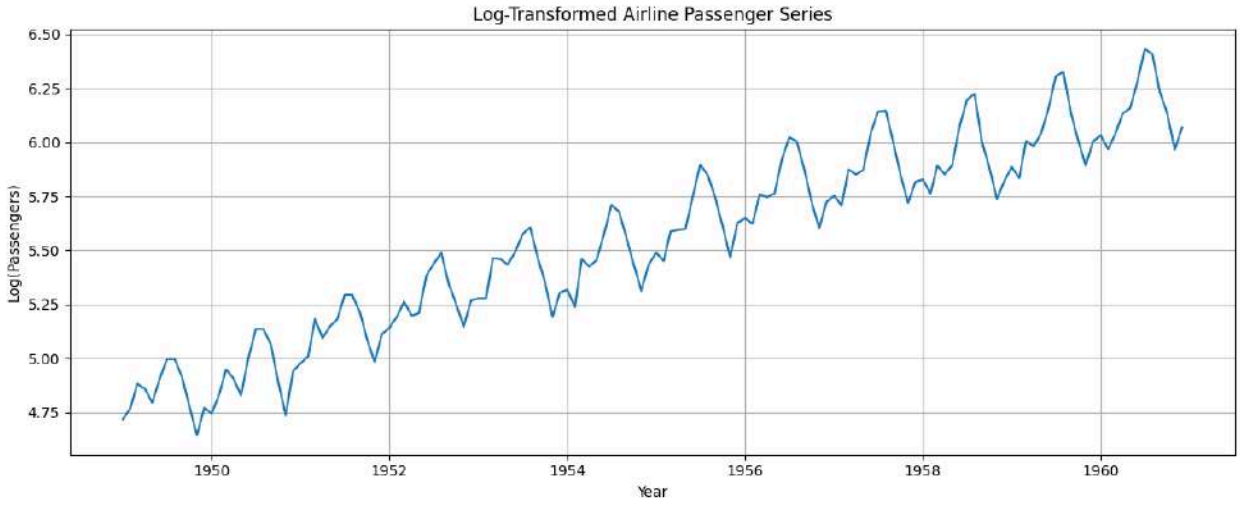
MISSING VALUES
Passengers 0
dtype: int64

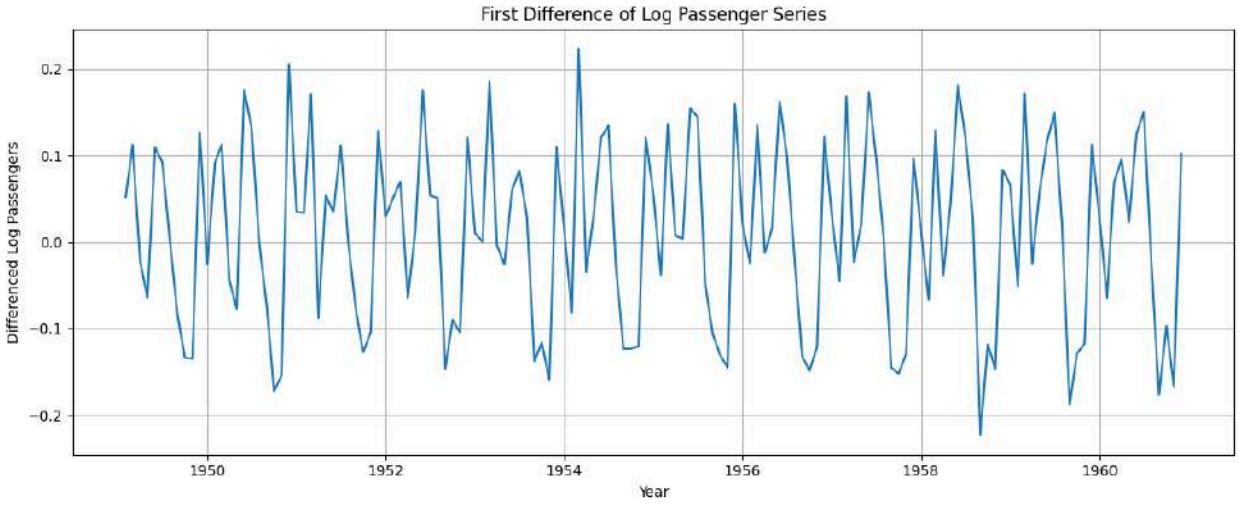
STATISTICAL SUMMARY
Passengers
count 144.000000
mean 280.298611
std 119.966317
min 104.000000
25% 180.000000
50% 265.500000
75% 360.500000
max 622.000000



ADF TEST: Original Series
ADF Statistic: 0.8154
p-value: 0.9919
Critical Values:

1%: -3.4817
5%: -2.8840
10%: -2.5788
Result: Series is non-stationary (fail to reject H0).





ADF TEST: First Difference of Log Series
ADF Statistic: -2.7171
p-value: 0.0711
Critical Values:
1%: -3.4825
5%: -2.8844
10%: -2.5790
Result: Series is non-stationary (fail to reject H0).

ADF TEST: Second Difference of Log Series
ADF Statistic: -8.1966
p-value: 0.0
Critical Values:
1%: -3.4825
5%: -2.8844
10%: -2.5790
Result: Series is stationary (reject H0).

Selected differencing order d = 2

Selected ARIMA differencing order: 2

DIFFERENCED DATAFRAME

Month	Differenced_Log_Passengers
1949-03-01	0.059932
1949-04-01	-0.135107
1949-05-01	-0.041032
1949-06-01	0.173506
1949-07-01	-0.017547
1949-08-01	-0.091937
1949-09-01	-0.084557
1949-10-01	-0.048974
1949-11-01	-0.001201
1949-12-01	0.261026

TOTAL SAMPLES: 144
TRAINING SAMPLES: 115
TESTING SAMPLES: 29

TRAINING PERIOD: 1949-01-01 00:00:00 to 1958-07-01 00:00:00
TESTING PERIOD: 1958-08-01 00:00:00 to 1960-12-01 00:00:00

ARIMA MODEL SUMMARY

SARIMAX Results

Dep. Variable:	Passengers	No. Observations:	115
Model:	ARIMA(2, 2, 2)	Log Likelihood	98.741
Date:	Fri, 09 Oct 2026	AIC	-187.483
Time:	16:39:56	BIC	-173.846
Sample:	01-01-1949 - 07-01-1958	HQIC	-181.949
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	-0.5747	0.198	-2.902	0.004	-0.963	-0.187
ar.L2	-0.0099	0.130	-0.076	0.940	-0.265	0.246
ma.L1	-0.1652	4.496	-0.037	0.971	-8.977	8.647
ma.L2	-0.8339	3.756	-0.222	0.824	-8.195	6.527
sigma2	0.0098	0.044	0.221	0.825	-0.077	0.096

Ljung-Box (L1) (Q):	0.00	Jarque-Bera (JB):	5.61
Prob(Q):	0.95	Prob(JB):	0.06
Heteroskedasticity (H):	1.00	Skew:	0.18
Prob(H) (two-sided):	0.99	Kurtosis:	1.97

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

ACTUAL VS PREDICTED PASSENGERS
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/statespace/mlemodel.py:737:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals

mlefit = super().fit(

Month	Actual Passengers	Predicted Passengers
1958-08-01	505	505.19
1958-09-01	404	506.80
1958-10-01	359	516.34
1958-11-01	310	521.53
1958-12-01	337	529.32
1959-01-01	360	535.79
1959-02-01	342	543.14
1959-03-01	406	550.14
1959-04-01	396	557.48
1959-05-01	420	564.78
1959-06-01	472	572.26
1959-07-01	548	579.78
1959-08-01	559	587.44
1959-09-01	463	595.18
1959-10-01	407	603.03
1959-11-01	362	610.98
1959-12-01	405	619.03
1960-01-01	417	627.19
1960-02-01	391	635.46

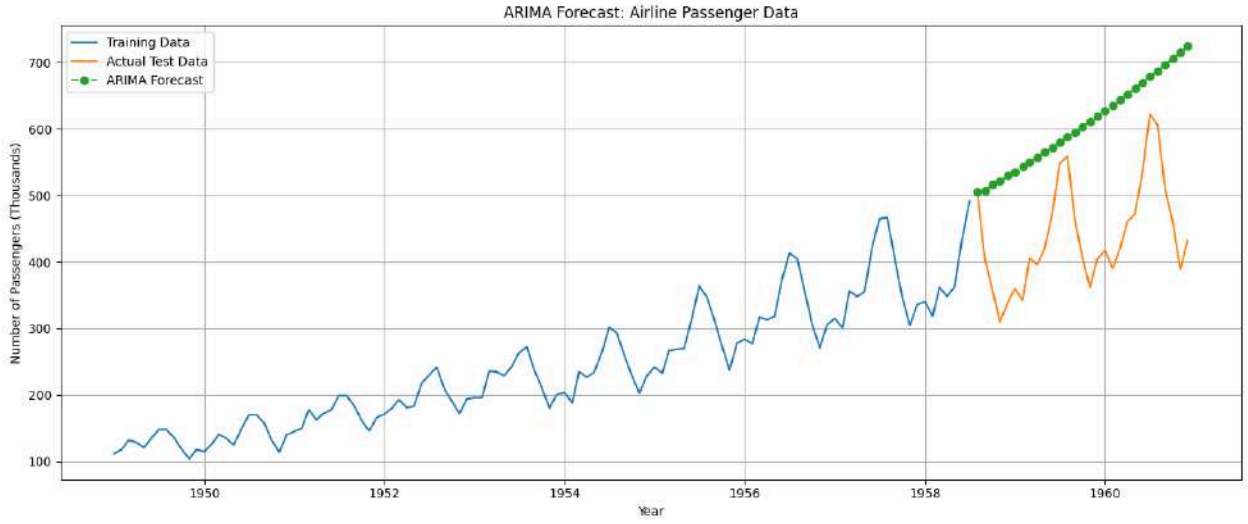
1960-03-01	419	643.84
1960-04-01	461	652.33
1960-05-01	472	660.93
1960-06-01	535	669.64
1960-07-01	622	678.47
1960-08-01	606	687.42
1960-09-01	508	696.48
1960-10-01	461	705.66
1960-11-01	390	714.97
1960-12-01	432	724.39

FORECAST EVALUATION METRICS

MSE: 33552.6662

MAE: 166.4141

RMSE: 183.1739



FORECAST RESULTS SAVED

File: Airline_Passenger_Forecast.csv

TIME SERIES ANALYSIS COMPLETED

S.pargavi

#241801196

Ex6a: Exploratory Data Analysis on Netflix Titles Dataset

Ex6a: Exploratory Data Analysis (EDA) — Explanation

Objective: To perform Exploratory Data Analysis on the Netflix Titles dataset using Python to understand the distribution of movies and TV shows, ratings, release trends, genres, countries, and other important patterns.

Step-by-step explanation of the code

Step 1: Install and import libraries

Python

Run

None

```
!pip install -q wordcloud seaborn

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
from collections import Counter
from wordcloud import WordCloud
```

```
from IPython.display import display
```

Explanation:

- pandas: Loads and analyzes the dataset using DataFrames.
- numpy: Provides numerical operations.
- matplotlib.pyplot: Creates graphs and charts.
- seaborn: Creates statistical visualizations, such as count plots and heatmaps.
- re: Supports pattern matching in text.
- Counter: Counts the frequency of countries, genres, directors, and actors.
- WordCloud: Creates a visual representation of common words.
- display: Displays DataFrames neatly in Google Colab.
- pip install: Installs the required visualization libraries.

Step 2: Load and explore the dataset

Python

Run

None

```
df = pd.read_csv("/content/netflix_titles.csv")

display(df.head())
display(df.tail())

print(df.shape)
df.info()

print(df.isnull().sum())
print(df.duplicated().sum())
```

```
display(df.describe(include="all").T)
```

Explanation:

- `read_csv()` loads the Netflix dataset into a DataFrame called `df`.
- `head()` displays the first five rows.
- `tail()` displays the last five rows.
- `shape` gives the number of rows and columns.
- `info()` displays column names, data types, and non-null counts.
- `isnull().sum()` counts missing values in each column.
- `duplicated().sum()` counts duplicate rows.
- `describe(include="all")` provides summary statistics for numerical and categorical columns.
- `.T` transposes the summary to make it easier to read.

Step 3: Prepare the data

Python

Run

None

```
df = df.drop_duplicates().copy()

df["date_added"] = pd.to_datetime(
    df["date_added"].astype("string").str.strip(),
    errors="coerce"
)

df["release_year"] = pd.to_numeric(
    df["release_year"], errors="coerce"
)

df["duration_num"] = pd.to_numeric(
```

```
    df["duration"].astype("string").str.extract(r"(\d+)")[0],
    errors="coerce"
)
```

Explanation:

- `drop_duplicates()` removes identical rows.
- `copy()` creates a separate DataFrame for further analysis.
- `to_datetime()` converts the date-added column into a date format.
- `str.strip()` removes extra spaces from date values.
- `to_numeric()` converts release years into numeric values.
- `str.extract(r"(\d+)")` extracts the numerical part of duration, such as `120` from `120 min` or `3` from `3 Seasons`.
- `errors="coerce"` converts invalid values into missing values instead of stopping the program.

The helper function:

Python

Run

None

```
def split_values(value):
    if pd.isna(value):
        return []
    return [
        item.strip()
        for item in str(value).split(",")
        if item.strip()
    ]
```

This function separates comma-separated values into individual items. For example, `"India, United States"` becomes `["India", "United States"]`. It is used when counting countries, genres, directors, and actors.

Step 4: Analyze Movies vs TV Shows

Python

Run

```
None
print(df["type"].value_counts())

sns.countplot(
    data=df,
    x="type",
    order=df["type"].value_counts().index
)

plt.title("Movies vs TV Shows")
plt.xlabel("Content Type")
plt.ylabel("Number of Titles")
plt.show()
```

Explanation:

- `value_counts()` counts the number of Movies and TV Shows.
- `countplot()` displays the counts as a bar chart.
- `title()` adds a heading.
- `xlabel()` and `ylabel()` name the axes.
- `show()` displays the graph.

Purpose: To identify which content type is more common in the Netflix dataset.

Step 5: Analyze ratings

Python

Run

None

```
rating_counts = df["rating"].fillna("Not Rated").value_counts()

rating_counts.plot(kind="bar")
plt.title("Distribution of Netflix Ratings")
plt.xlabel("Rating")
plt.ylabel("Number of Titles")
plt.show()
```

Explanation:

- `fillna("Not Rated")` labels missing ratings as `"Not Rated"`.
- `value_counts()` counts each rating, such as TV-MA, PG, and R.
- `plot(kind="bar")` creates a bar chart.

Purpose: To understand the rating categories represented in Netflix content.

Step 6: Find the top 10 countries

Python

Run

None

```
country_counts = Counter()

for countries in df["country"]:
    country_counts.update(split_values(countries))

top_countries = pd.Series(country_counts).sort_values(
    ascending=False
).head(10)
```

Explanation:

- `Counter()` creates a frequency counter.
- The loop processes the country information for every title.
- `split_values()` separates multiple countries listed in one row.
- `update()` increases each country's count.
- `pd.Series()` converts the results into a Pandas Series.
- `sort_values(ascending=False)` sorts counts from highest to lowest.
- `head(10)` selects the top ten countries.

The remaining code displays these results as a horizontal bar chart.

Purpose: To identify countries with the most Netflix title credits. A title associated with multiple countries contributes to each country's count.

Step 7: Analyze release years

Python

Run

```
release_counts = (

df["release_year"].dropna().astype(int).value_counts().sort_index()
)

release_counts.plot()
plt.title("Netflix Titles by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.show()
```

Explanation:

- `dropna()` removes missing release years.
- `astype(int)` converts years to integers.
- `value_counts()` counts the titles for each year.

- `sort_index()` arranges years in chronological order.
- `plot()` displays the yearly trend.

Purpose: To identify years in which many Netflix titles were released.

Step 8: Analyze titles added to Netflix

Python

Run

```
None
added = df.dropna(subset=["date_added"]).copy()
added["year_added"] = added["date_added"].dt.year

year_counts = added["year_added"].value_counts().sort_index()
cumulative_counts = year_counts.cumsum()
```

Explanation:

- `dropna(subset=["date_added"])` keeps rows with a valid date-added value.
- `.dt.year` extracts the year from each date.
- `value_counts()` counts the titles added each year.
- `sort_index()` sorts the years chronologically.
- `cumsum()` calculates a running total of titles added across the years represented in the dataset.

Two graphs are created:

1. Titles added per year: shows the number of titles added during each year.
2. Cumulative content growth: shows the running total of titles added over time.

Purpose: To study Netflix's content-addition trend.

Step 9: Analyze movie durations

Python

Run

```
None
movies = df[df["type"] == "Movie"].copy()
movie_duration = movies["duration_num"].dropna()

plt.hist(movie_duration, bins=30)
plt.title("Distribution of Movie Durations")
plt.xlabel("Duration in Minutes")
plt.ylabel("Number of Movies")
plt.show()
```

Explanation:

- `df[df["type"] == "Movie"]` selects only movies.
- `duration_num` contains the numerical duration values.
- `dropna()` excludes missing durations.
- `plt.hist()` creates a histogram.
- `bins=30` divides the duration range into 30 intervals.

Purpose: To understand the distribution of movie lengths in minutes.

Step 10: Analyze TV shows by seasons

Python

Run

```
None
tv_shows = df[df["type"] == "TV Show"].copy()

season_counts = (
```

```
tv_shows["duration_num"].dropna().astype(int).value_counts().sort_inde
x()
)

season_counts.plot(kind="bar")
plt.title("TV Shows by Number of Seasons")
plt.xlabel("Number of Seasons")
plt.ylabel("Number of TV Shows")
plt.show()
```

Explanation:

- Selects only TV Shows.
- Extracts the numerical season count from the duration column.
- Counts how many shows have each season count.
- Creates a bar chart to compare the number of shows with different season counts.

Purpose: To identify whether single-season or multi-season shows are more common.

Step 11: Analyze the top 10 genres

Python

Run

None
```
genre_counts = Counter()

for genres in df["listed_in"]:
    genre_counts.update(split_values(genres))

top_genres = pd.Series(genre_counts).sort_values(
```

```
    ascending=False
).head(10)
```

Explanation:

- `listed_in` contains genre or category labels.
- The loop extracts all genres from each title.
- `Counter()` calculates genre frequencies.
- Sorting and `head(10)` select the ten most frequent genres.

The results are shown in a horizontal bar chart.

Purpose: To identify the most common content categories on Netflix.

Step 12: Analyze directors

Python

Run

None
```
director_counts = Counter()

for directors in df["director"]:
    director_counts.update(split_values(directors))

top_directors = pd.Series(director_counts).sort_values(
    ascending=False
).head(10)
```

Explanation:

- Reads the director column.
- Separates multiple directors when they appear in one entry.
- Counts each director's appearances.
- Selects the ten most frequent directors.

- Displays them in a horizontal bar chart.

Purpose: To identify directors associated with the greatest number of Netflix titles in the dataset.

Step 13: Analyze actors

Python

Run

None

```
actor_counts = Counter()

for actors in df["cast"]:
    actor_counts.update(split_values(actors))

top_actors = pd.Series(actor_counts).sort_values(
    ascending=False
).head(10)
```

Explanation:

- Reads the cast column.
- Separates the actors listed in each row.
- Counts each actor's appearances across titles.
- Selects and plots the top ten actors.

Purpose: To identify actors who appear most frequently in the dataset. This measures listed appearances, not the number of scenes or screen time.

Step 14: Generate a word cloud

Python

Run

None

```
descriptions = " ".join(
    df["description"].dropna().astype(str)
)

wordcloud = WordCloud(
    width=1200,
    height=600,
    background_color="white",
    max_words=150
).generate(descriptions)

plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("Word Cloud of Netflix Descriptions")
plt.show()
```

Explanation:

- `dropna()` removes missing descriptions.
- `astype(str)` converts descriptions to strings.
- `" ".join()` combines all descriptions into one text.
- `WordCloud()` creates an image based on word frequency.
- `max_words=150` limits the cloud to 150 words.
- `imshow()` displays the word cloud.
- `axis("off")` hides the axes.

Purpose: To identify words frequently used in Netflix descriptions. The code does not remove common stopwords, so frequent words such as “the” or “and” may also appear.

Step 15: Analyze the months when content is added

Python

Run

```
None
month_counts =
added["date_added"].dt.month.value_counts().sort_index()
```

Explanation:

- `.dt.month` extracts the month number from each date.
- `value_counts()` counts how many titles were added in each month.
- `sort_index()` arranges the months from January to December.
- The code maps month numbers to month names and plots the results.

Purpose: To identify which months have the highest number of Netflix additions in the dataset.

Step 16: Compare countries by content type

Python

Run

```
None
country_type_df = pd.DataFrame(country_type_records)

country_type_counts = pd.crosstab(
    country_type_df["Country"],
    country_type_df["Type"]
)
```

Explanation:

- `country_type_records` stores one country and content-type combination per record.
- `pd.DataFrame()` converts those records into a DataFrame.
- `pd.crosstab()` counts Movies and TV Shows for each country.
- The code selects the ten countries with the most total title credits and creates a grouped bar chart.

Purpose: To compare the number of Movies and TV Shows associated with different countries.

Step 17: Compare ratings for Movies and TV Shows

Python

Run

```
None
rating_type = pd.crosstab(
    df["rating"].fillna("Not Rated"),
    df["type"]
)
```

Explanation:

- pd.crosstab() creates a frequency table of ratings against content types.
- Rows represent ratings.
- Columns represent Movies and TV Shows.
- The table and grouped bar chart help compare the rating distributions between the two content types.

Purpose: To determine which ratings are common among Movies and which are common among TV Shows.

Step 18: Find the oldest and newest titles by release year

Python

Run

```
None
valid_release = df.dropna(subset=["release_year"]).copy()

oldest_year = valid_release["release_year"].min()
newest_year = valid_release["release_year"].max()
```

Explanation:

- Removes records without a release year.
- `min()` finds the earliest release year.
- `max()` finds the latest release year.
- Filtering the DataFrame using each year displays the corresponding title or titles.

Purpose: To identify the earliest- and latest-release-year entries in the dataset. This is based on release year, not the date a title was added to Netflix.

Step 19: Create a genre correlation heatmap

Python

Run

None

```
genre_matrix = pd.DataFrame(
    0, index=df.index, columns=top_genre_names
)

for idx, genres in df["listed_in"].items():
    for genre in split_values(genres):
        if genre in genre_matrix.columns:
            genre_matrix.loc[idx, genre] = 1

genre_corr = genre_matrix.corr()

sns.heatmap(
    genre_corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-1,
```

```
    vmax=1
)
plt.title("Genre Co-occurrence Correlation Heatmap")
plt.show()
```

Explanation:

- Creates a matrix with one row per title and one column for each of the top ten genres.
- `0` means the title is not listed in that genre.
- `1` means the title is listed in that genre.
- `corr()` calculates the correlation between genre indicator columns.
- `heatmap()` displays correlations using colors.
- `annot=True` displays the numerical values inside the cells.
- `fmt=".2f"` shows values to two decimal places.

Purpose: To explore which of the selected genres tend to appear together across titles. Because these are binary indicators, the correlation measures association between genre memberships rather than proving that two genres cause each other to occur.

Step 20: Save the results

Python

Run

```
None
df.to_csv("Netflix_EDA_Cleaned.csv", index=False)

top_countries.to_csv("Netflix_Top_Countries.csv")
top_genres.to_csv("Netflix_Top_Genres.csv")

summary.to_csv("Netflix_EDA_Summary.csv", index=False)
```

Explanation:

- `to_csv()` saves the DataFrame or Series as a CSV file.
- `index=False` prevents the DataFrame index from being saved as an extra column.
- The cleaned dataset, top countries, top genres, and summary are saved for later use.

The files will appear in Google Colab's Files panel after the code finishes running.

Fullcode:

```
!pip install -q wordcloud seaborn

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
from collections import Counter
from wordcloud import WordCloud
from IPython.display import display

# ---------------------------------------------------
# STEP 1: LOAD THE DATASET
# ---------------------------------------------------

df = pd.read_csv("/netflix_titles.csv")

print("FIRST FIVE ROWS")
display(df.head())

print("\nLAST FIVE ROWS")
display(df.tail())

print("\nDATASET SHAPE:", df.shape)
```

```
print("\nCOLUMN INFORMATION")
df.info()

print("\nMISSING VALUES")
print(df.isnull().sum())

print("\nDUPLICATE ROWS:", df.duplicated().sum())

print("\nSTATISTICAL SUMMARY")
display(df.describe(include="all").T)

# ----------------------------------------------------
# STEP 2: PREPARE THE DATA
# ----------------------------------------------------

df = df.drop_duplicates().copy()

df["date_added"] = pd.to_datetime(
    df["date_added"].astype("string").str.strip(),
    errors="coerce"
)

df["release_year"] = pd.to_numeric(
    df["release_year"], errors="coerce"
)

df["duration_num"] = pd.to_numeric(
    df["duration"].astype("string").str.extract(r"(\d+)")[0],
    errors="coerce"
)

def split_values(value):
    if pd.isna(value):
        return []
    return [
        item.strip()
        for item in str(value).split(",")
        if item.strip()
    ]
```

```
print("\nDATASET AFTER REMOVING DUPLICATES")
print("Rows and columns:", df.shape)

# ------------------------------------------------
# STEP 3: MOVIES VS TV SHOWS
# ------------------------------------------------

print("\nMOVIES VS TV SHOWS")
print(df["type"].value_counts())

plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="type", order=df["type"].value_counts().index)
plt.title("Movies vs TV Shows")
plt.xlabel("Content Type")
plt.ylabel("Number of Titles")
plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 4: RATINGS DISTRIBUTION
# ------------------------------------------------

rating_counts = df["rating"].fillna("Not Rated").value_counts()

print("\nRATING DISTRIBUTION")
print(rating_counts)

plt.figure(figsize=(10, 5))
rating_counts.plot(kind="bar")
plt.title("Distribution of Netflix Ratings")
plt.xlabel("Rating")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 5: TOP 10 COUNTRIES
# ------------------------------------------------
```

```
country_counts = Counter()

for countries in df["country"]:
    country_counts.update(split_values(countries))

top_countries = pd.Series(country_counts).sort_values(
    ascending=False
).head(10)

print("\nTOP 10 COUNTRIES")
display(top_countries.rename_axis("Country").reset_index(
    name="Title Count"
))

plt.figure(figsize=(10, 5))
top_countries.sort_values().plot(kind="barh")
plt.title("Top 10 Countries by Netflix Titles")
plt.xlabel("Number of Titles")
plt.ylabel("Country")
plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 6: RELEASE YEAR ANALYSIS
# ------------------------------------------------

release_counts = (
    df["release_year"].dropna().astype(int).value_counts().sort_index()
)

print("\nTOP 10 RELEASE YEARS")
display(
    df["release_year"].value_counts().head(10)
    .rename_axis("Release Year")
    .reset_index(name="Title Count")
)

plt.figure(figsize=(12, 5))
release_counts.plot()
```

```
plt.title("Netflix Titles by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 7: TITLES ADDED TO NETFLIX BY YEAR
# ------------------------------------------------

added = df.dropna(subset=["date_added"]).copy()
added["year_added"] = added["date_added"].dt.year

year_counts = added["year_added"].value_counts().sort_index()
cumulative_counts = year_counts.cumsum()

print("\nTOP YEARS BY TITLES ADDED")
display(
    year_counts.sort_values(ascending=False).head(10)
    .rename_axis("Year Added")
    .reset_index(name="Title Count")
)

plt.figure(figsize=(12, 5))
year_counts.plot(kind="bar")
plt.title("Netflix Titles Added Per Year")
plt.xlabel("Year Added")
plt.ylabel("Number of Titles")
plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 5))
cumulative_counts.plot()
plt.title("Cumulative Netflix Content Growth")
plt.xlabel("Year Added")
plt.ylabel("Cumulative Titles Added")
plt.tight_layout()
plt.show()

# ------------------------------------------------
```

```
# STEP 8: MOVIE DURATION ANALYSIS
# ------------------------------------------------

movies = df[df["type"] == "Movie"].copy()
movie_duration = movies["duration_num"].dropna()

print("\nMOVIE DURATION SUMMARY (MINUTES)")
print(movie_duration.describe())

plt.figure(figsize=(10, 5))
plt.hist(movie_duration, bins=30)
plt.title("Distribution of Movie Durations")
plt.xlabel("Duration in Minutes")
plt.ylabel("Number of Movies")
plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 9: TV SHOWS BY NUMBER OF SEASONS
# ------------------------------------------------

tv_shows = df[df["type"] == "TV Show"].copy()

season_counts = (

tv_shows["duration_num"].dropna().astype(int).value_counts().sort_index()
)

print("\nTV SHOWS BY NUMBER OF SEASONS")
display(
    season_counts.rename_axis("Seasons")
    .reset_index(name="TV Show Count")
)

plt.figure(figsize=(10, 5))
season_counts.plot(kind="bar")
plt.title("TV Shows by Number of Seasons")
plt.xlabel("Number of Seasons")
plt.ylabel("Number of TV Shows")
plt.tight_layout()
```

```
plt.show()

# ------------------------------------------------
# STEP 10: TOP 10 GENRES
# ------------------------------------------------

genre_counts = Counter()

for genres in df["listed_in"]:
    genre_counts.update(split_values(genres))

top_genres = pd.Series(genre_counts).sort_values(
    ascending=False
).head(10)

print("\nTOP 10 GENRES")
display(
    top_genres.rename_axis("Genre").reset_index(name="Title Count")
)

plt.figure(figsize=(10, 5))
top_genres.sort_values().plot(kind="barh")
plt.title("Top 10 Netflix Genres")
plt.xlabel("Number of Titles")
plt.ylabel("Genre")
plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 11: TOP 10 DIRECTORS
# ------------------------------------------------

director_counts = Counter()

for directors in df["director"]:
    director_counts.update(split_values(directors))

top_directors = pd.Series(director_counts).sort_values(
    ascending=False
).head(10)
```

```
print("\nTOP 10 DIRECTORS")
display(
    top_directors.rename_axis("Director")
    .reset_index(name="Title Count")
)

plt.figure(figsize=(10, 5))
top_directors.sort_values().plot(kind="barh")
plt.title("Top 10 Directors on Netflix")
plt.xlabel("Number of Titles")
plt.ylabel("Director")
plt.tight_layout()
plt.show()

# -------------------------------------------------
# STEP 12: TOP 10 ACTORS
# -------------------------------------------------

actor_counts = Counter()

for actors in df["cast"]:
    actor_counts.update(split_values(actors))

top_actors = pd.Series(actor_counts).sort_values(
    ascending=False
).head(10)

print("\nTOP 10 ACTORS")
display(
    top_actors.rename_axis("Actor")
    .reset_index(name="Title Count")
)

plt.figure(figsize=(10, 5))
top_actors.sort_values().plot(kind="barh")
plt.title("Top 10 Actors in Netflix Titles")
plt.xlabel("Number of Titles")
plt.ylabel("Actor")
plt.tight_layout()
```

```
plt.show()

# ------------------------------------------------
# STEP 13: WORD CLOUD OF DESCRIPTIONS
# ------------------------------------------------

descriptions = " ".join(
    df["description"].dropna().astype(str)
)

if descriptions.strip():
    wordcloud = WordCloud(
        width=1200,
        height=600,
        background_color="white",
        max_words=150
    ).generate(descriptions)

    plt.figure(figsize=(14, 7))
    plt.imshow(wordcloud, interpolation="bilinear")
    plt.axis("off")
    plt.title("Word Cloud of Netflix Descriptions")
    plt.show()
else:
    print("No descriptions available.")

# ------------------------------------------------
# STEP 14: MONTHS WHEN CONTENT IS ADDED
# ------------------------------------------------

month_names = {
    1: "January", 2: "February", 3: "March",
    4: "April", 5: "May", 6: "June",
    7: "July", 8: "August", 9: "September",
    10: "October", 11: "November", 12: "December"
}

month_counts = added["date_added"].dt.month.value_counts().sort_index()
month_counts.index = [month_names[m] for m in month_counts.index]
```

```
print("\nCONTENT ADDED BY MONTH")
display(
    month_counts.sort_values(ascending=False)
    .rename_axis("Month")
    .reset_index(name="Title Count")
)

plt.figure(figsize=(11, 5))
month_counts.plot(kind="bar")
plt.title("Netflix Titles Added by Month")
plt.xlabel("Month")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 15: COUNTRIES — MOVIES VS TV SHOWS
# ------------------------------------------------

country_type_records = []

for _, row in df[["country", "type"]].dropna(
    subset=["country"]
).iterrows():
    for country in split_values(row["country"]):
        country_type_records.append({
            "Country": country,
            "Type": row["type"]
        })

country_type_df = pd.DataFrame(country_type_records)

if not country_type_df.empty:
    country_type_counts = pd.crosstab(
        country_type_df["Country"],
        country_type_df["Type"]
    )

    top_10_country_names = (
```

```
        country_type_counts.sum(axis=1).nlargest(10).index
    )

    country_type_top10 = country_type_counts.loc[top_10_country_names]

    print("\nMOVIES VS TV SHOWS BY COUNTRY")
    display(country_type_top10)

    country_type_top10.plot(kind="bar", figsize=(12, 6))
    plt.title("Movies vs TV Shows by Top Countries")
    plt.xlabel("Country")
    plt.ylabel("Number of Titles")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()

# ------------------------------------------------
# STEP 16: RATINGS FOR MOVIES VS TV SHOWS
# ------------------------------------------------

rating_type = pd.crosstab(
    df["rating"].fillna("Not Rated"),
    df["type"]
)

print("\nRATING COUNTS BY CONTENT TYPE")
display(rating_type)

rating_type.plot(kind="bar", figsize=(12, 6))
plt.title("Ratings for Movies vs TV Shows")
plt.xlabel("Rating")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

# ------------------------------------------------
# STEP 17: OLDEST AND NEWEST TITLES
# ------------------------------------------------
```

```
valid_release = df.dropna(subset=["release_year"]).copy()

if not valid_release.empty:
    oldest_year = valid_release["release_year"].min()
    newest_year = valid_release["release_year"].max()

    print("\nOLDEST RELEASE YEAR:", int(oldest_year))
    display(
        valid_release[valid_release["release_year"] == oldest_year][
            ["title", "type", "release_year", "country"]
        ]
    )

    print("\nNEWEST RELEASE YEAR:", int(newest_year))
    display(
        valid_release[valid_release["release_year"] == newest_year][
            ["title", "type", "release_year", "country"]
        ]
    )

# ------------------------------------------------
# STEP 18: GENRE CO-OCCURRENCE CORRELATION HEATMAP
# ------------------------------------------------

top_genre_names = list(top_genres.index)

genre_matrix = pd.DataFrame(
    0, index=df.index, columns=top_genre_names
)

for idx, genres in df["listed_in"].items():
    for genre in split_values(genres):
        if genre in genre_matrix.columns:
            genre_matrix.loc[idx, genre] = 1

genre_corr = genre_matrix.corr()

print("\nGENRE CORRELATION MATRIX")
display(genre_corr)
```

```
plt.figure(figsize=(11, 8))
sns.heatmap(
    genre_corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-1,
    vmax=1
)
plt.title("Genre Co-occurrence Correlation Heatmap")
plt.tight_layout()
plt.show()

# ----------------------------------------------------
# STEP 19: SAVE RESULTS
# ----------------------------------------------------

df.to_csv("Netflix_EDA_Cleaned.csv", index=False)

top_countries.to_csv(
    "Netflix_Top_Countries.csv",
    header=["Title Count"]
)

top_genres.to_csv(
    "Netflix_Top_Genres.csv",
    header=["Title Count"]
)

summary = pd.DataFrame({
    "Metric": [
        "Total titles after duplicate removal",
        "Number of Movies",
        "Number of TV Shows",
        "Missing values remaining"
    ],
    "Value": [
        len(df),
        (df["type"] == "Movie").sum(),
        (df["type"] == "TV Show").sum(),
```

```
        int(df.isnull().sum().sum())
    ]
})

summary.to_csv("Netflix_EDA_Summary.csv", index=False)

print("\nEDA COMPLETED SUCCESSFULLY")
print("Saved files:")
print("1. Netflix_EDA_Cleaned.csv")
print("2. Netflix_Top_Countries.csv")
print("3. Netflix_Top_Genres.csv")
print("4. Netflix_EDA_Summary.csv")
```

Output:

FIRST FIVE ROWS

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a Cape Town t...

2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Act...	To protect his family from a powerful drug lor...
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA	1 Season	Docuseries, Reality TV	Feuds, flirtations and toilet talk go down amo...
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, Romantic TV Shows, TV ...	In a city of coaching centers known to train I...

LAST FIVE ROWS

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
8802	s8803	Movie	Zodiac	David Fincher	Mark Ruffalo, Jake Gyllenhaal, Robert Downey J...	United States	November 20, 2019	2007	R	158 min	Cult Movies, Dramas, Thrillers	A political cartoonist, a crime reporter and a...

8803	s8804	TV Show	Zombie Dumb	NaN	NaN	NaN	July 1, 2019	2018	TV-Y7	2 Seasons	Kids' TV, Korean TV Shows, TV Comedies	While living alone in a spooky town, a young g...
8804	s8805	Movie	Zombieland	Ruben Fleischer	Jesse Eisenberg, Woody Harrelson, Emma Stone, ...	United States	November 1, 2019	2009	R	88 min	Comedies, Horror Movies	Looking to survive in a world taken over by zo...
8805	s8806	Movie	Zoom	Peter Hewitt	Tim Allen, Courteney Cox, Chevy Chase, Kate Ma...	United States	January 11, 2020	2006	PG	88 min	Children & Family Movies, Comedies	Dragged from civilian life, a former superhero...
8806	s8807	Movie	Zubaan	Mozez Singh	Vicky Kaushal, Sarah-Jane Dias, Raaghav Chanan...	India	March 2, 2019	2015	TV-14	111 min	Dramas, International Movies, Music & Musicals	A scrappy but poor boy worms his way into a ty...

DATASET SHAPE: (8807, 12)

COLUMN INFORMATION
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):

```
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       8807 non-null   object
 1   type          8807 non-null   object
 2   title         8807 non-null   object
 3   director      6173 non-null   object
 4   cast          7982 non-null   object
 5   country       7976 non-null   object
 6   date_added    8797 non-null   object
 7   release_year  8807 non-null   int64
 8   rating        8803 non-null   object
 9   duration      8804 non-null   object
 10  listed_in     8807 non-null   object
 11  description   8807 non-null   object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB
```

MISSING VALUES

```
show_id            0
type               0
title              0
director        2634
cast             825
country          831
date_added        10
release_year       0
rating             4
duration           3
listed_in          0
description        0
dtype: int64
```

DUPLICATE ROWS: 0

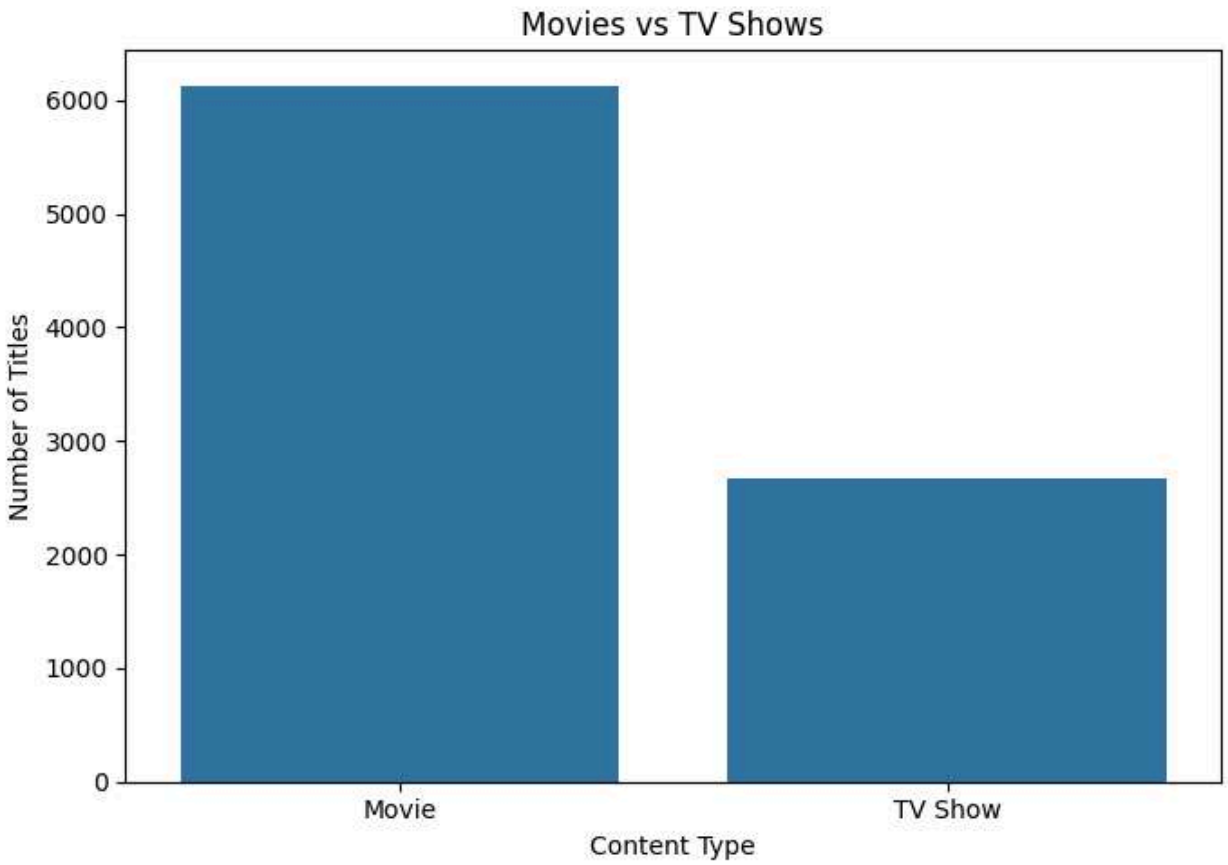
STATISTICAL SUMMARY

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
show_id	8807	8807	s8807	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
type	8807	2	Movie	6131	NaN	NaN	NaN	NaN	NaN	NaN	NaN

title	8807	8807	Zubaan	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
director	6173	4528	Rajiv Chilaka	19	NaN	NaN	NaN	NaN	NaN	NaN	NaN
cast	7982	7692	David Attenborough	19	NaN	NaN	NaN	NaN	NaN	NaN	NaN
country	7976	748	United States	2818	NaN	NaN	NaN	NaN	NaN	NaN	NaN
date_added	8797	1767	January 1, 2020	109	NaN	NaN	NaN	NaN	NaN	NaN	NaN
release_year	8807.0	NaN	NaN	NaN	2014.180198	8.819312	1925.0	2013.0	2017.0	2019.0	2021.0
rating	8803	17	TV-MA	3207	NaN	NaN	NaN	NaN	NaN	NaN	NaN
duration	8804	220	1 Season	1793	NaN	NaN	NaN	NaN	NaN	NaN	NaN
listed_in	8807	514	Dramas, International Movies	362	NaN	NaN	NaN	NaN	NaN	NaN	NaN
description	8807	8775	Paranormal activity at a lush, abandoned prope...	4	NaN	NaN	NaN	NaN	NaN	NaN	NaN

DATASET AFTER REMOVING DUPLICATES
Rows and columns: (8807, 13)

MOVIES VS TV SHOWS
type
Movie 6131
TV Show 2676
Name: count, dtype: int64

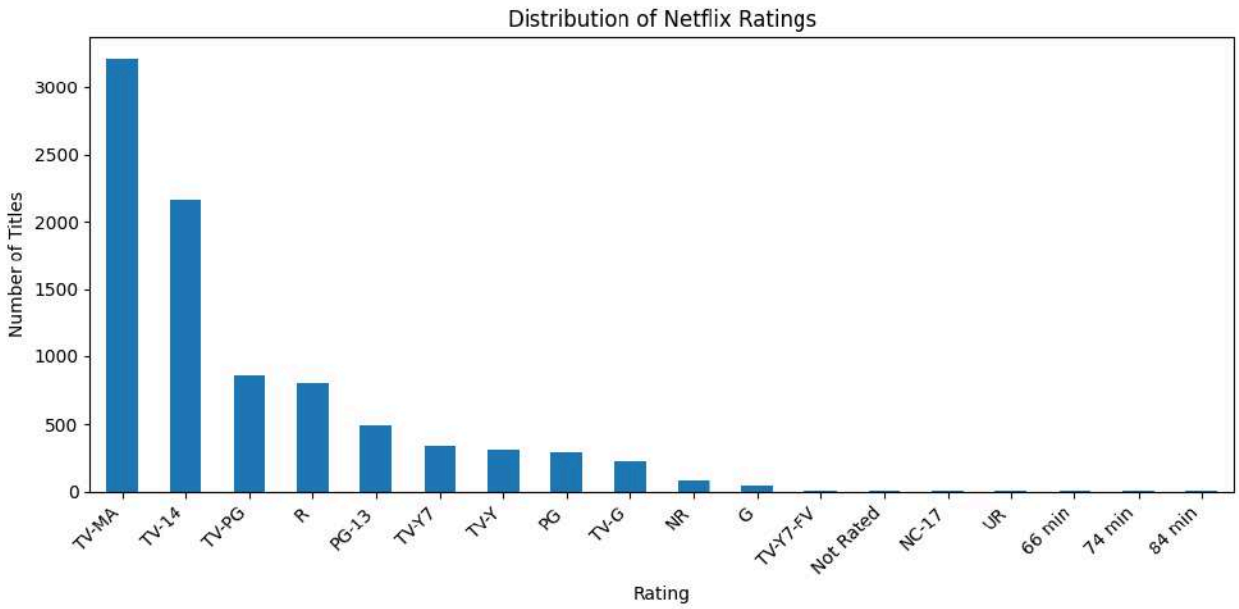


RATING DISTRIBUTION

rating

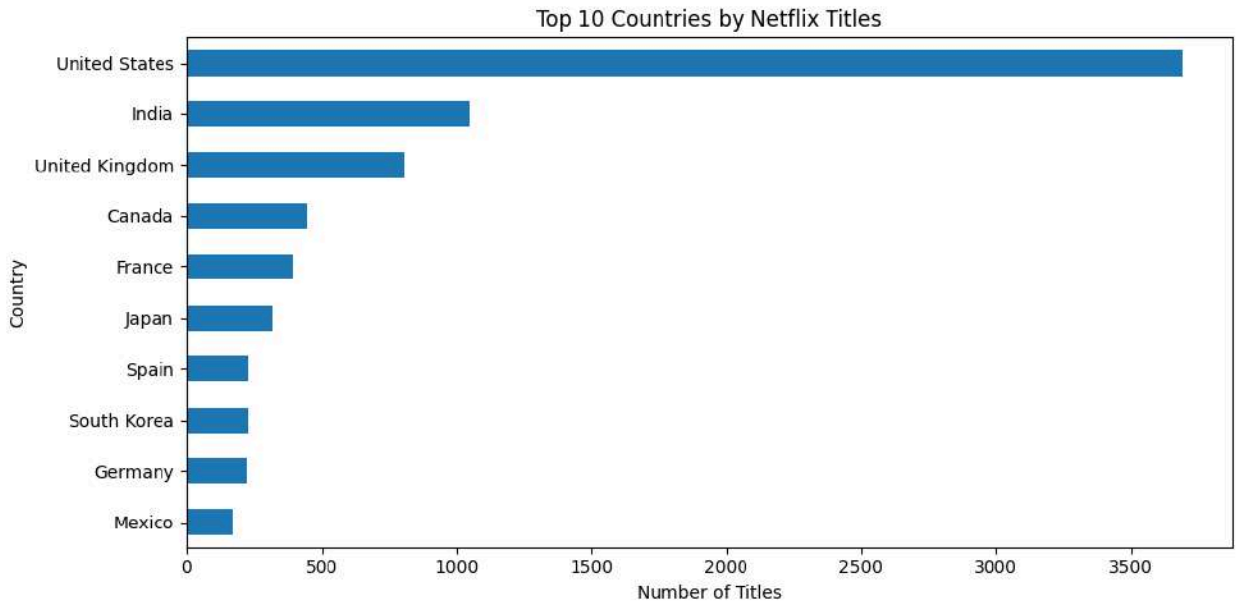
TV-MA	3207
TV-14	2160
TV-PG	863
R	799
PG-13	490
TV-Y7	334
TV-Y	307
PG	287
TV-G	220
NR	80
G	41
TV-Y7-FV	6
Not Rated	4
NC-17	3
UR	3
66 min	1
74 min	1
84 min	1

Name: count, dtype: int64



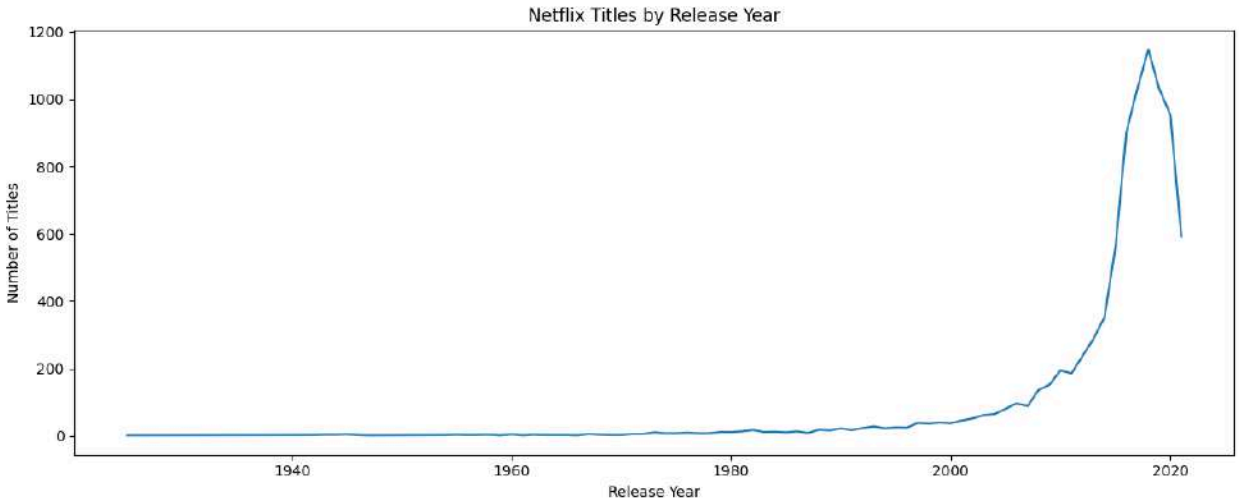
TOP 10 COUNTRIES

	Country	Title Count
0	United States	3690
1	India	1046
2	United Kingdom	806
3	Canada	445
4	France	393
5	Japan	318
6	Spain	232
7	South Korea	231
8	Germany	226
9	Mexico	169



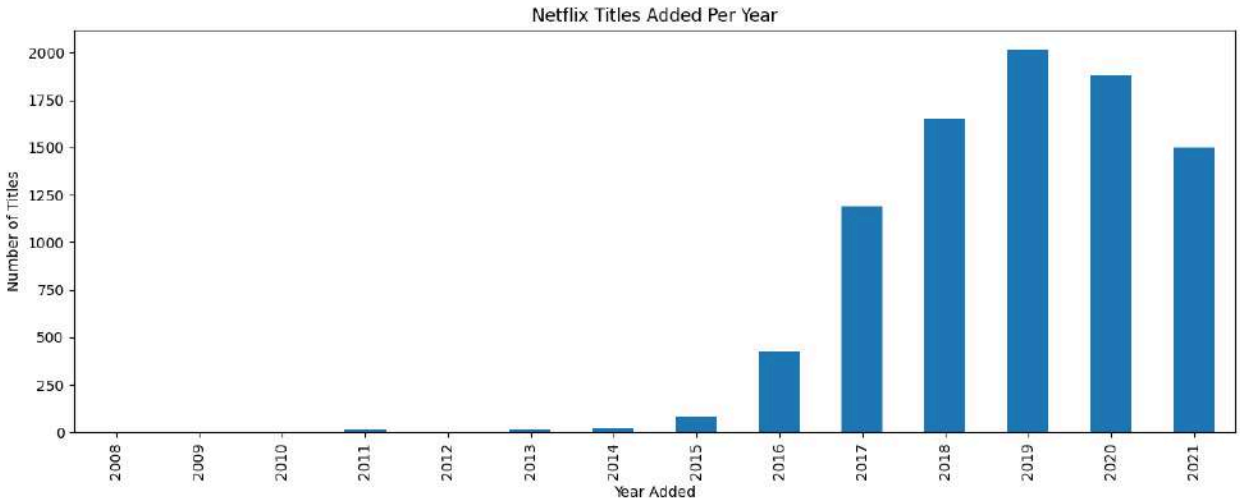
TOP 10 RELEASE YEARS

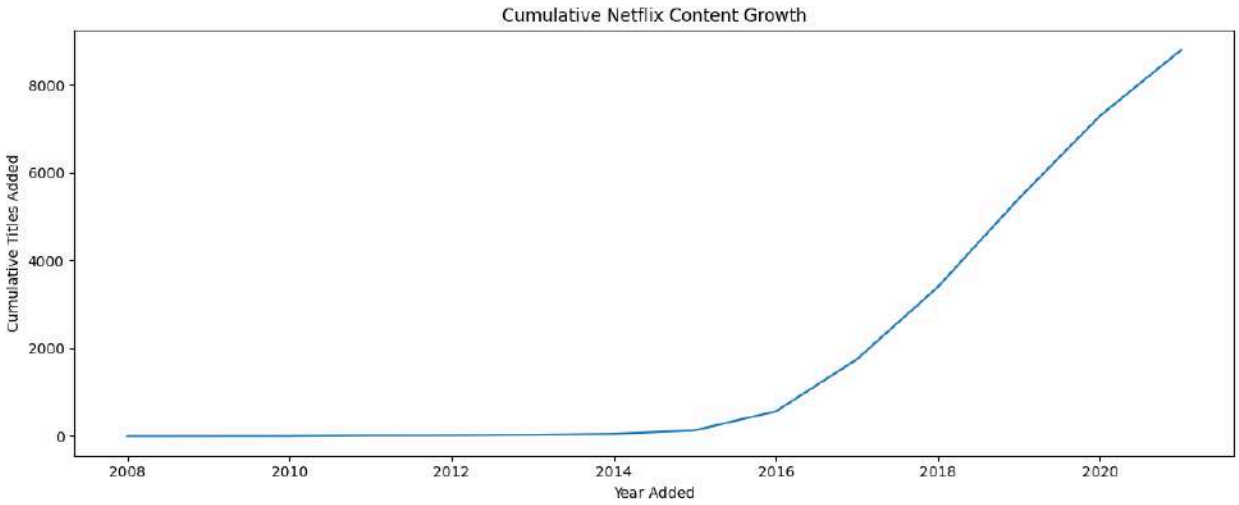
	Release Year	Title Count
0	2018	1147
1	2017	1032
2	2019	1030
3	2020	953
4	2016	902
5	2021	592
6	2015	560
7	2014	352
8	2013	288
9	2012	237



TOP YEARS BY TITLES ADDED

	Year Added	Title Count
0	2019	2016
1	2020	1879
2	2018	1649
3	2021	1498
4	2017	1188
5	2016	429
6	2015	82
7	2014	24
8	2011	13
9	2013	11

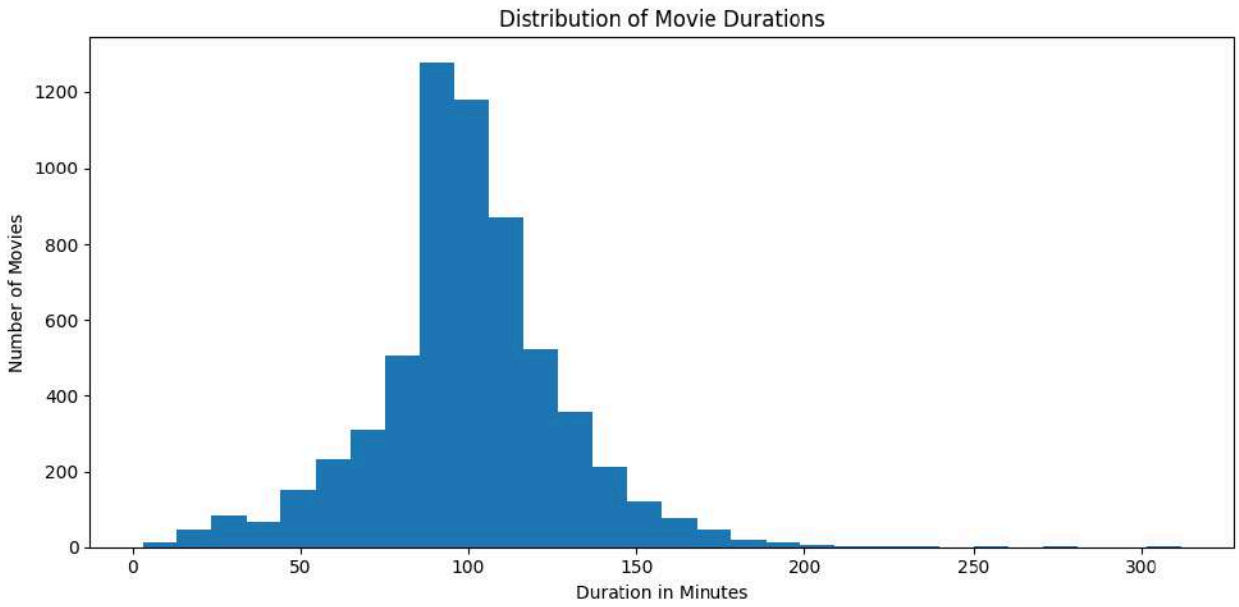




MOVIE DURATION SUMMARY (MINUTES)

count	6128.0
mean	99.577187
std	28.290593
min	3.0
25%	87.0
50%	98.0
75%	114.0
max	312.0

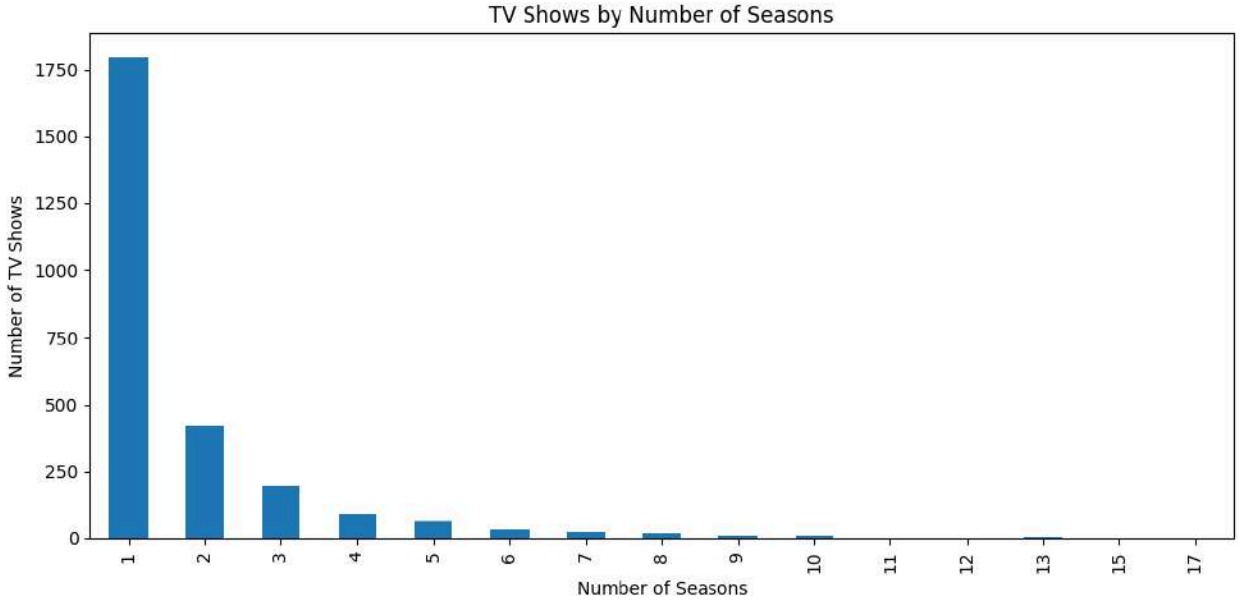
Name: duration_num, dtype: Float64



TV SHOWS BY NUMBER OF SEASONS

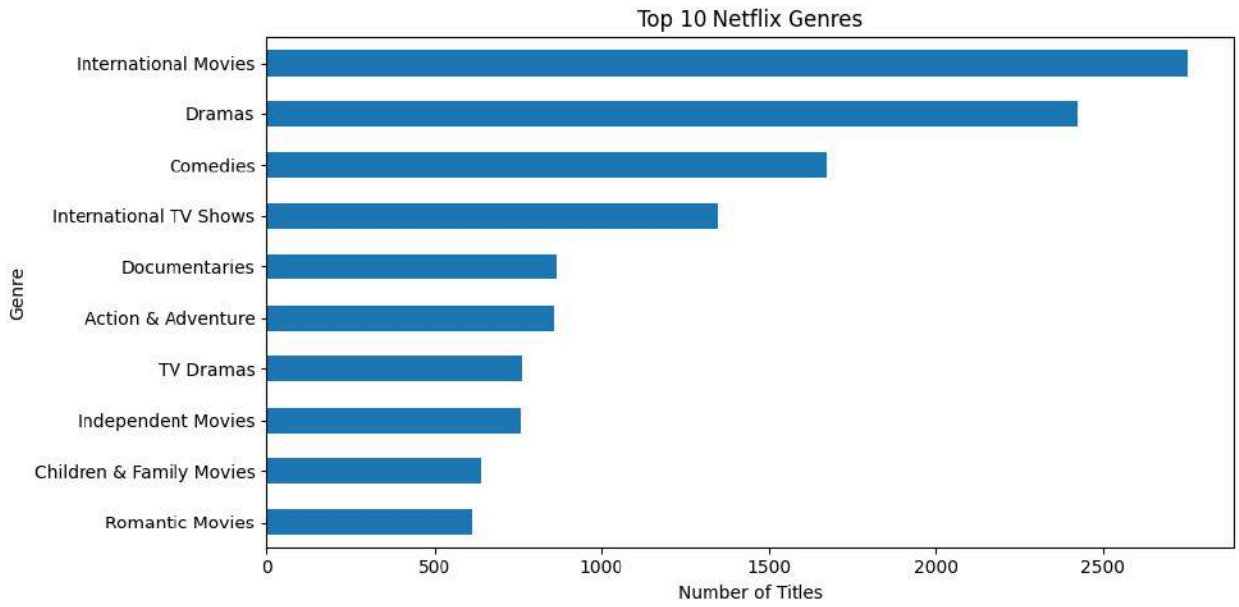
	Seasons	TV Show Count
0	1	1793
1	2	425
2	3	199
3	4	95
4	5	65
5	6	33
6	7	23
7	8	17
8	9	9
9	10	7
10	11	2
11	12	2
12	13	3

13	15	2
14	17	1



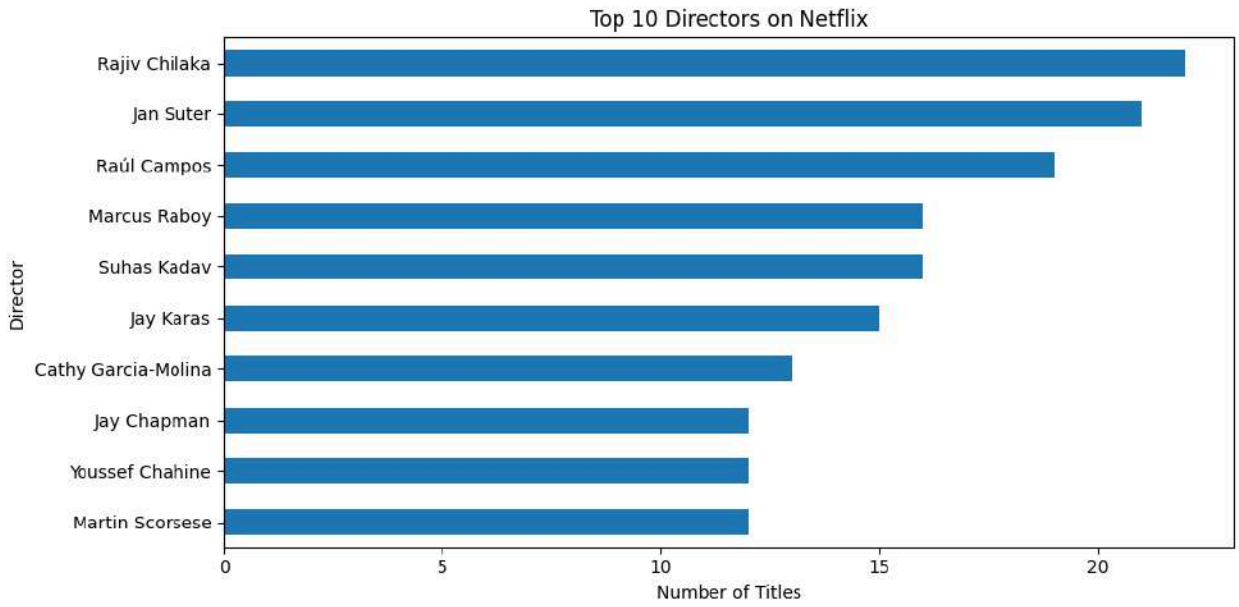
TOP 10 GENRES

	Genre	Title Count
0	International Movies	2752
1	Dramas	2427
2	Comedies	1674
3	International TV Shows	1351
4	Documentaries	869
5	Action & Adventure	859
6	TV Dramas	763
7	Independent Movies	756
8	Children & Family Movies	641
9	Romantic Movies	616



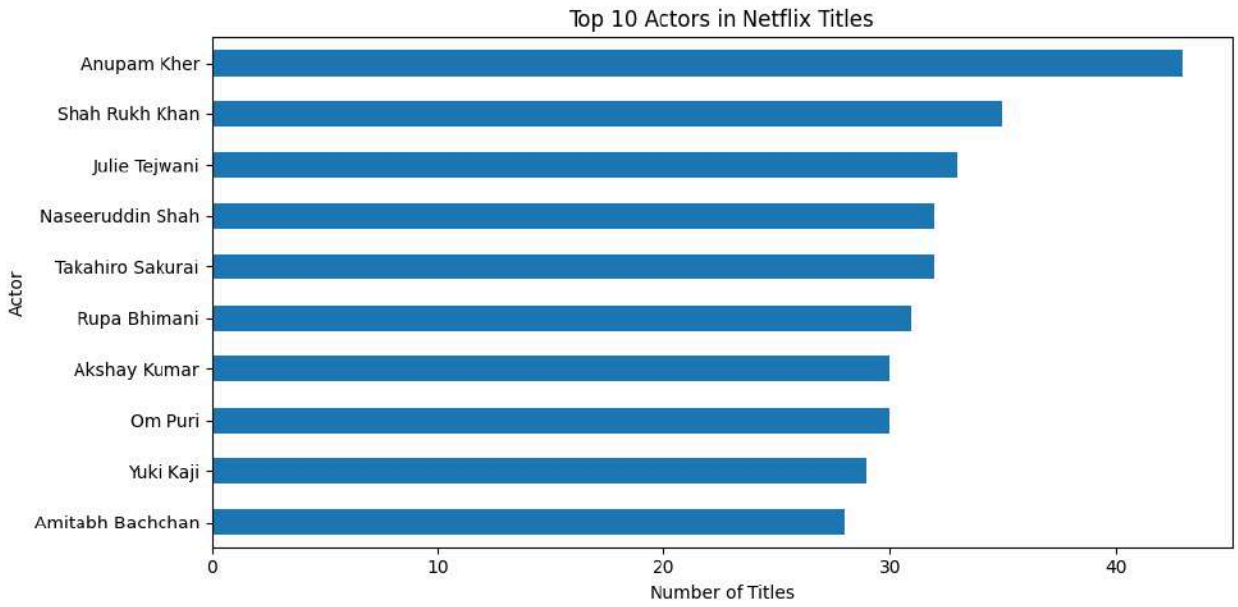
TOP 10 DIRECTORS

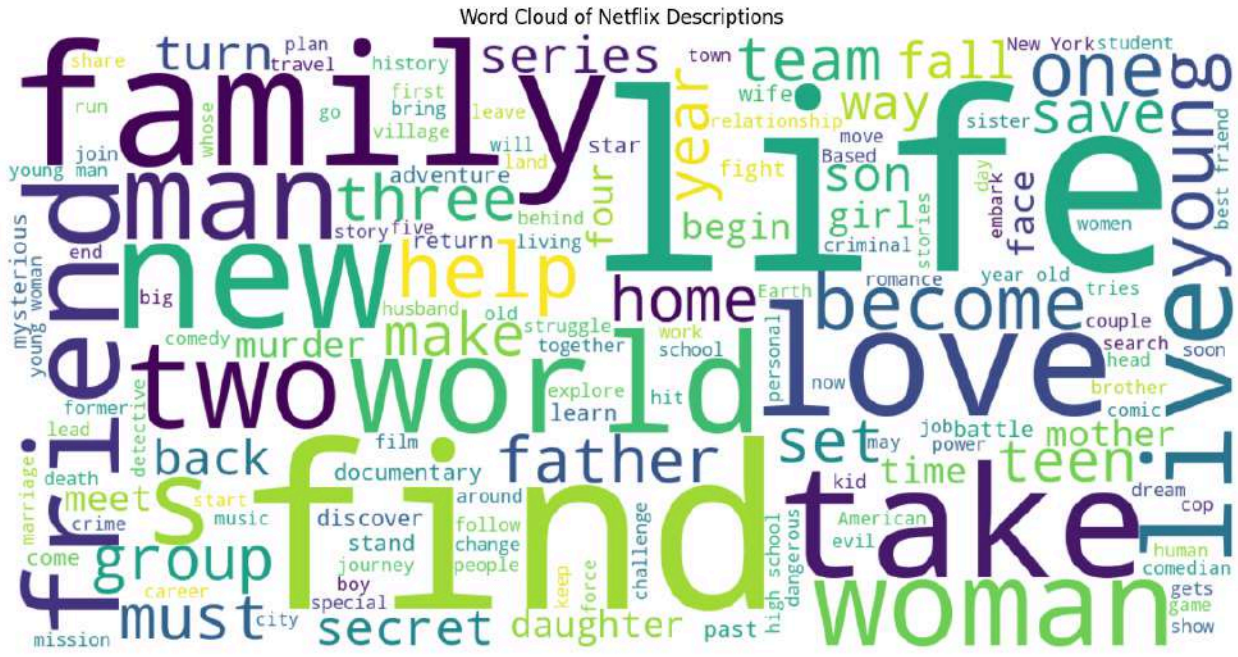
	Director	Title Count
0	Rajiv Chilaka	22
1	Jan Suter	21
2	Raúl Campos	19
3	Suhas Kadav	16
4	Marcus Raboy	16
5	Jay Karas	15
6	Cathy Garcia-Molina	13
7	Martin Scorsese	12
8	Youssef Chahine	12
9	Jay Chapman	12



TOP 10 ACTORS

	Actor	Title Count
0	Anupam Kher	43
1	Shah Rukh Khan	35
2	Julie Tejwani	33
3	Takahiro Sakurai	32
4	Naseeruddin Shah	32
5	Rupa Bhimani	31
6	Om Puri	30
7	Akshay Kumar	30
8	Yuki Kaji	29
9	Amitabh Bachchan	28

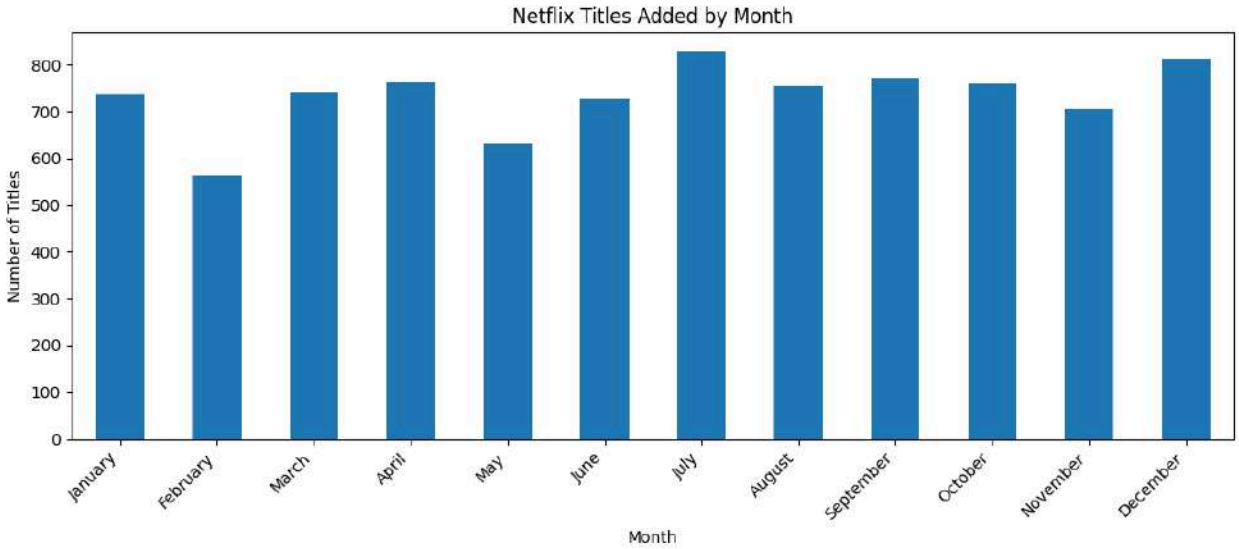




CONTENT ADDED BY MONTH

	Month	Title Count
0	July	827
1	December	813
2	September	770

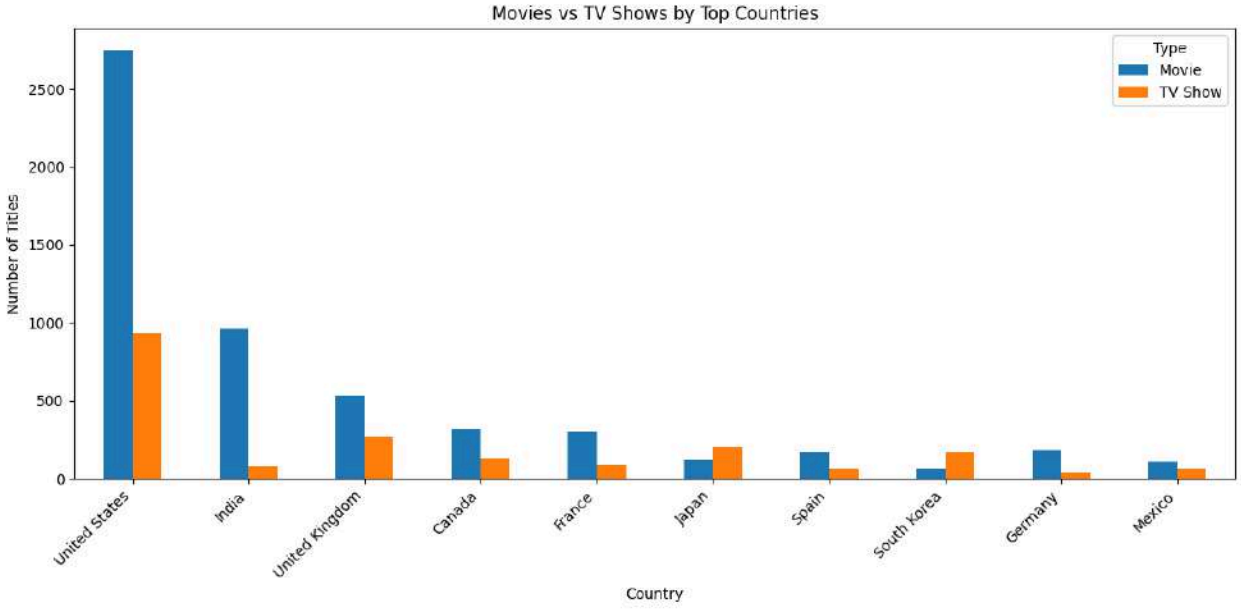
3	April	764
4	October	760
5	August	755
6	March	742
7	January	738
8	June	728
9	November	705
10	May	632
11	February	563



MOVIES VS TV SHOWS BY COUNTRY

Type	Movie	TV Show
Country		
United States	2752	938
India	962	84
United Kingdom	534	272
Canada	319	126

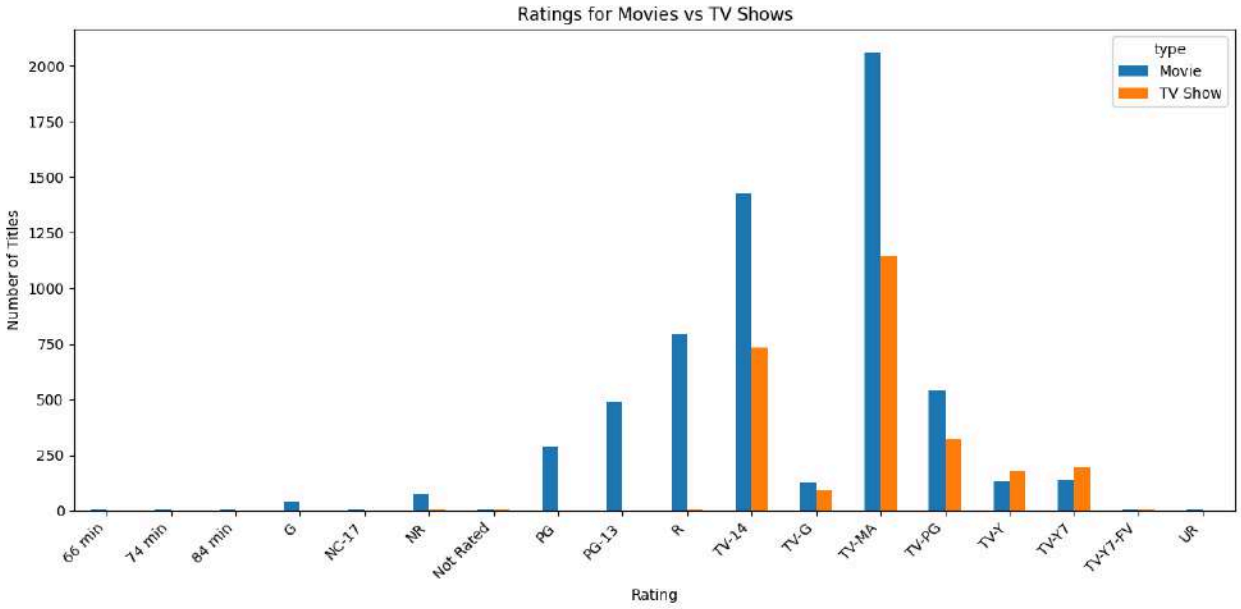
France	303	90
Japan	119	199
Spain	171	61
South Korea	61	170
Germany	182	44
Mexico	111	58



RATING COUNTS BY CONTENT TYPE

type	Movie	TV Show
rating		
66 min	1	0
74 min	1	0
84 min	1	0
G	41	0
NC-17	3	0
NR	75	5

Not Rated	2	2
PG	287	0
PG-13	490	0
R	797	2
TV-14	1427	733
TV-G	126	94
TV-MA	2062	1145
TV-PG	540	323
TV-Y	131	176
TV-Y7	139	195
TV-Y7-FV	5	1
UR	3	0



OLDEST RELEASE YEAR: 1925

	title	type	release_year	country
4250	Pioneers: First Women Filmmakers*	TV Show	1925	NaN

NEWEST RELEASE YEAR: 2021

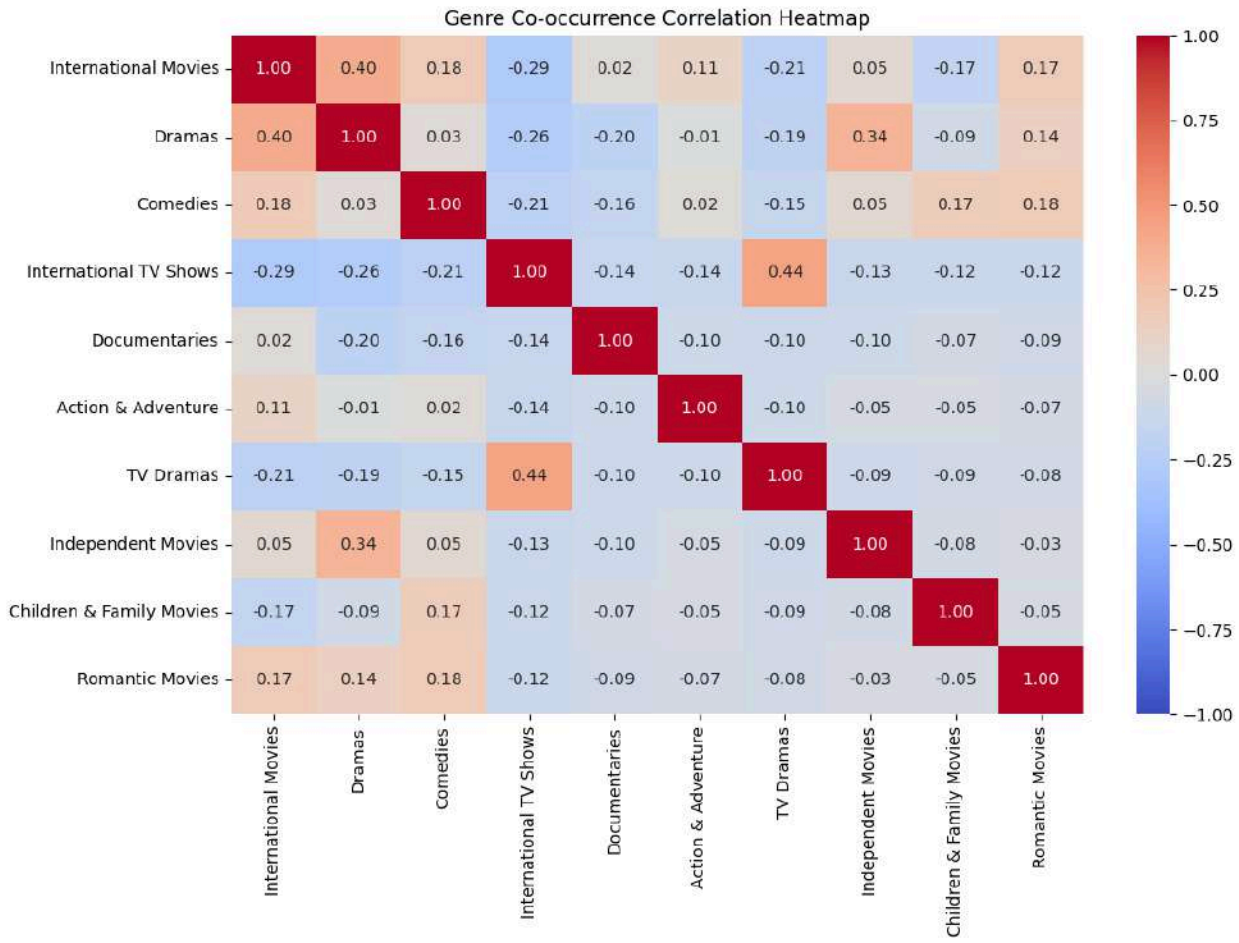
	title	type	release_year	country
1	Blood & Water	TV Show	2021	South Africa
2	Ganglands	TV Show	2021	NaN
3	Jailbirds New Orleans	TV Show	2021	NaN
4	Kota Factory	TV Show	2021	India
5	Midnight Mass	TV Show	2021	NaN
...	...	...	...	...
1468	What Happened to Mr. Cha?	Movie	2021	South Korea
1551	Hilda	TV Show	2021	United Kingdom, Canada, United States
1696	Polly Pocket	TV Show	2021	Canada, United States, Ireland
2920	Love Is Blind	TV Show	2021	United States
8437	The Netflix Afterparty	TV Show	2021	United States

592 rows × 4 columns

GENRE CORRELATION MATRIX

	International Movies	Dramas	Comedies	International TV Shows	Documentaries	Action & Adventure	TV Dramas	Independent Movies	Children & Family Movies	Romantic Movies

International Movies	1.000000	0.397291	0.175388	-0.286974	0.020089	0.106995	-0.207632	0.048768	-0.165307	0.172419
Dramas	0.397291	1.000000	0.026352	-0.262542	-0.201513	-0.005756	-0.189955	0.344427	-0.085738	0.137744
Comedies	0.175388	0.026352	1.000000	-0.206213	-0.157375	0.019240	-0.149200	0.051966	0.165050	0.181443
International TV Shows	-0.286974	-0.262542	-0.206213	1.000000	-0.140841	-0.139940	0.444622	-0.130440	-0.119261	-0.116734
Documentaries	0.020089	-0.201513	-0.157375	-0.140841	1.000000	-0.099790	-0.101902	-0.101389	-0.067784	-0.090735
Action & Adventure	0.106995	-0.005756	0.019240	-0.139940	-0.099790	1.000000	-0.101250	-0.051557	-0.053803	-0.066147
TV Dramas	-0.207632	-0.189955	-0.149200	0.444622	-0.101902	-0.101250	1.000000	-0.094376	-0.086288	-0.084459
Independent Movies	0.048768	0.344427	0.051966	-0.130440	-0.101389	-0.051557	-0.094376	1.000000	-0.076492	-0.033180
Children & Family Movies	-0.165307	-0.085738	0.165050	-0.119261	-0.067784	-0.053803	-0.086288	-0.076492	1.000000	-0.047700
Romantic Movies	0.172419	0.137744	0.181443	-0.116734	-0.090735	-0.066147	-0.084459	-0.033180	-0.047700	1.000000



EDA COMPLETED SUCCESSFULLY
Saved files:
1. Netflix_EDA_Cleaned.csv
2. Netflix_Top_Countries.csv
3. Netflix_Top_Genres.csv
4. Netflix_EDA_Summary.csv

S.pargavi
#241801196
Ex7: Model Planning and Building - Clustering

Ex7: Model Planning and Building - Clustering

Explanation of Each Step

Aim: To apply the K-Means clustering algorithm to the Mall Customer Segmentation dataset and group customers based on their annual income and spending score for personalized marketing.

Step 1: Import the required libraries

Python

Run

```
None
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

Explanation:

- `pandas` is used to load and manipulate the dataset.
- `numpy` supports numerical operations.
- `matplotlib.pyplot` creates graphs and visualizations.

- `seaborn` creates statistical plots such as pair plots and cluster plots.
- `StandardScaler` standardizes the selected features.
- `KMeans` implements the K-Means clustering algorithm.

Step 2: Load the dataset

Python

Run

```
None
url =
"https://raw.githubusercontent.com/rahkum96/Mall-Customer-Segmentation
-Data/master/Mall_Customers.csv"
df = pd.read_csv(url)

print("DATASET LOADED SUCCESSFULLY")
print("Total samples:", df.shape[0])
print("Total features:", df.shape[1])
```

Explanation:

- `pd.read_csv(url)` reads the customer dataset into a Pandas DataFrame.
- `df.shape[0]` returns the number of customer records.
- `df.shape[1]` returns the number of columns or features.

The dataset contains 200 customer records and five original columns.

Step 3: Display the first and last five records

Python

Run

```
None
print(df.head())
```

```
print(df.tail())
```

Explanation:

- `df.head()` displays the first five records.
- `df.tail()` displays the last five records.

These functions help us understand how the data is organized and what information each row contains.

Step 4: Check the shape and data types

Python

Run

```
None
print(df.shape)
print(df.dtypes)
```

Explanation:

- `df.shape` displays the number of rows and columns.
- `df.dtypes` displays the data type of each column.

For example, customer IDs are integers, while gender is text. Income and spending score are numerical features suitable for clustering.

Step 5: Check missing values and duplicates

Python

Run

```
None
print(df.isnull().sum())
print(df.duplicated().sum())
```

Explanation:

- `df.isnull().sum()` counts missing values in every column.
- `df.duplicated().sum()` counts duplicate rows.

These checks help identify incomplete or repeated records that could affect the analysis.

Step 6: Display summary statistics

Python

Run

```
None
print(df.describe())
```

Explanation:

The `describe()` function displays statistical information about numerical columns, including:

- `count` – number of non-missing values.
- `mean` – average value.
- `std` – standard deviation.
- `min` and `max` – minimum and maximum values.
- `25%`, `50%`, and `75%` – quartiles.

This helps us understand the distribution of customer income and spending scores.

Step 7: Select the important features

Python

Run

```
None
features = df[["Annual Income (k$)", "Spending Score (1-100)"]]
```

```
print(features.head())
```

Explanation:

Two features are selected for customer segmentation:

- Annual Income (k$): The customer's annual income in thousands of dollars.
- Spending Score (1-100): A score representing the customer's spending behavior.

These two numerical features are used to group customers with similar income and spending patterns.

Step 8: Visualize the selected features using a scatter plot

Python

Run

```
None
plt.figure(figsize=(8, 5))
plt.scatter(features["Annual Income (k
)"], features["SpendingScore(1 − 100)"])plt.xlabel("AnnualIncome(k)")
plt.ylabel("Spending Score (1-100)")
plt.title("Annual Income vs Spending Score")
plt.grid(True)
plt.show()
```

Explanation:

- The X-axis represents annual income.
- The Y-axis represents spending score.
- Each point represents one customer.

The scatter plot helps us visually identify patterns and possible groups among customers before applying clustering.

Step 9: Create a pair plot

Python

Run

```
None
sns.pairplot(features)
plt.suptitle("Pair Plot of Selected Features", y=1.02)
plt.show()
```

Explanation:

The pair plot shows the distributions of individual features and the relationship between them. Since we selected two features, it helps us examine their distributions and relationship before clustering.

Step 10: Standardize the selected features

Python

Run

```
None
scaler = StandardScaler()
X_scaled = scaler.fit_transform(features)

print(pd.DataFrame(X_scaled, columns=features.columns).head())
```

Explanation:

Standardization transforms the selected features so that they have approximately zero mean and unit standard deviation.

The formula is:

z=x−μσz = \frac{x-\mu}{\sigma}z=σx−μ

Where:

- xxx is the original value.
- μ\muμ is the feature's mean.
- σ\sigmaσ is the feature's standard deviation.

Standardization prevents a feature's numerical scale from dominating the distance calculations used by K-Means.

Step 11: Find the optimal number of clusters using the Elbow Method

Python

Run

None

```
wcss = []

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.xticks(range(1, 11))
plt.grid(True)
plt.show()
```

Explanation:

The Elbow Method helps select a suitable number of clusters.

- The loop tests values of KKK from 1 to 10.
- `kmeans.fit(X_scaled)` fits K-Means for each value of KKK.

- `kmeans.inertia_` returns the within-cluster sum of squares (WCSS).
- The graph plots WCSS against the number of clusters.

WCSS generally decreases as KKK increases. We look for the point where the decrease begins to slow significantly, forming an elbow. The example code uses K=5K=5K=5, a commonly used choice for this dataset.

Step 12: Train the K-Means clustering model

Python

Run

None

```
optimal_k = 5

kmeans = KMeans(
    n_clusters=optimal_k,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(X_scaled)

print(df.head(10))
```

Explanation:

- `n_clusters=5` specifies five customer groups.
- `random_state=42` makes the initialization reproducible.
- `n_init=10` runs the algorithm with multiple initial centroid choices.
- `fit_predict()` fits the model and assigns each customer a cluster label.
- `df["Cluster"]` stores each customer's assigned cluster.

The labels are numbers from 0 to 4. They identify groups, but do not represent rankings.

Step 13: Calculate the cluster centroid coordinates

Python

Run

```
None
centroids = scaler.inverse_transform(kmeans.cluster_centers_)

centroid_df = pd.DataFrame(
    centroids,
    columns=["Annual Income (k$)", "Spending Score (1-100)"]
)

print(centroid_df)
```

Explanation:

K-Means calculates the center, or centroid, of each cluster in standardized feature space.

- `kmeans.cluster_centers_` retrieves the centroid coordinates.
- `scaler.inverse_transform()` converts them back to the original income and spending-score scales.
- The DataFrame displays the centroid coordinates for all five clusters.

These coordinates represent the approximate average income and spending score of each cluster.

Step 14: Visualize the customer clusters

Python

Run

```
None
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(
    data=df,
    x="Annual Income (k
)", y = "SpendingScore(1 − 100)", hue = "Cluster", palette = "viridis", s = 80)plt.scatter(centroids[:, 0], centr
)")
plt.ylabel("Spending Score (1-100)")
plt.title("Customer Segmentation using K-Means")
plt.legend()
plt.grid(True)
plt.show()
```

Explanation:

- `hue="Cluster"` displays different colors for the five customer groups.
- Each point represents a customer.
- The large red `X` markers indicate the cluster centroids.
- The axes show income and spending score in their original units.

This graph helps us compare the characteristics of each customer segment visually.

Step 15: Count the customers in each cluster

Python

Run

```
None
print(df["Cluster"].value_counts().sort_index())
```

Explanation:

- `value_counts()` counts the customers assigned to each cluster.
- `sort_index()` displays the cluster labels in numerical order.

This identifies the size of each segment and helps the company understand how its customers are distributed across the groups.

Step 16: Calculate average income and spending score

Python

Run

```
None
cluster_summary = df.groupby("Cluster").agg(
    Customer_Count=("CustomerID", "count"),
    Average_Annual_Income=("Annual Income (k$)", "mean"),
    Average_Spending_Score=("Spending Score (1-100)", "mean")
).round(2)

print(cluster_summary)
```

Explanation:

- `groupby("Cluster")` groups customers by their assigned cluster.
- `Customer_Count` counts the records in each cluster.
- `Average_Annual_Income` calculates the mean income for each cluster.
- `Average_Spending_Score` calculates the mean spending score.
- `round(2)` rounds the results to two decimal places.

This summary helps identify the general spending behavior and income level of each customer segment.

Step 17: Recommend marketing strategies

Python

Run

```
None
for cluster, row in cluster_summary.iterrows():
```

```
    income = row["Average_Annual_Income"]
    spending = row["Average_Spending_Score"]

    if income >= 60 and spending >= 60:
        strategy = "Premium offers and loyalty benefits."
    elif income >= 60 and spending < 40:
        strategy = "Personalized promotions to encourage purchases."
    elif income < 40 and spending >= 60:
        strategy = "Affordable products, discounts, and bundle deals."
    elif income < 40 and spending < 40:
        strategy = "Budget-friendly offers."
    else:
        strategy = "Targeted discounts and personalized
recommendations."

    print(f"Cluster {cluster}: {strategy}")
```

Explanation:

This code uses each cluster's average income and spending score to suggest a marketing strategy.

- High-income, high-spending customers may respond well to premium offers and loyalty rewards.
- High-income, low-spending customers may benefit from personalized promotions.
- Lower-income, high-spending customers may respond to affordable bundles.
- Lower-income, low-spending customers may be interested in budget-friendly offers.

These are initial recommendations based on simple thresholds; actual marketing decisions should also consider customer behavior and campaign results.

Step 18: Save the clustered dataset

Python

Run

```
None
df.to_csv("Mall_Customers_Clustered.csv", index=False)
print("Clustered dataset saved successfully.")
```

Explanation:

- `to_csv()` saves the original dataset together with the newly assigned `Cluster` column.
- `index=False` prevents Pandas from adding an extra index column.

The saved file can be used for further analysis, reporting, and customer-targeting decisions.

Full code:

```
# Customer Segmentation using K-Means

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Step 1: Load the dataset
url =
"https://raw.githubusercontent.com/rahkum96/Mall-Customer-Segmentation-Data/master/Mall_Customers.csv"
df = pd.read_csv(url)

print("DATASET LOADED SUCCESSFULLY")
print("Total samples:", df.shape[0])
```

```
print("Total features:", df.shape[1])

# Step 2: Display first and last five records
print("\nFirst 5 records:")
print(df.head())

print("\nLast 5 records:")
print(df.tail())

# Step 3: Check shape and data types
print("\nDataset shape:", df.shape)

print("\nData types:")
print(df.dtypes)

# Step 4: Check missing and duplicate values
print("\nMissing values:")
print(df.isnull().sum())

print("\nNumber of duplicate records:", df.duplicated().sum())

# Step 5: Display summary statistics
print("\nSummary statistics:")
print(df.describe())

# Step 6: Select features for clustering
features = df[["Annual Income (k$)", "Spending Score (1-100)"]]

print("\nSelected features:")
print(features.head())

# Step 7: Visualize selected features using a scatter plot
plt.figure(figsize=(8, 5))
plt.scatter(features["Annual Income (k$)"],
            features["Spending Score (1-100)"])
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Annual Income vs Spending Score")
plt.grid(True)
plt.show()
```

```
# Step 8: Pair plot
sns.pairplot(features)
plt.suptitle("Pair Plot of Selected Features", y=1.02)
plt.show()

# Step 9: Standardize the selected features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(features)

print("\nStandardized features (first 5 rows):")
print(pd.DataFrame(X_scaled, columns=features.columns).head())

# Step 10: Find optimal K using the Elbow Method
wcss = []

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.xticks(range(1, 11))
plt.grid(True)
plt.show()

# Step 11: Train the K-Means model
# K = 5 is commonly used for this dataset.
optimal_k = 5

kmeans = KMeans(
    n_clusters=optimal_k,
    random_state=42,
    n_init=10
)
```

```
df["Cluster"] = kmeans.fit_predict(X_scaled)

print("\nCluster labels assigned successfully.")
print(df.head(10))

# Step 12: Calculate centroid coordinates in original units
centroids = scaler.inverse_transform(kmeans.cluster_centers_)

centroid_df = pd.DataFrame(
    centroids,
    columns=["Annual Income (k$)", "Spending Score (1-100)"]
)

print("\nCluster centroid coordinates:")
for i, row in centroid_df.iterrows():
    print(
        f"Cluster {i}: Annual Income = {row['Annual Income (k$)']:.2f} k$,
"
        f"Spending Score = {row['Spending Score (1-100)']:.2f}"
    )

# Step 13: Visualize clusters and centroids
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="Annual Income (k$)",
    y="Spending Score (1-100)",
    hue="Cluster",
    palette="viridis",
    s=80
)

plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker="X",
    s=250,
    color="red",
    label="Centroids"
```

```
)

plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Customer Segmentation using K-Means")
plt.legend()
plt.grid(True)
plt.show()

# Step 14: Count customers in each cluster
print("\nNumber of customers in each cluster:")
print(df["Cluster"].value_counts().sort_index())

# Step 15: Calculate average income and spending score
cluster_summary = df.groupby("Cluster").agg(
    Customer_Count=("CustomerID", "count"),
    Average_Annual_Income=("Annual Income (k$)", "mean"),
    Average_Spending_Score=("Spending Score (1-100)", "mean")
).round(2)

print("\nCluster Analysis:")
print(cluster_summary)

# Step 16: Recommend marketing strategies
print("\nMARKETING RECOMMENDATIONS:")

for cluster, row in cluster_summary.iterrows():
    income = row["Average_Annual_Income"]
    spending = row["Average_Spending_Score"]

    if income >= 60 and spending >= 60:
        strategy = "Reward loyal customers with premium offers and loyalty
benefits."
    elif income >= 60 and spending < 40:
        strategy = "Provide personalized promotions to encourage
purchases."
    elif income < 40 and spending >= 60:
        strategy = "Offer affordable products, discounts, and bundle
deals."
    elif income < 40 and spending < 40:
```

```
        strategy = "Use budget-friendly offers to attract customer
interest."
    else:
        strategy = "Use targeted discounts and personalized product
recommendations."

    print(f"Cluster {cluster}: {strategy}")

# Step 17: Save clustered dataset
df.to_csv("Mall_Customers_Clustered.csv", index=False)
print("\nClustered dataset saved as Mall_Customers_Clustered.csv")

# Download the output file in Google Colab
from google.colab import files
files.download("Mall_Customers_Clustered.csv")
```

Output:

DATASET LOADED SUCCESSFULLY
Total samples: 200
Total features: 5

First 5 records:
  CustomerID Genre Age Annual Income (k
)$SpendingScore(1 - 100)01Male19153912Male21158123Female2016634Female23167745Female311740Last5re$
) Spending Score (1-100)
195 196 Female 35 120 79
196 197 Female 45 126 28
197 198 Male 32 126 74
198 199 Male 32 137 18
199 200 Male 30 137 83

Dataset shape: (200, 5)

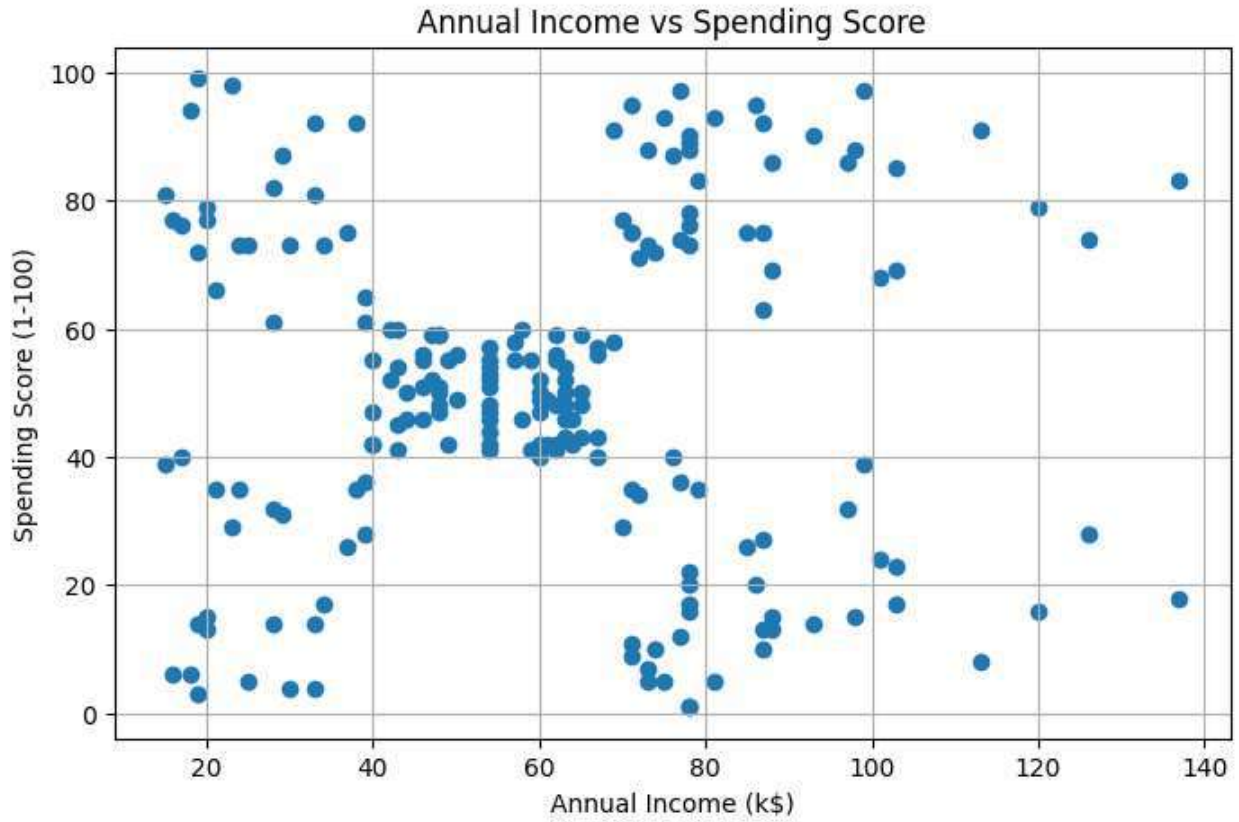
Data types:
CustomerID int64
Genre object
Age int64
Annual Income (k
$)int64SpendingScore(1 - 100)int64dtype: objectMissingvalues: CustomerID0Genre0Age0AnnualIncome(k$
) 0
Spending Score (1-100) 0

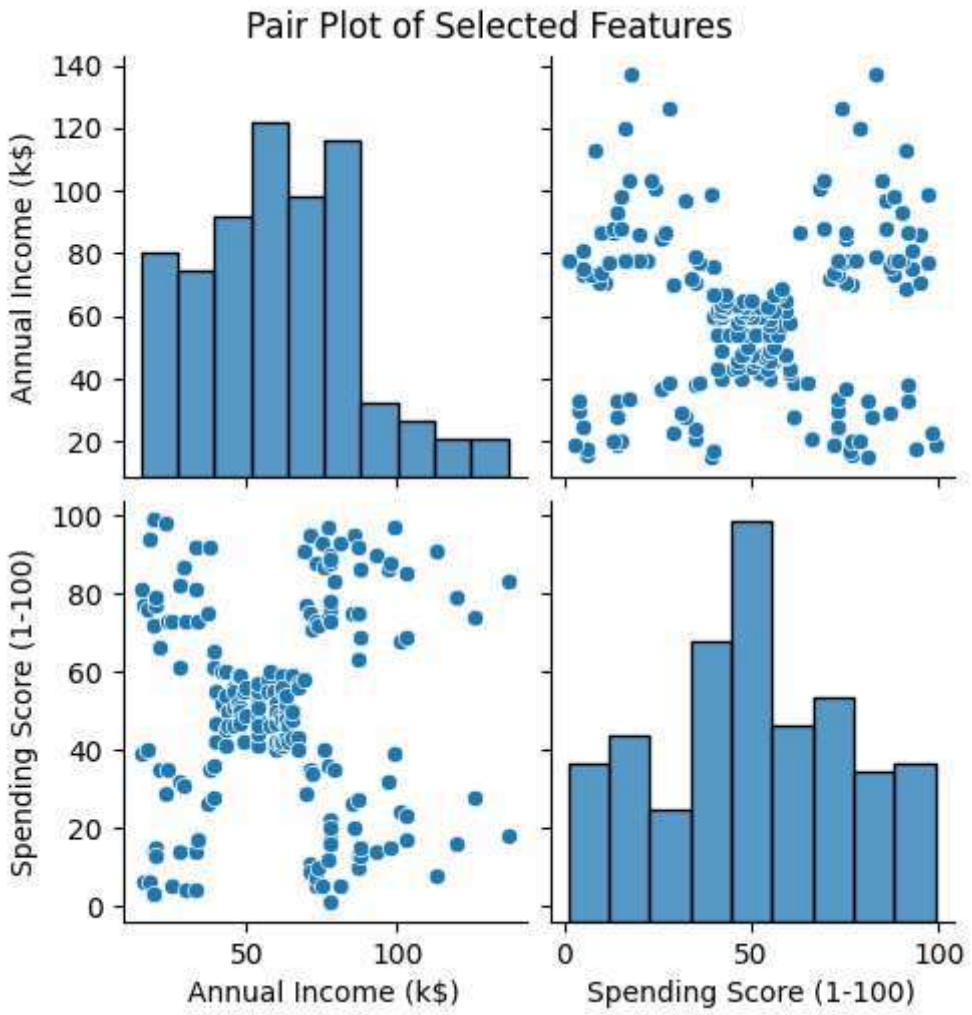
dtype: int64

Number of duplicate records: 0

Summary statistics:

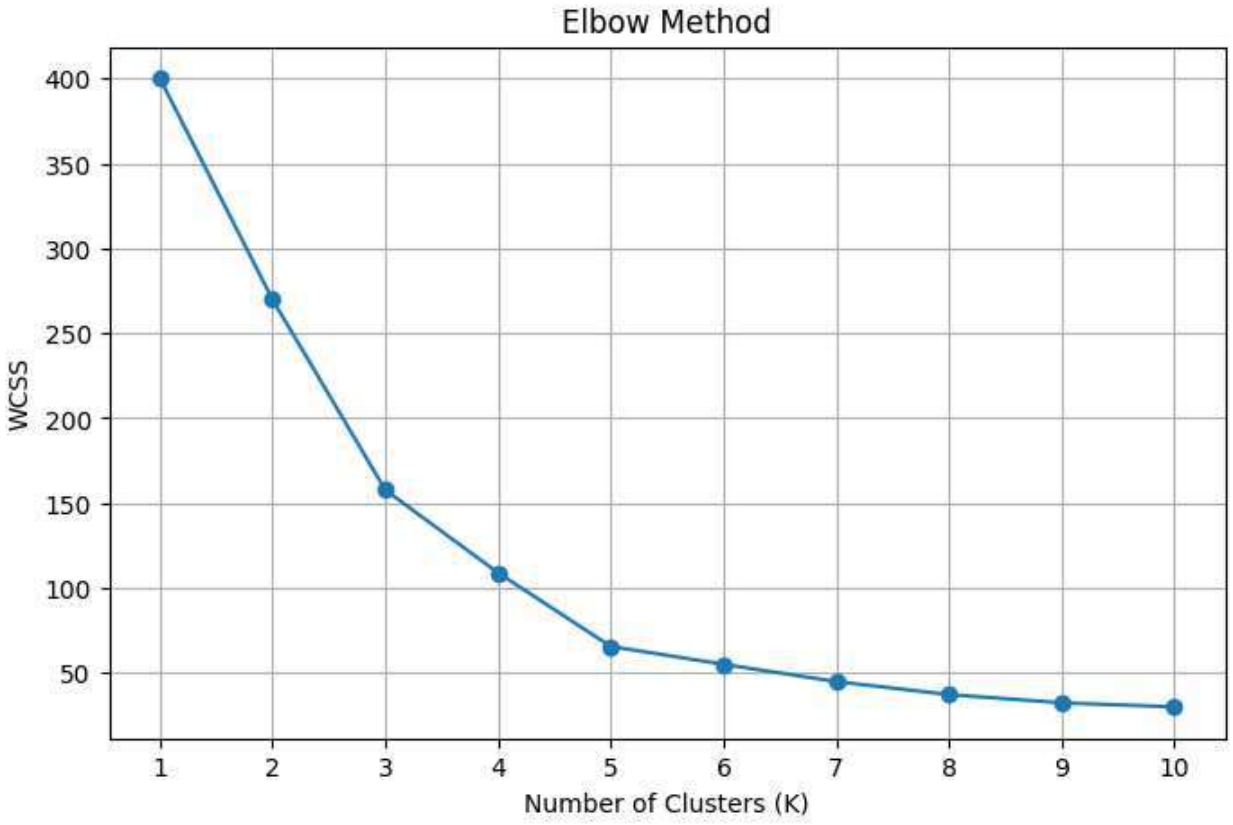
CustomerID	Age	Annual Income (k
$)SpendingScore(1 - 100)count200.000000200.000000200.000000200.000000mean100.50000038.85000060.560$		
) Spending Score (1-100)		
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40





Standardized features (first 5 rows):

	Annual Income (k$)	Spending Score (1-100)
0	-1.738999	-0.434801
1	-1.738999	1.195704
2	-1.700830	-1.715913
3	-1.700830	1.040418
4	-1.662660	-0.395980



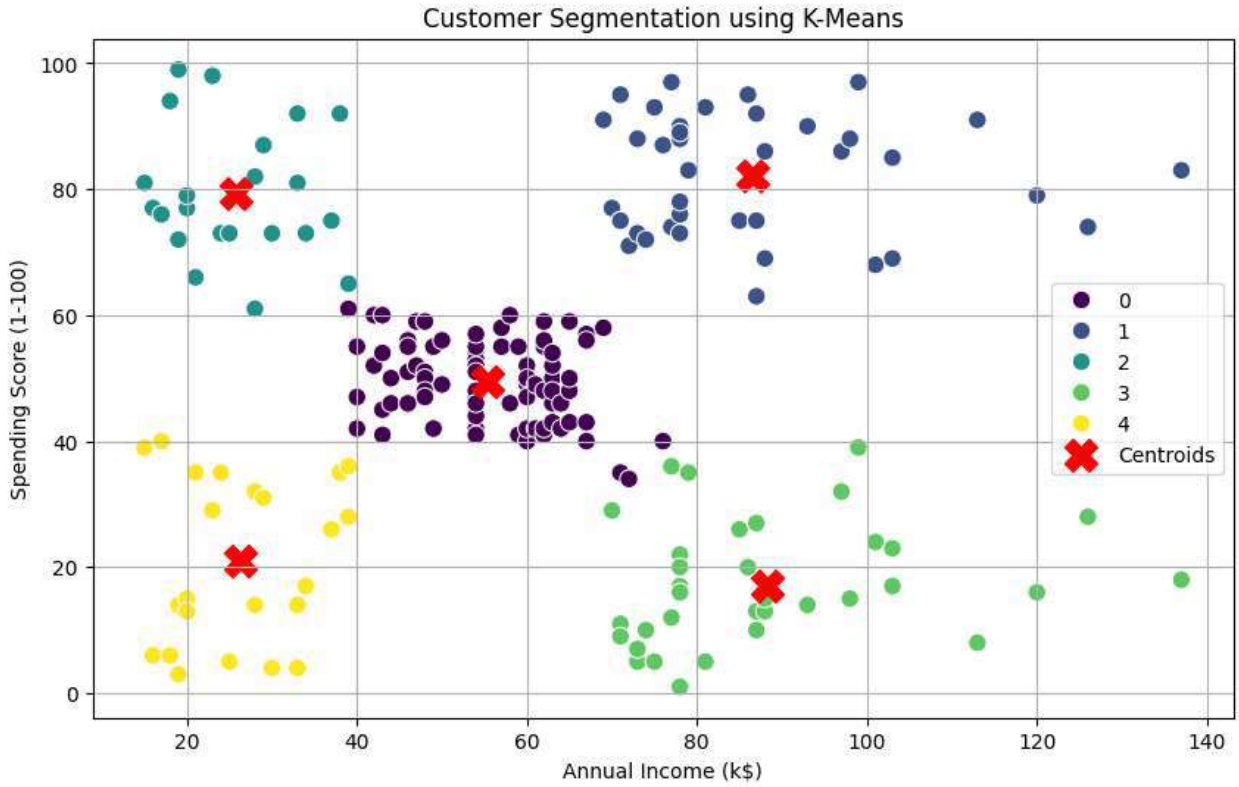
Cluster labels assigned successfully.

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)	\
0	1	Male	19	15	39	
1	2	Male	21	15	81	
2	3	Female	20	16	6	
3	4	Female	23	16	77	
4	5	Female	31	17	40	
5	6	Female	22	17	76	
6	7	Female	35	18	6	
7	8	Female	23	18	94	
8	9	Male	64	19	3	
9	10	Female	30	19	72	

	Cluster
0	4
1	2
2	4
3	2
4	4
5	2
6	4
7	2
8	4

9 2

Cluster centroid coordinates:
Cluster 0: Annual Income = 55.30 k, $SpendingScore = 49.52 Cluster 1: AnnualIncome = 86.54k$, Spending Score = 82.13
Cluster 2: Annual Income = 25.73 k, $SpendingScore = 79.36 Cluster 3: AnnualIncome = 88.20k$, Spending Score = 17.11
Cluster 4: Annual Income = 26.30 k$, Spending Score = 20.91



Number of customers in each cluster:
Cluster
0 81
1 39
2 22
3 35
4 23
Name: count, dtype: int64

Cluster Analysis:

Cluster	Customer_Count	Average_Annual_Income	Average_Spending_Score
0	81	55.30	49.52
1	39	86.54	82.13

2	22	25.73	79.36
3	35	88.20	17.11
4	23	26.30	20.91

MARKETING RECOMMENDATIONS:
Cluster 0: Use targeted discounts and personalized product recommendations.
Cluster 1: Reward loyal customers with premium offers and loyalty benefits.
Cluster 2: Offer affordable products, discounts, and bundle deals.
Cluster 3: Provide personalized promotions to encourage purchases.
Cluster 4: Use budget-friendly offers to attract customer interest.

Clustered dataset saved as Mall_Customers_Clustered.csv